



Unity를 이용한 2D 게임프로그래밍

Lecture 4

User Interface로 퀴즈 게임 만들기

강영민

동명대학교 게임학부

목차

- 1 학습목표
- 2 Unity UI 개요 및 Canvas
- 3 UI 요소 구성 – 이미지, 텍스트, 버튼
- 4 스크립터블 객체 (Scriptable Objects)
- 5 퀴즈 게임 스크립트 구현
- 6 타이머 구현
- 7 다중 질문 관리 및 진행 바
- 8 도전 과제
- 9 전체 아키텍처 정리

학습목표

이번 강의에서 배울 내용

- **User Interface(UI)**에 대해 이해한다.
- **캔버스(Canvas)**의 개념을 이해하고
캔버스 공간과 월드(카메라) 공간의 차이를 이해한다.
- 퀴즈 게임을 통해 사용자 인터페이스를 제어할 수 있다.

완성 목표

타이머, 다중 문제, 정답 확인, 점수 표시, 진행 바를 갖춘
퀴즈 게임 완성

프로젝트 생성

프로젝트 생성 설정

- **2D Core** 템플릿으로 생성
- URP (Universal Render Pipeline) 사용
- Main Camera: **Orthographic** 투영

기본 씬 구성

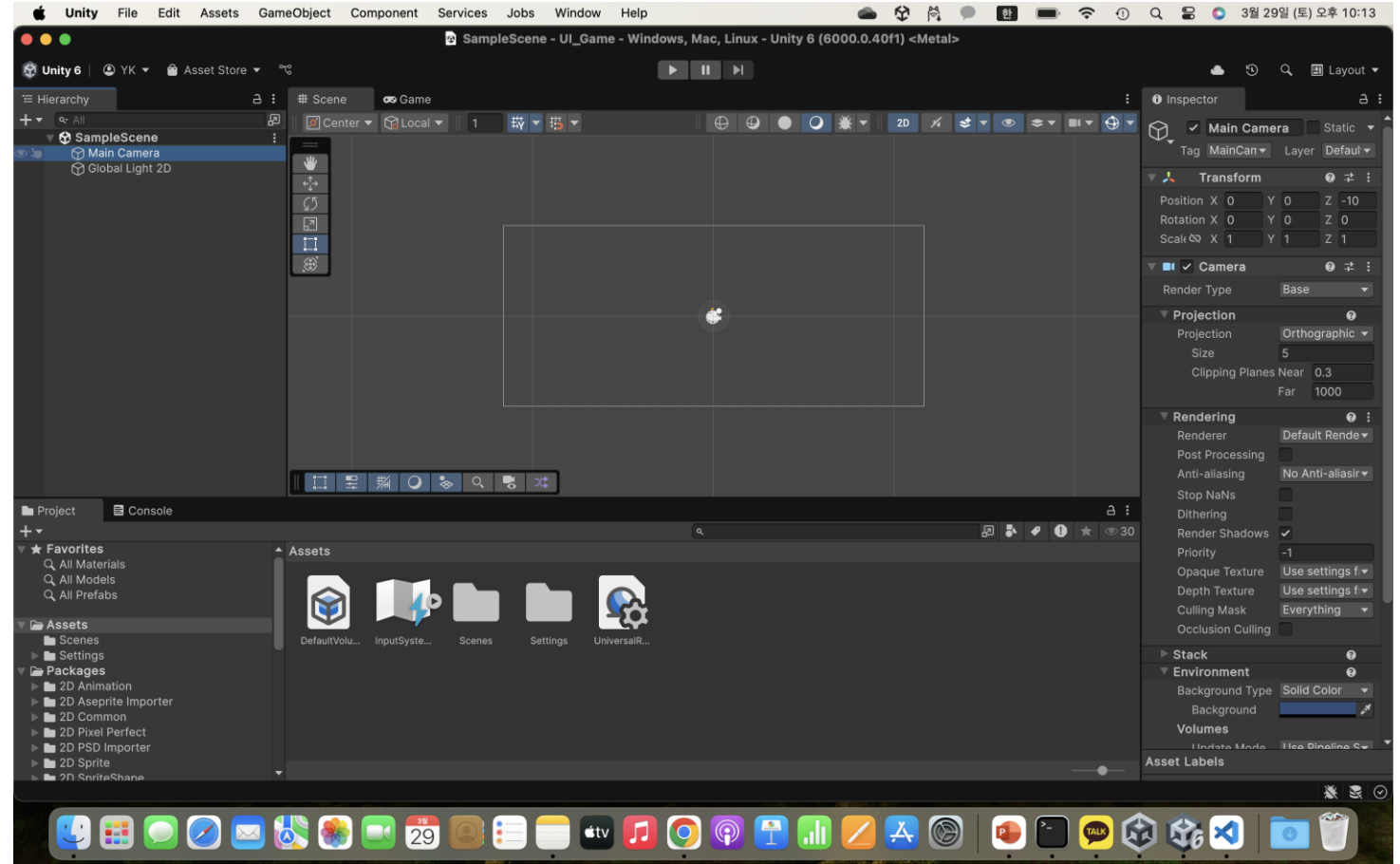
- **Main Camera** – 게임 화면 렌더링
- **Global Light 2D** – 2D 조명

Hierarchy 초기 구성

- ▽ SampleScene
 - Main Camera
 - Global Light 2D

프로젝트를 생성하자

- 2차원 게임으로 생성하자



캔버스(Canvas) 개념

Canvas란?

- 모든 UI 요소의 부모 오브젝트
- 캔버스는 스크린 공간(Screen Space)에 존재
- 복수의 캔버스 설치 가능
- **Sort Order**로 렌더링 순서 결정

생성 방법

Hierarchy 우클릭 → UI → Canvas

주의

Canvas 생성 시 **EventSystem**도 자동 생성됨 (UI 입력 처리에 필수)

Canvas Inspector 주요 속성

Render Mode	Screen Space - Overlay
Sort Order	0 (기본)
Target Display	Display 1

Canvas Scaler

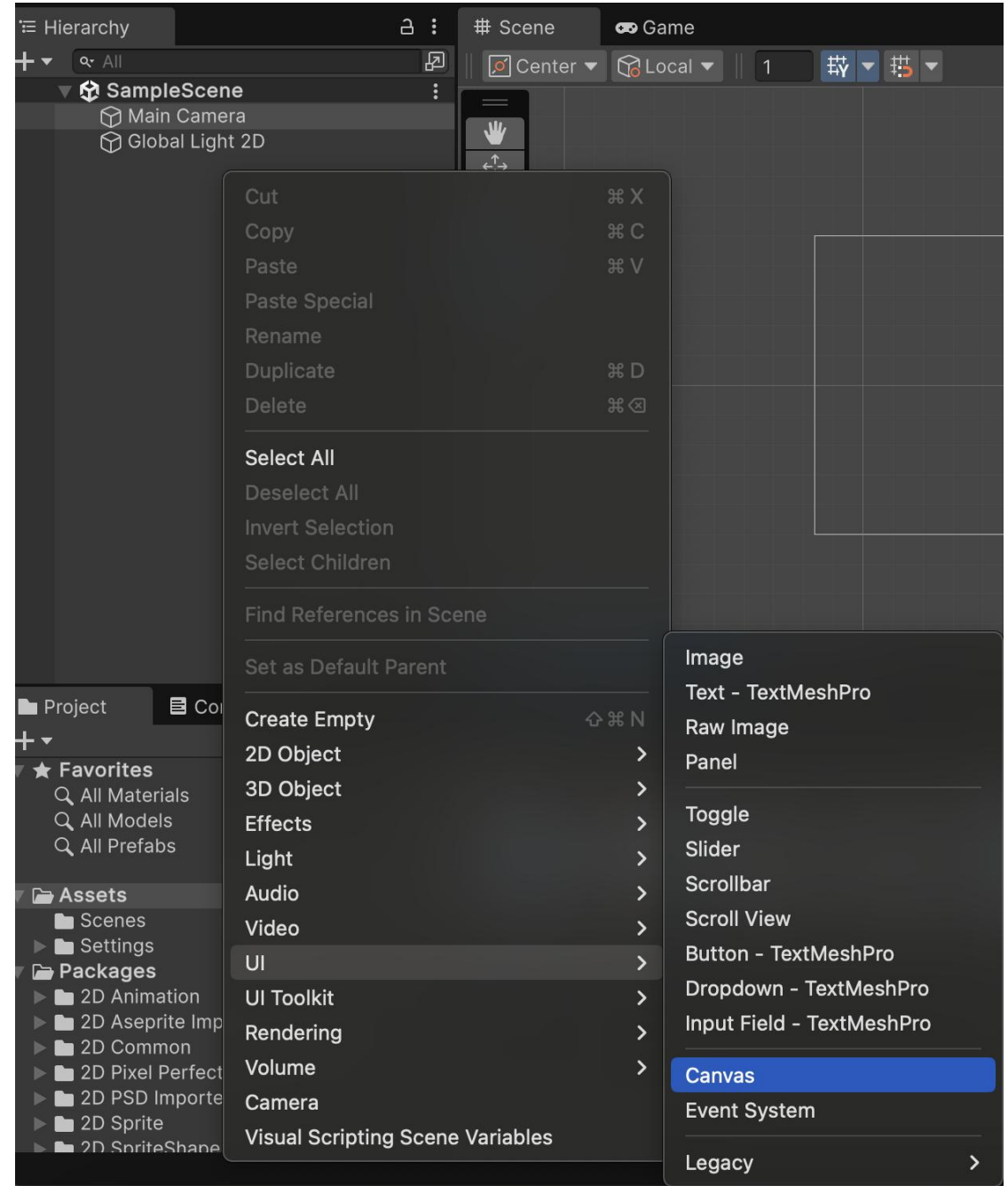
UI Scale Mode	Constant Pixel
Scale Factor	1

Graphic Raycaster

UI 클릭 이벤트 감지

캔버스(Canvas)

- Hierarchy
 - Create → UI → Canvas
- UI 요소들이 캔버스에 실림
 - 캔버스는 스크린 공간에 존재
- 복수의 캔버스도 설치 가능



카메라 공간 vs. 캔버스 공간

월드(카메라) 공간

- **Transform** 컴포넌트 사용
- 단위: Unity 월드 단위 (미터)
- 카메라가 비추는 범위만 게임 화면에 표시
- 예: 원(Circle) 스프라이트

캔버스(UI) 공간

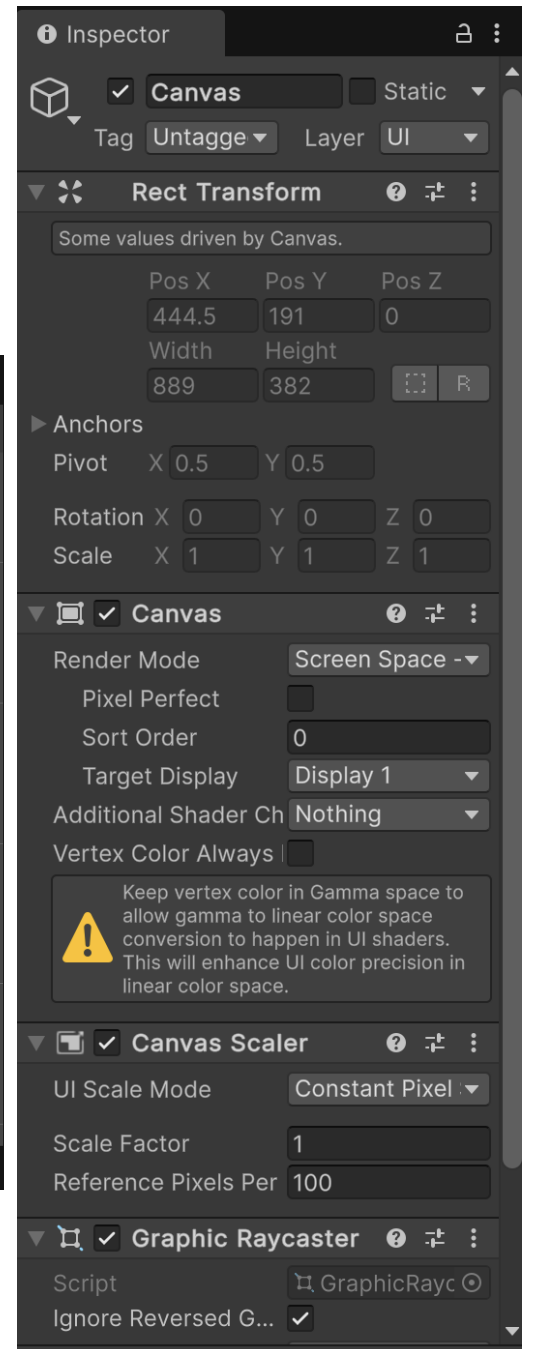
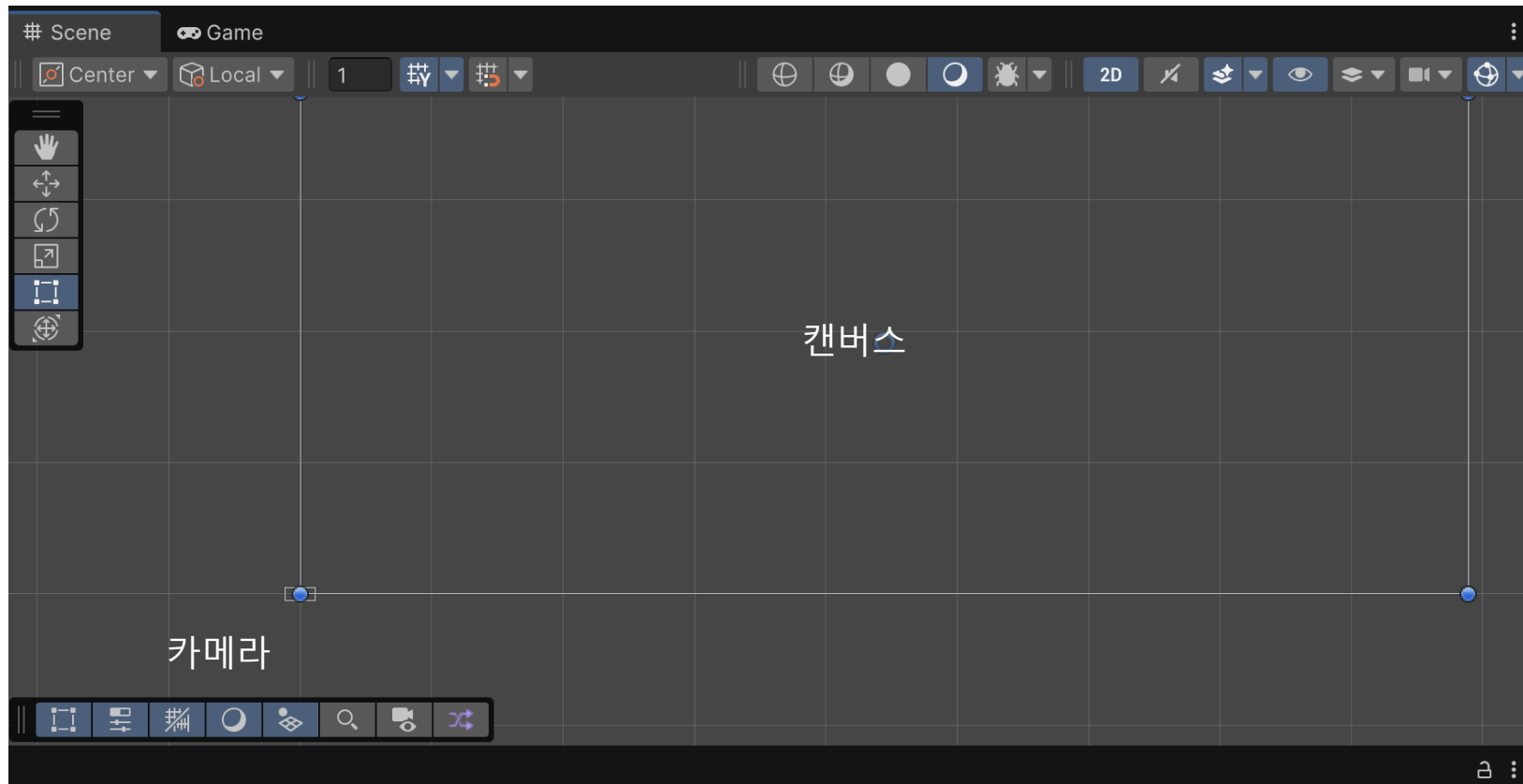
- **Rect Transform** 컴포넌트 사용
- 단위: 픽셀(px)
- 항상 화면 위에 오버레이
- Anchor, Pivot으로 위치 제어
- 예: Image, Text, Button

핵심 개념

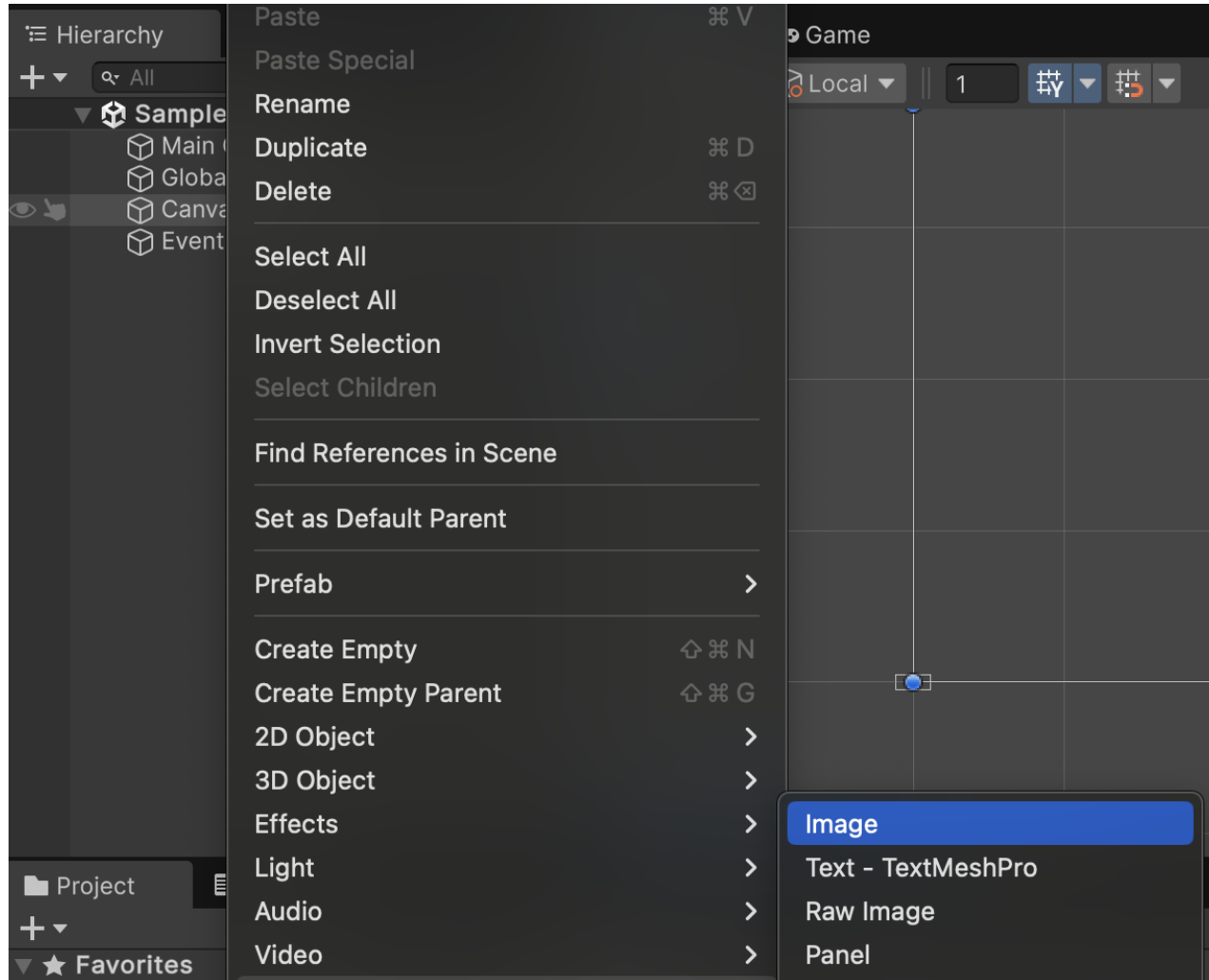
게임 화면(Game View)에는 캔버스 공간 + 카메라 공간이 함께 보인다.
캔버스는 카메라의 앞(화면 위)에 렌더링된다.

카메라 공간과 캔버스 공간

캔버스는
일반적 게임 객체의
변환과 다른 특성을 가짐

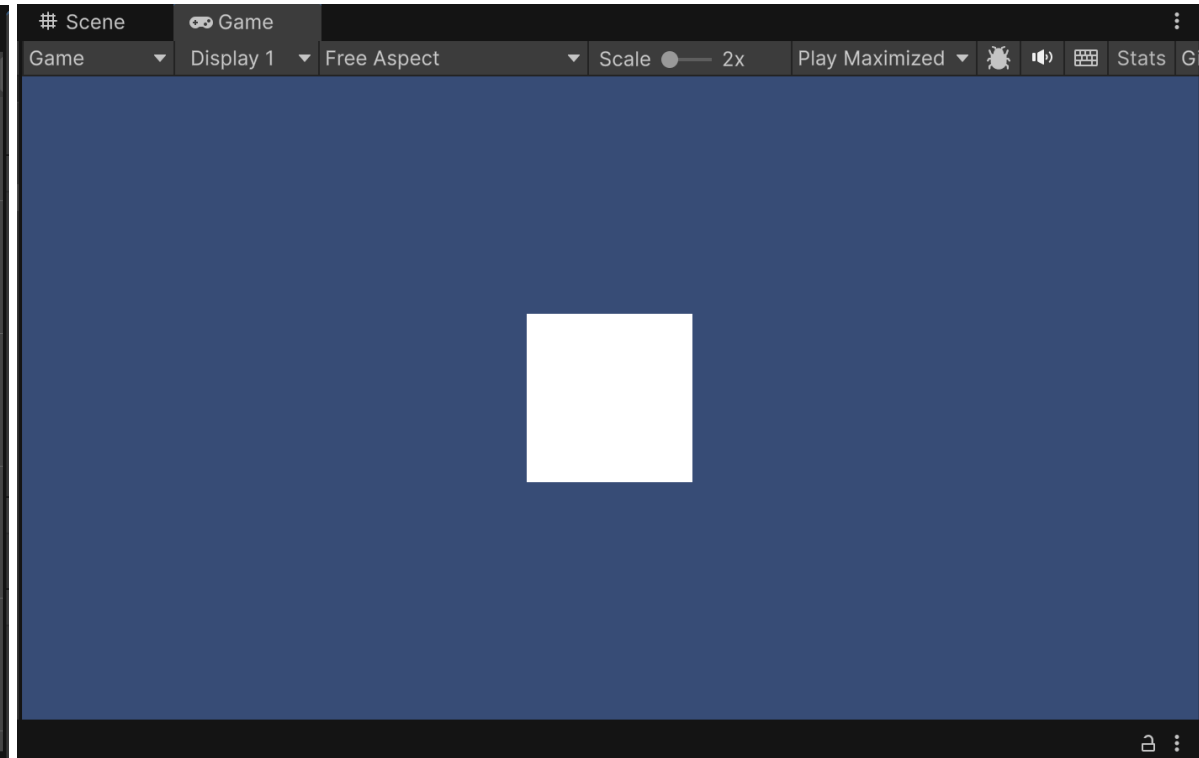
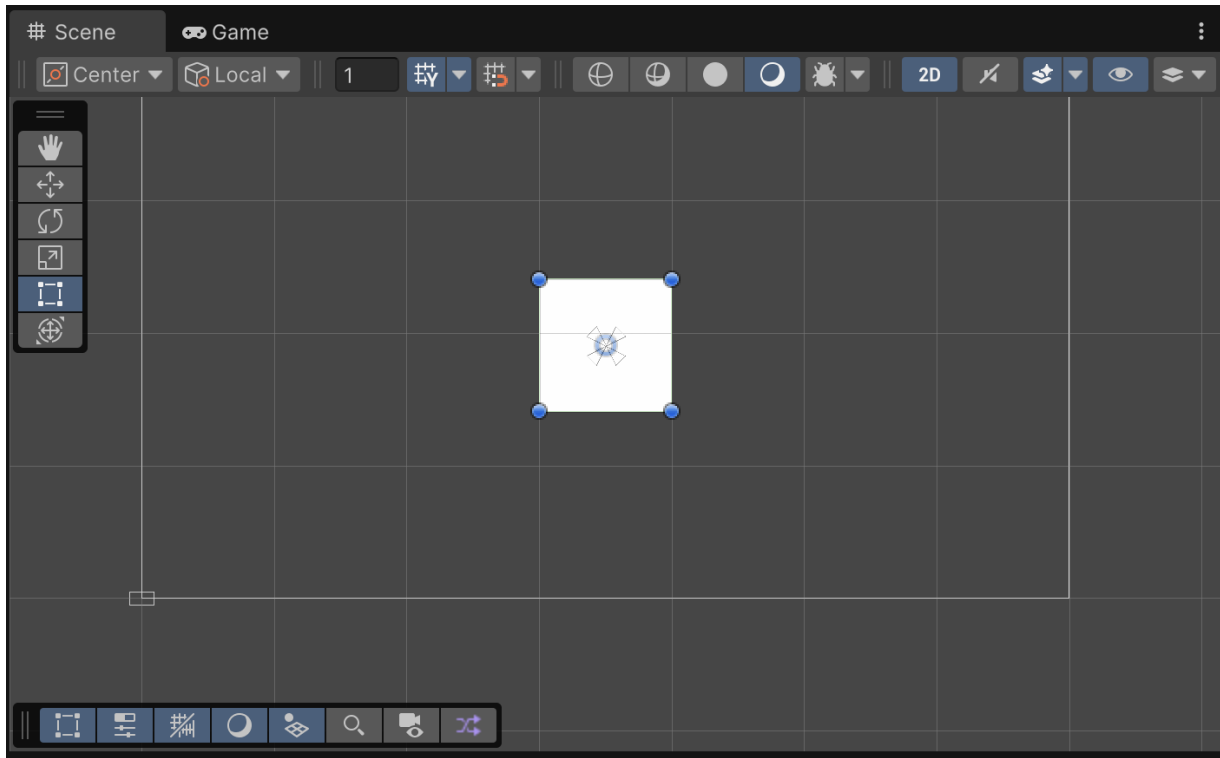


캔버스의 자식 객체로 UI Image 생성

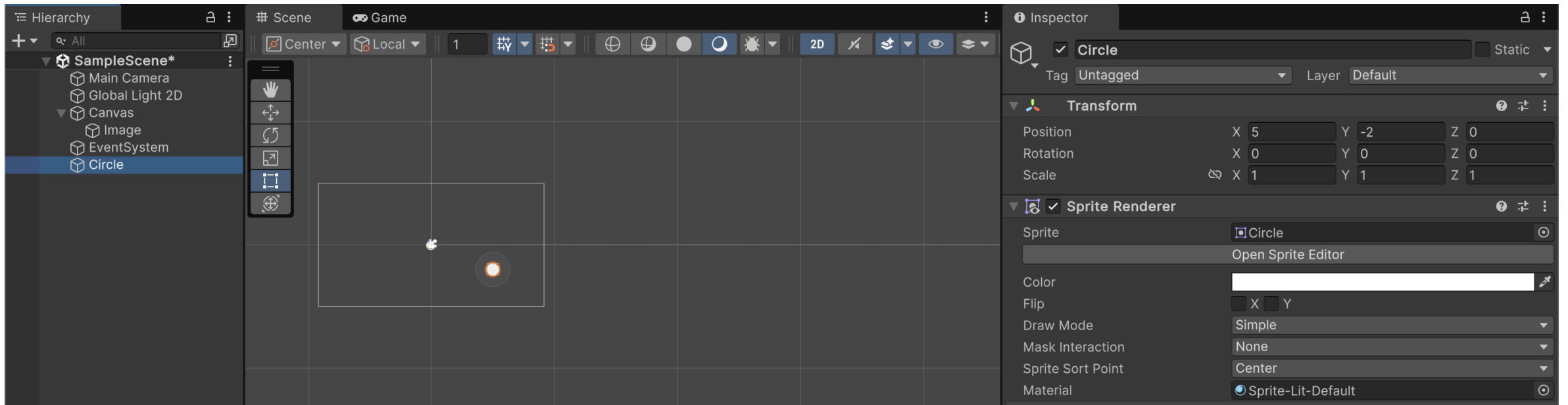


캔버스 공간에 이미지 존재 / 카메라 공간 밖

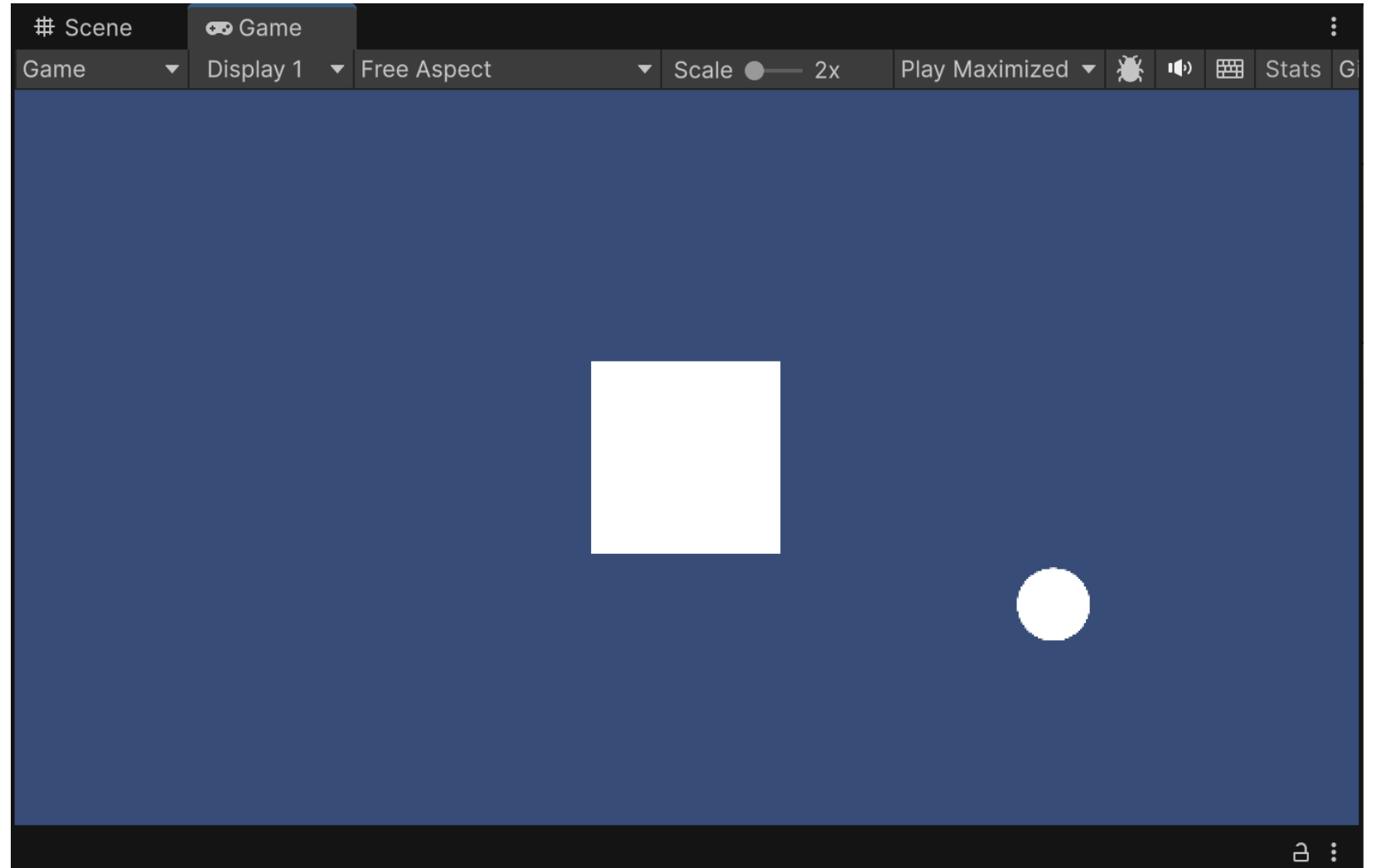
- 게임 화면을 보면 캔버스 공간이 보임



카메라 공간에 원을 하나 넣어 보자



게임 화면은 캔버스 공간과 카메라 공간이 같이 보임



Rect Transform – Anchor와 Pivot

Anchor (앵커)

- UI 요소의 기준점을 부모에 상대적으로 정의
- 화면 크기가 변해도 상대 위치 유지
- **Anchor Presets**: 클릭으로 빠른 설정
 - Shift + 선택: Pivot도 함께 설정
 - Alt + 선택: Position도 함께 설정

Pivot (피벗)

- UI 요소 자신의 회전/스케일 기준점
- (0,0) = 좌하단, (0.5, 0.5) = 중앙, (1,1) = 우상단

Stretch 앵커

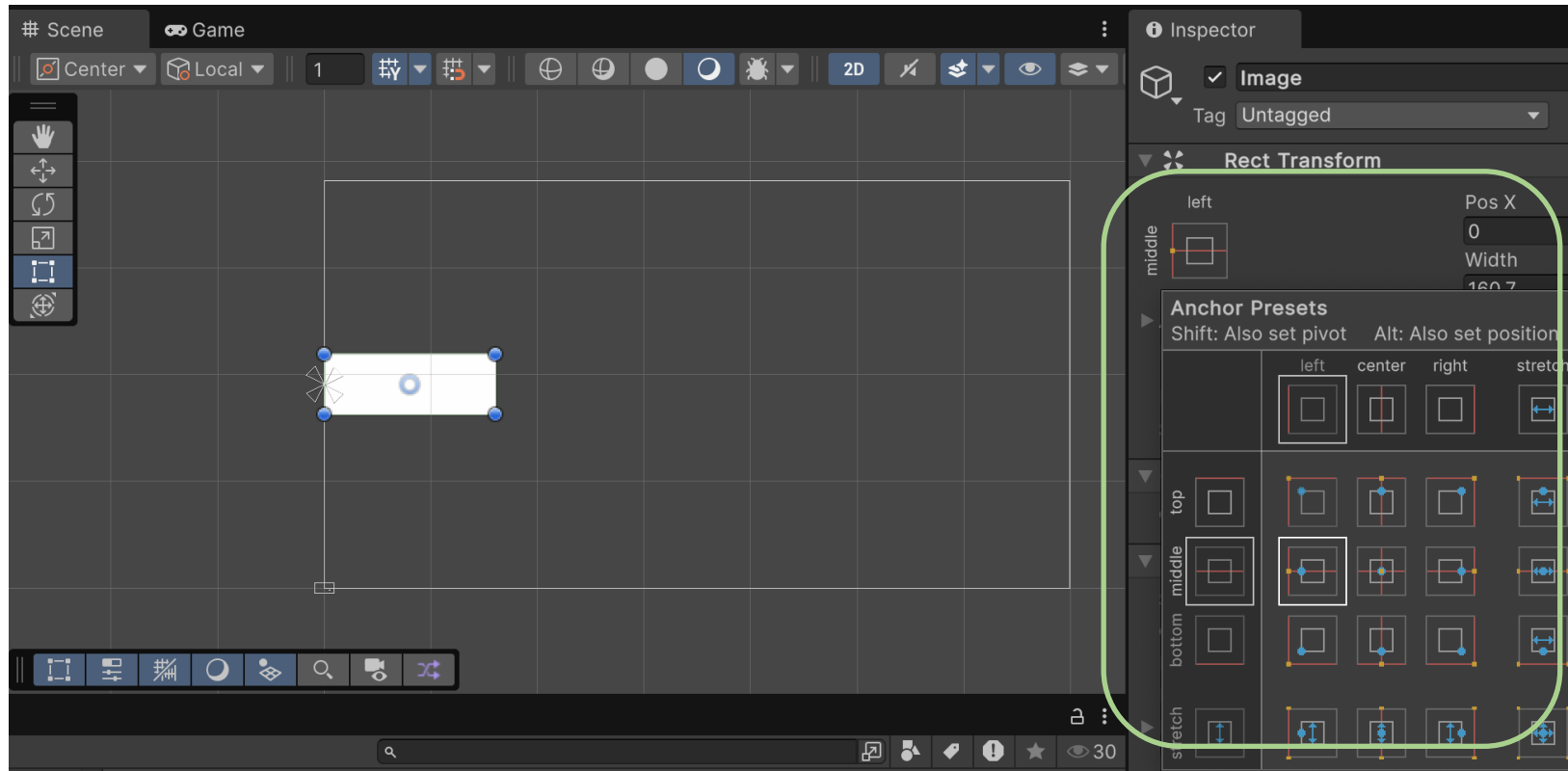
stretch 앵커를 사용하면
Left/Right/Top/Bottom 값으로
부모 크기에 맞춰 자동 늘어남

Canvas vs. Transform

Canvas의 Rect Transform은
일반 GameObject의 Transform과
다른 속성을 가짐 (Width, Height)

Anchor와 Pivot 테스트해 보기

- Anchor Preset + (Shift / Alt)



UI Image 생성 및 스프라이트 설정

UI Image 생성

Canvas 우클릭 → UI → Image

Image 컴포넌트 주요 속성:

- Source Image: 스프라이트 텍스처
- Color: 색상 / 알파(투명도)
- Image Type:
 - **Simple**: 단순 표시
 - **Sliced**: 9-슬라이스 (테두리 보존)
 - **Filled**: 채우기 (타이머 등)

9-슬라이스 (Sliced) 사용법

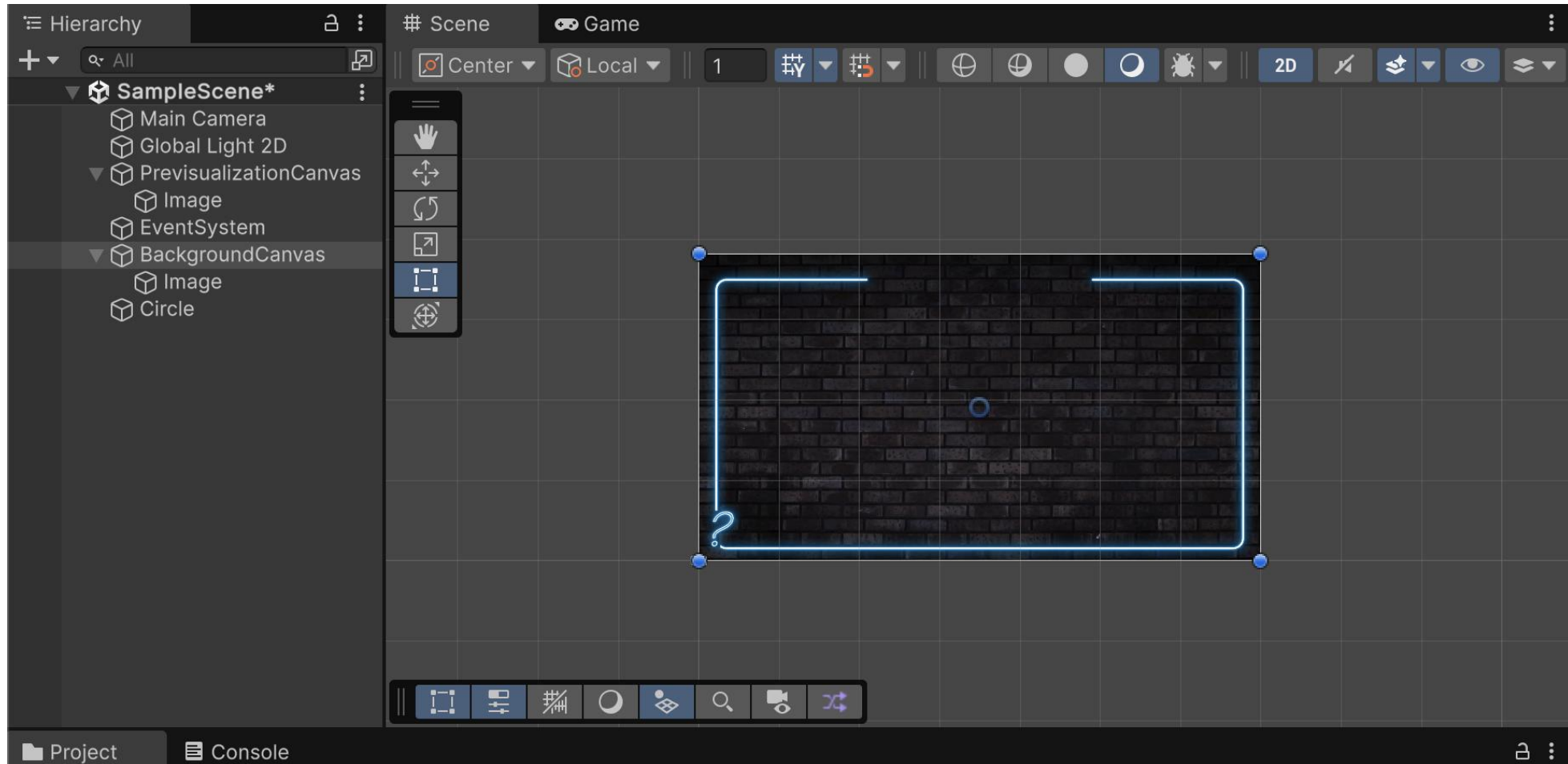
- 1 Image Type을 **Sliced**로 변경
- 2 해당 스프라이트를 Project에서 선택
- 3 **Sprite Editor** 실행
- 4 Border(L/R/T/B) 픽셀 값 지정
- 5 Apply 클릭

크기를 늘려도 테두리가 깨지지 않음!

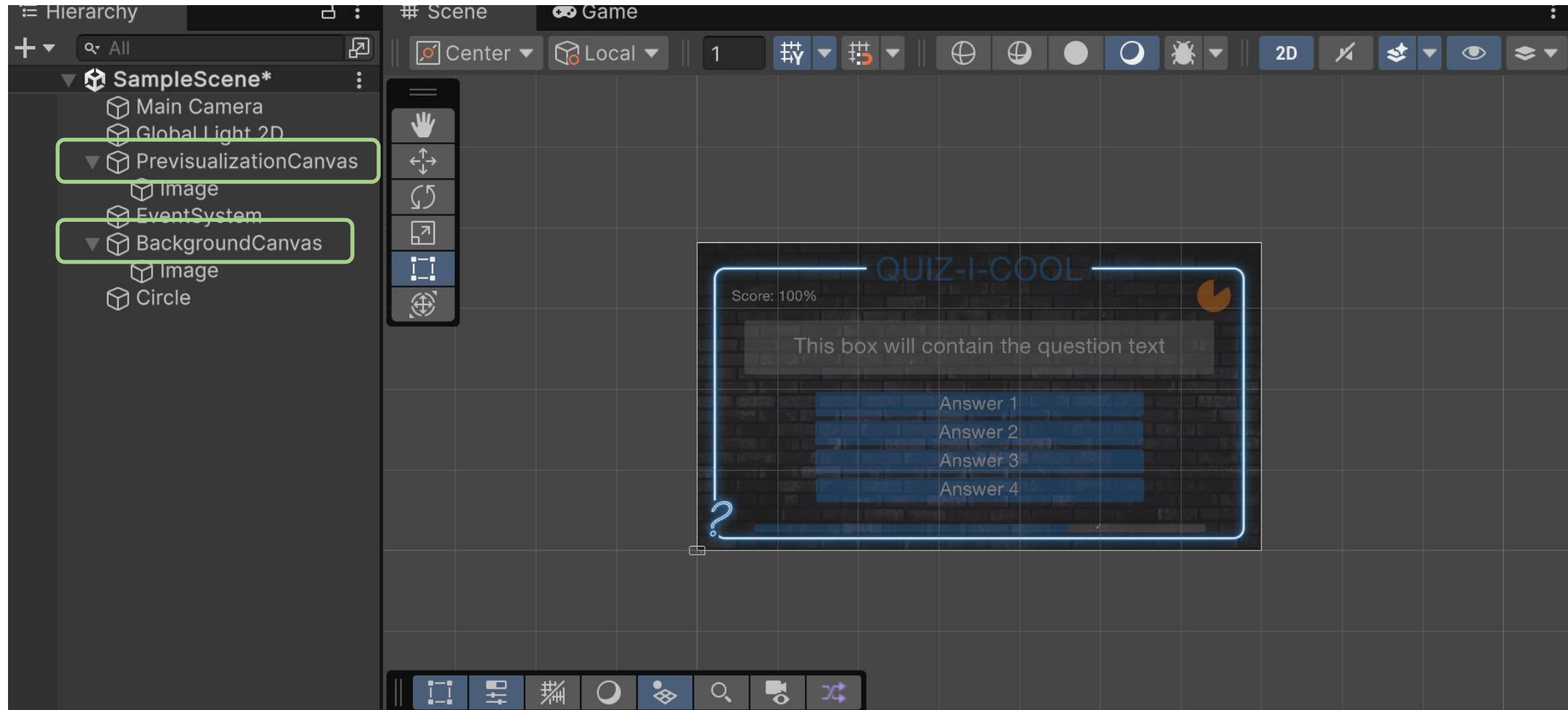
UI 캔버스에 놓인 이미지에 스프라이트 적용/알파 변경



두 번째 캔버스 생성 (Background)



두 캔버스가 겹쳐서 보이게 만들어 보라



TextMesh Pro (TMP) 설정

TMP란?

Unity의 고품질 텍스트 렌더링 시스템
생성: Canvas 우클릭 →
UI → **Text - TextMeshPro**

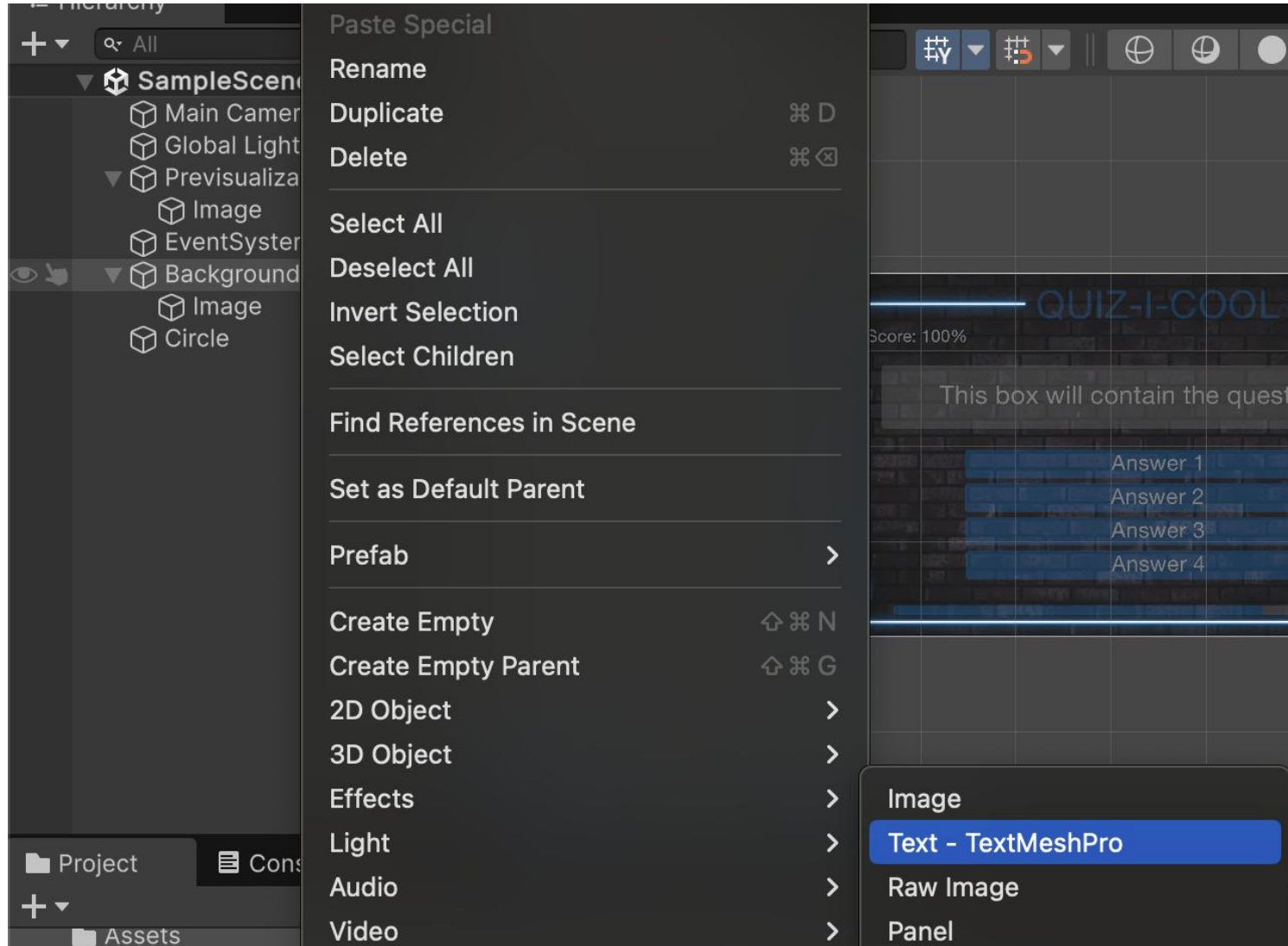
첫 사용 시 TMP Importer

처음 TMP를 사용하면 팝업 창이 뜬
Import TMP Essentials 클릭 필수!
(선택) Import TMP Examples & Extras

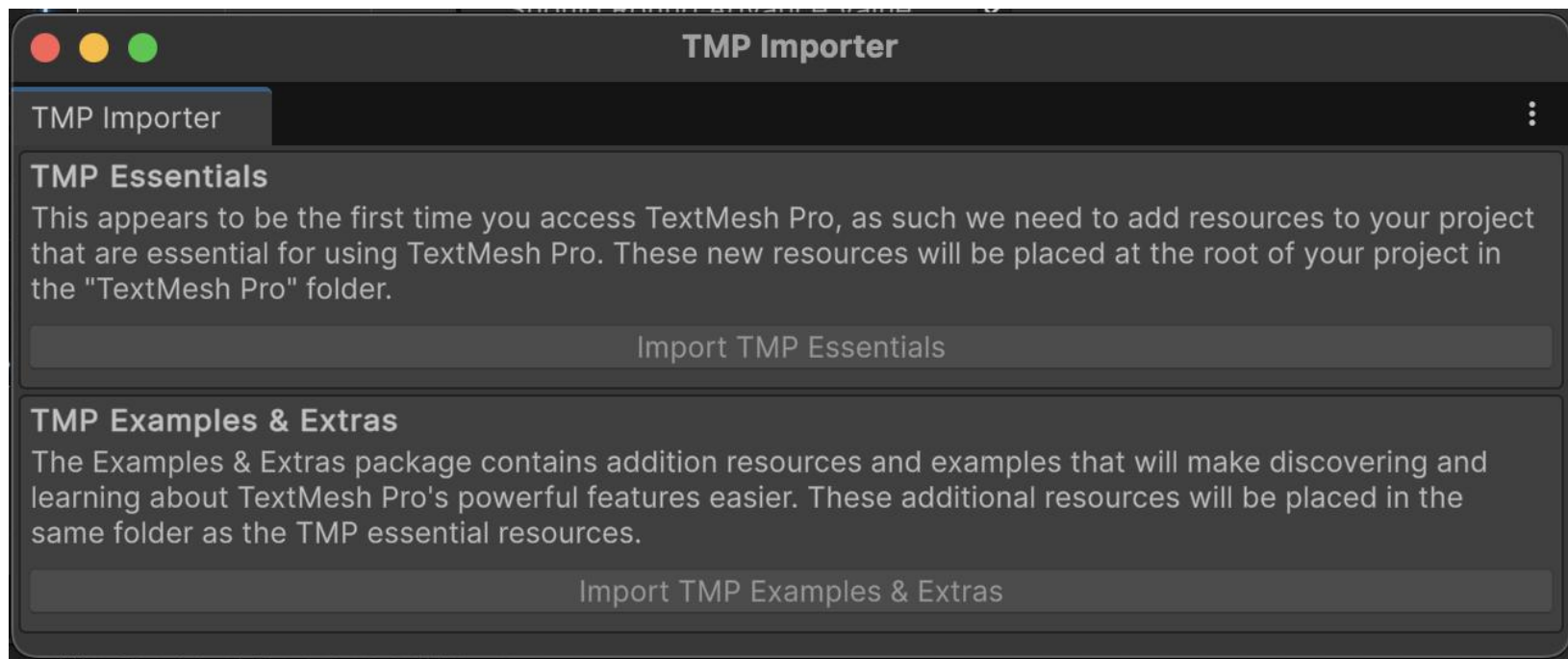
한국어 폰트 Asset 생성

- 1 폰트 파일(.ttf)을 Assets에 추가
- 2 Window → TextMeshPro → **Font Asset Creator**
- 3 Source Font 선택
- 4 Character Set을 **Custom Range**로
한국어 유니코드 범위 설정
- 5 **Generate Font Atlas**
- 6 **Save** 클릭

배경 캔버스에 텍스트 메시 프로를 추가한다

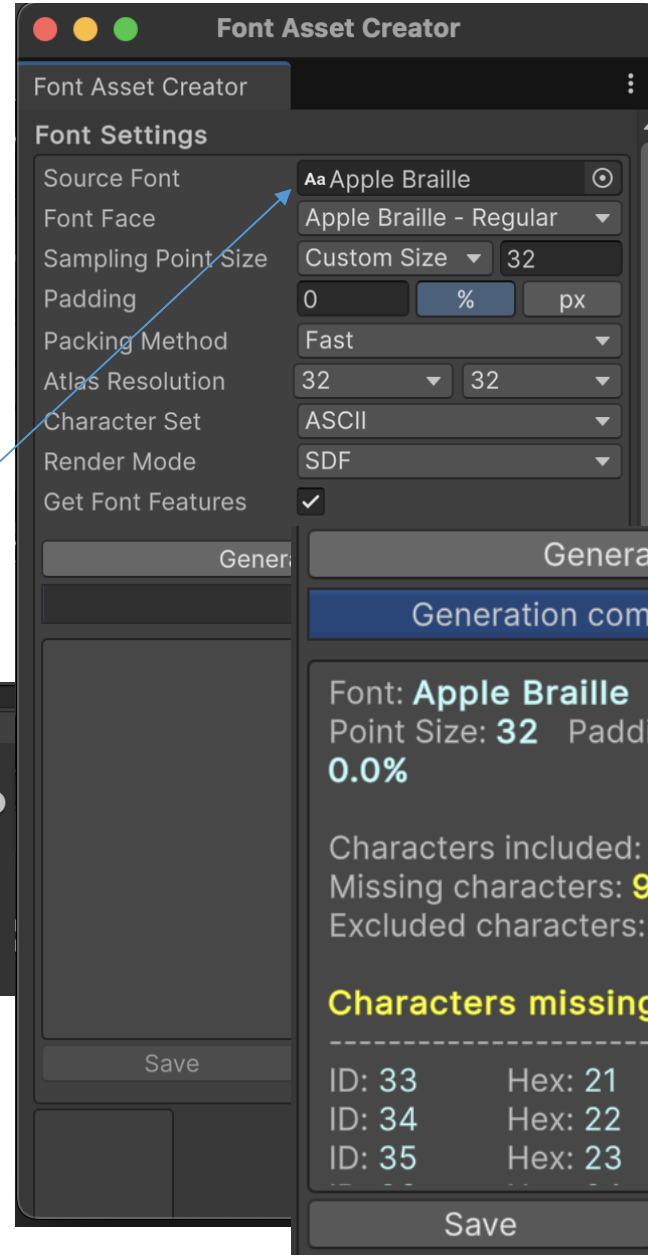
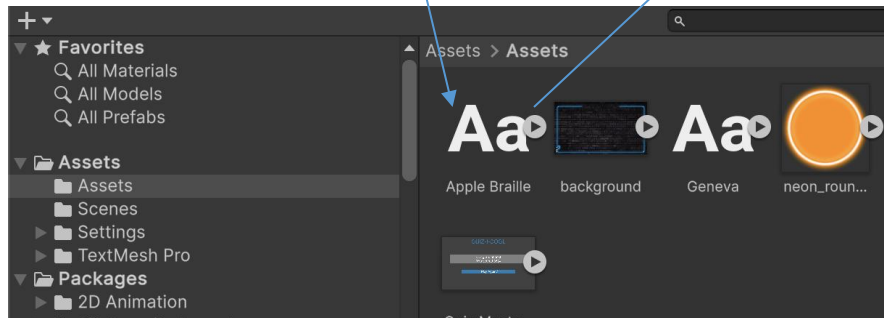


Text Mesh Pro를 처음 사용할 경우 필요한 자원 가져오기



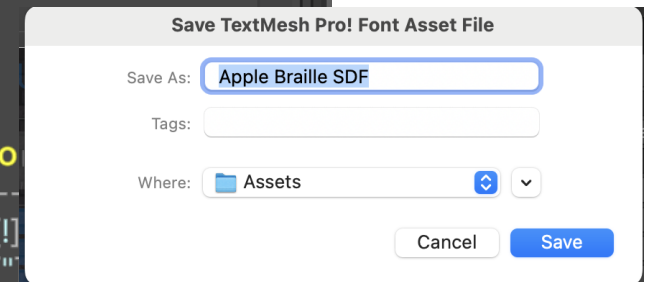
폰트 Asset 생성 필요

Font file

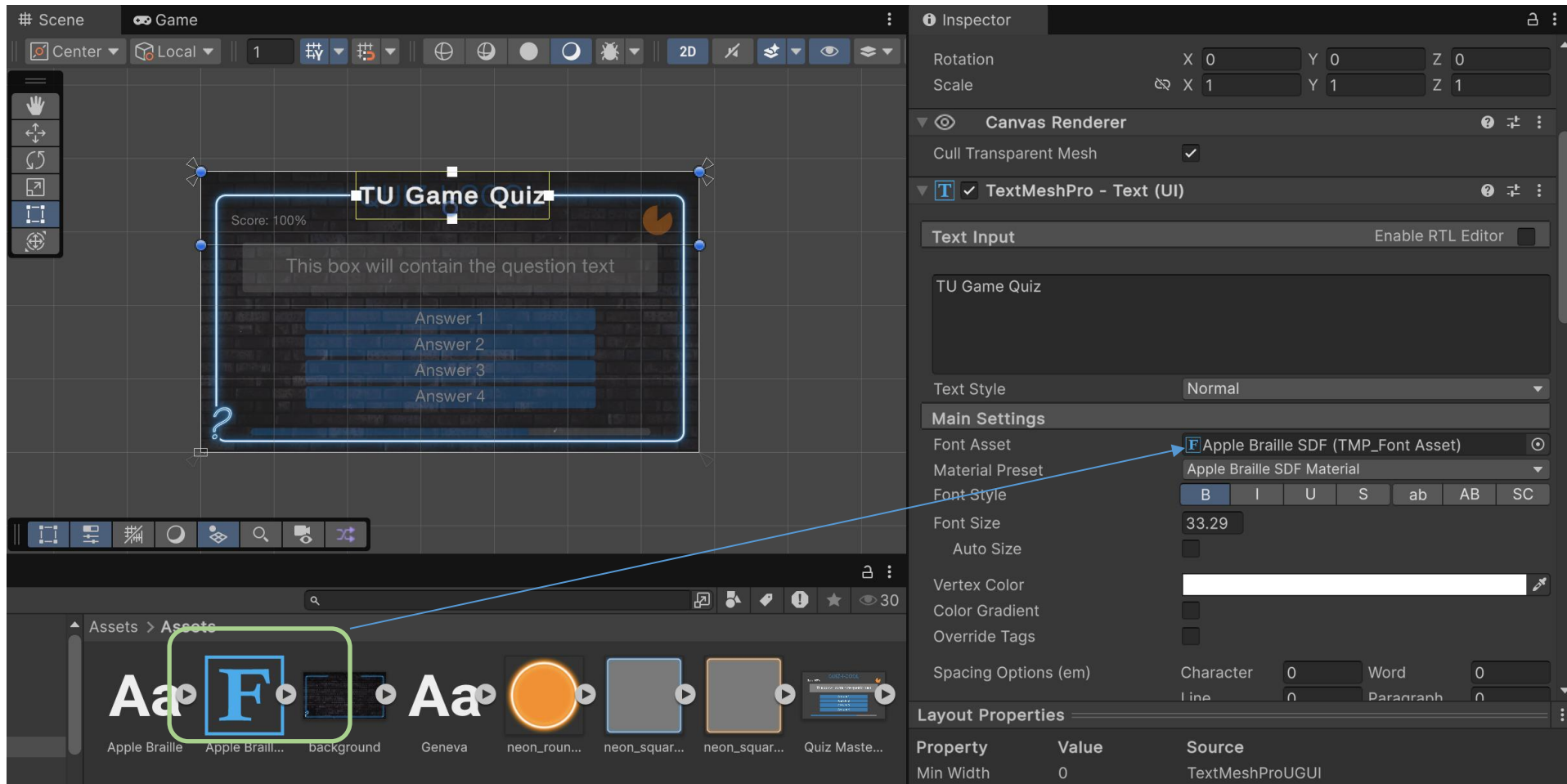


한국어 폰트 Asset 생성

- 1 폰트 파일(.ttf)을 Assets에 추가
- 2 Window → TextMeshPro → **Font Asset Creator**
- 3 Source Font 선택
- 4 Character Set을 **Custom Range**로 한국어 유니코드 범위 설정
- 5 **Generate Font Atlas**
- 6 **Save** 클릭



폰트를 적용하여 TMP 객체 다루기



버튼(Button) 구성

Button 생성

Canvas 우클릭 → UI →
Button - TextMeshPro

Button 컴포넌트 속성:

- Interactable: 클릭 가능 여부
- Transition: 상태별 색상 변화
- On Click(): 클릭 이벤트 핸들러

Vertical Layout Group

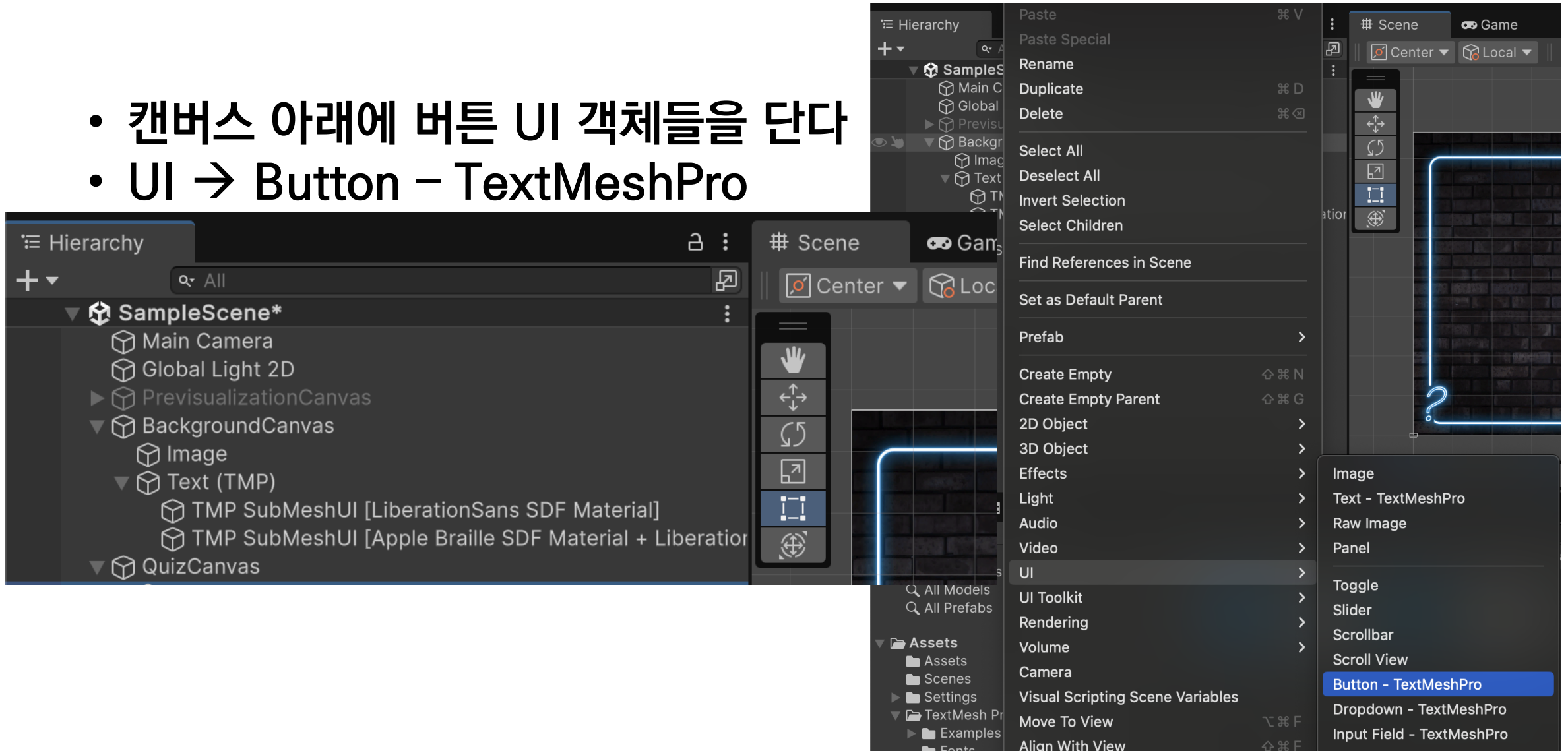
버튼을 담는 빈 오브젝트에
Vertical Layout Group 컴포넌트 추가
⇒ 자식 버튼들이 자동으로 수직 배치

Hierarchy 구성 예시

```
▽ QuizCanvas
  Quiz Text (TMP)
  ▽ AnswerButtonGroup
    Answer Button (0)
    Answer Button (1)
    Answer Button (2)
    Answer Button (3)
  TimerImage
  Timer
  ▽ BackgroundCanvas
    Image
    Text (TMP)
```

Canvas 추가하고 버튼 달아보기

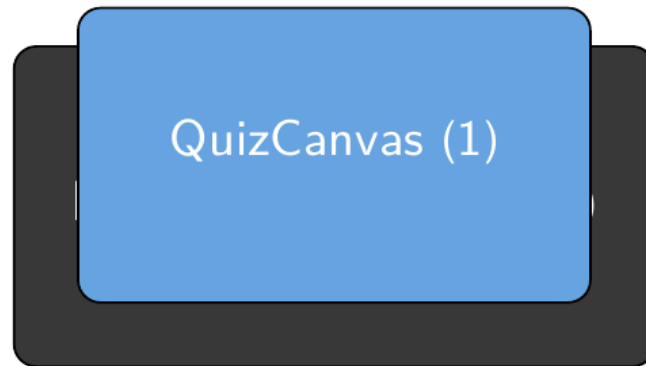
- 캔버스 아래에 버튼 UI 객체들을 단다
- UI → Button – TextMeshPro



Sort Order – 캔버스 레이아웃

Sort Order 개념

- 숫자가 클수록 앞에 렌더링
- 기본값: 0
- 예시:
 - BackgroundCanvas: Sort Order = 0
 - QuizCanvas: Sort Order = 1



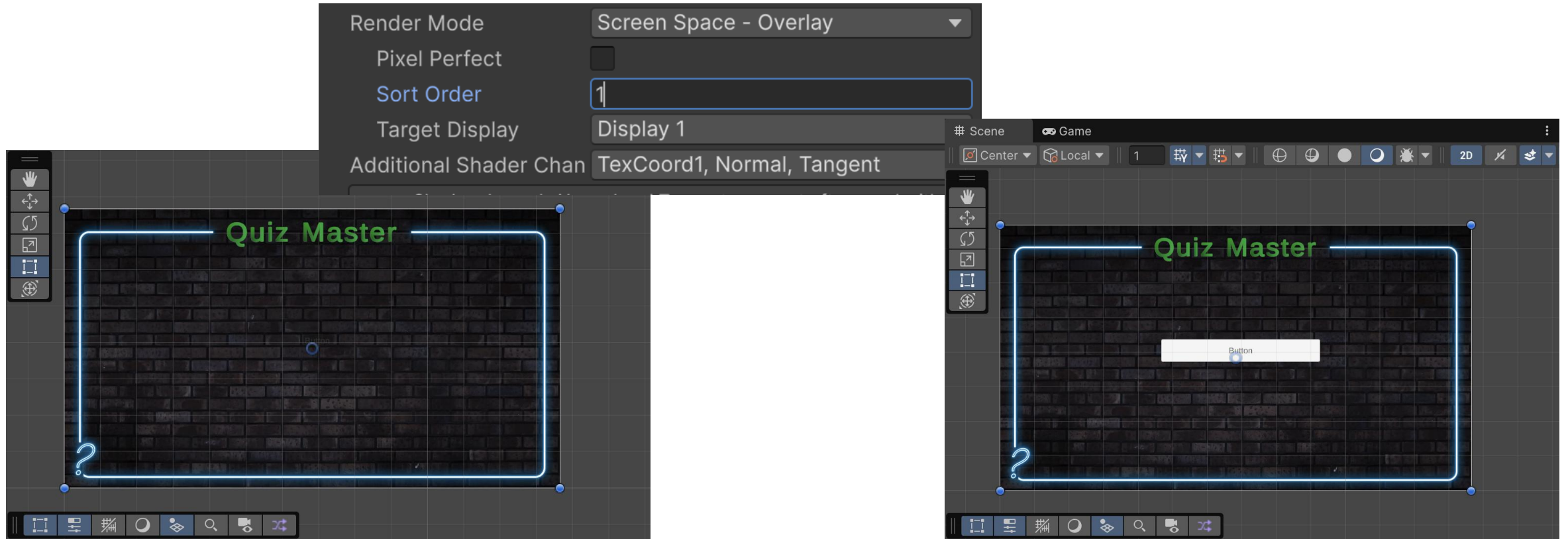
Sort Order가 높을수록 위에 표시

설정 위치

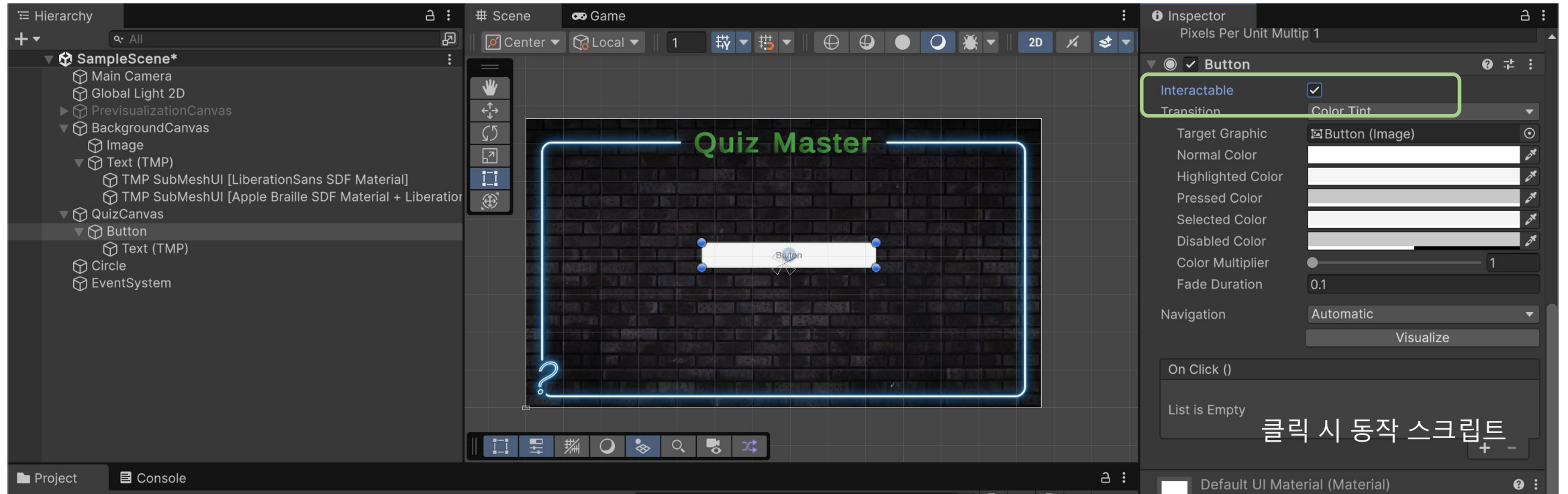
Canvas 선택 → Inspector →
Canvas 컴포넌트 → **Sort Order**

퀴즈 캔버스가 배경 캔버스 위에 보이게 하자

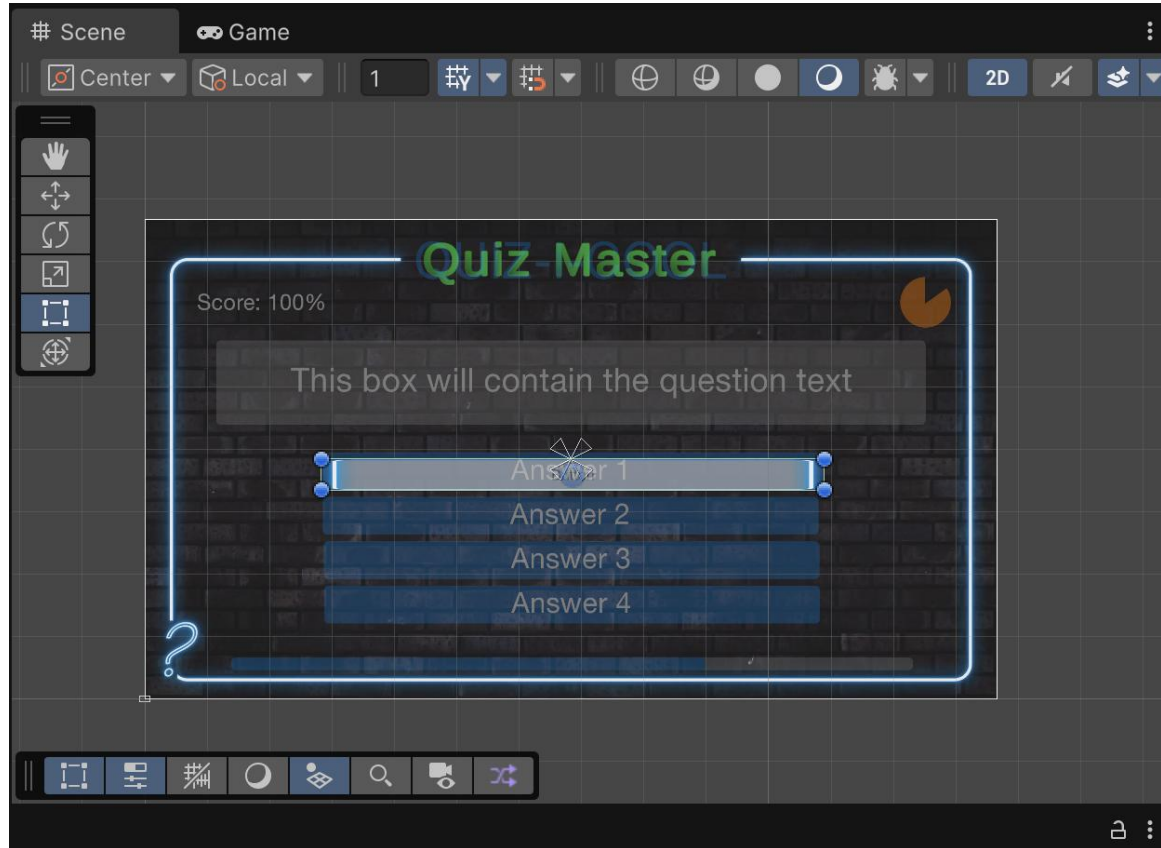
이것은 sort order를 통해 조정할 수 있다



버튼의 Interactability



Button에 스프라이트 적용



테두리가 이상하게 나타남

좌우로 크기를 늘렸기 때문

버튼에 적용된 이미지를 그리는 방식 변경

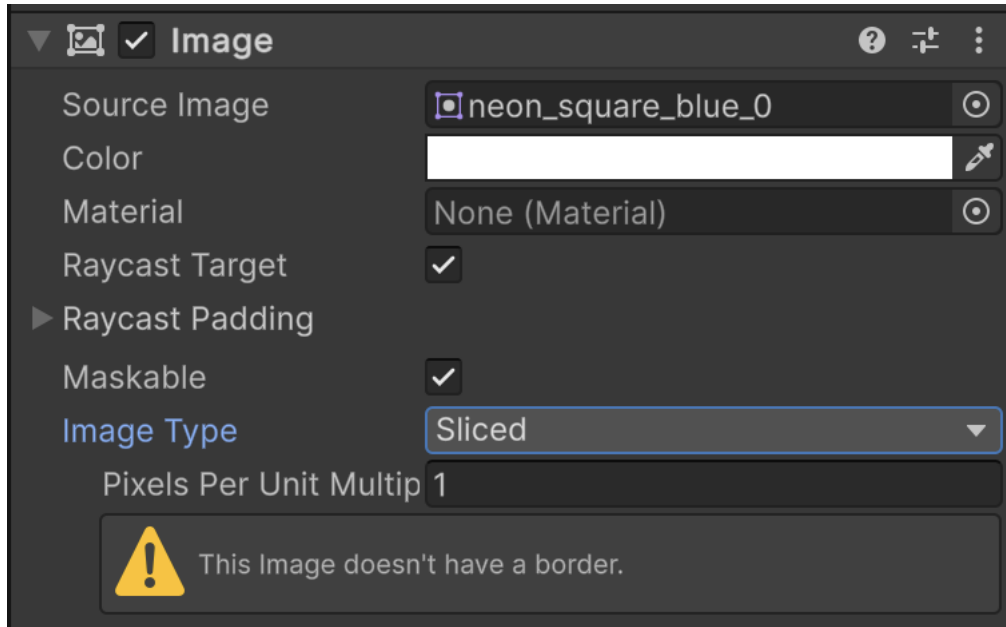
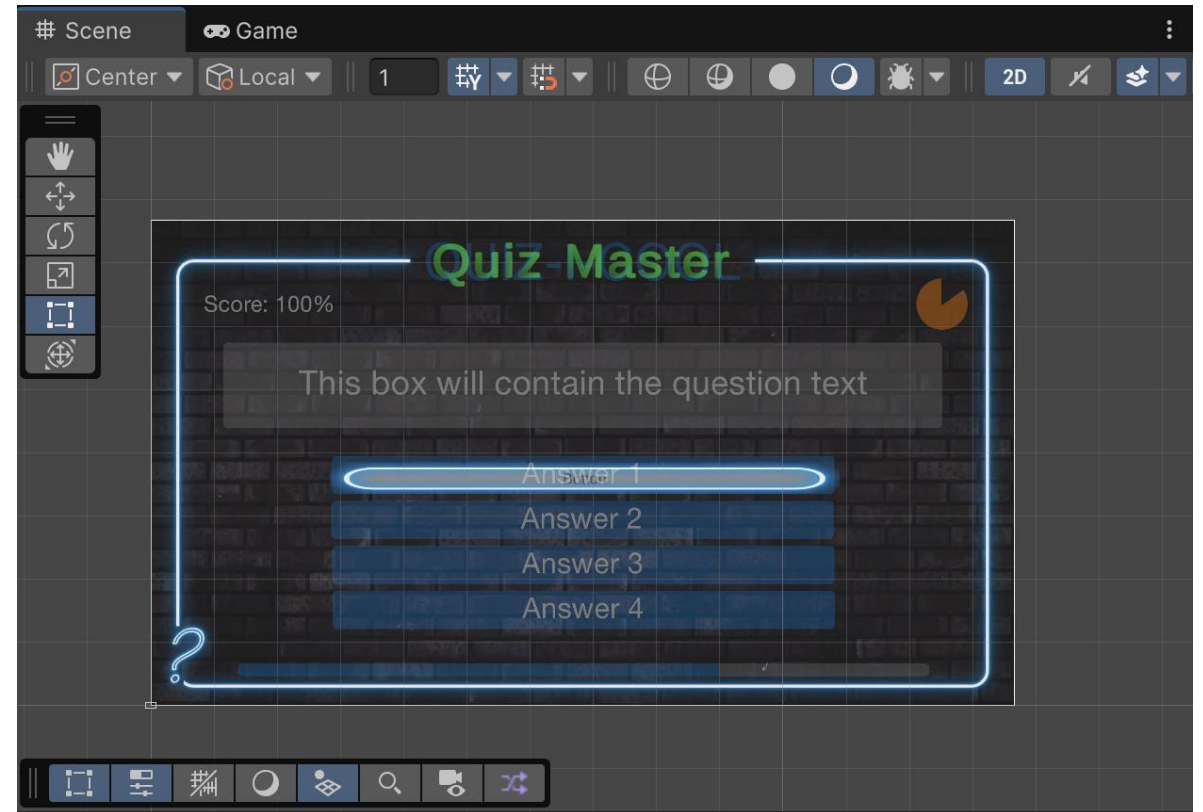
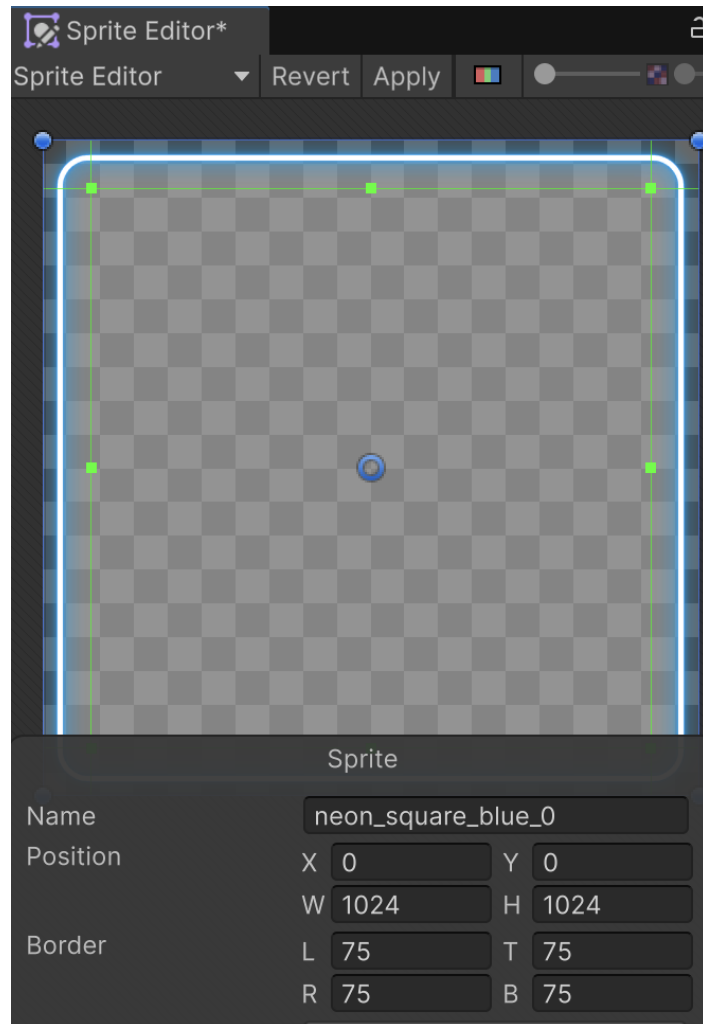


Image Type: Simple → Sliced

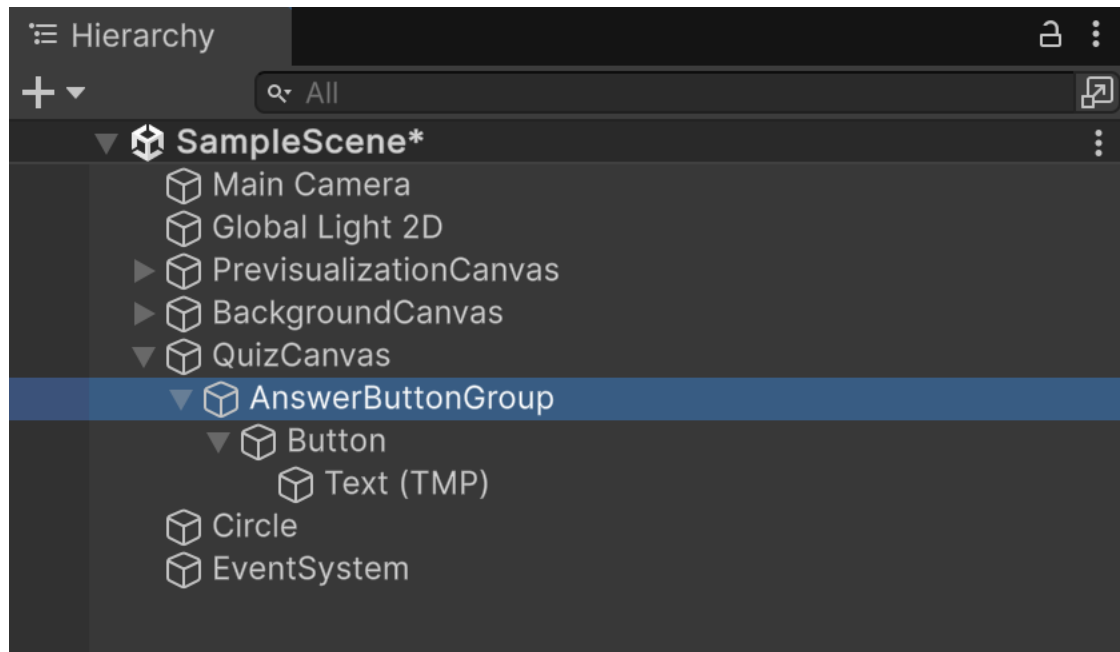
경고가 나타남 → 스프라이트 에디터로 이 스프라이에 대해 경계 지정 필요

해당 스프라이트 선택하고 스프라이트 에디터 실행

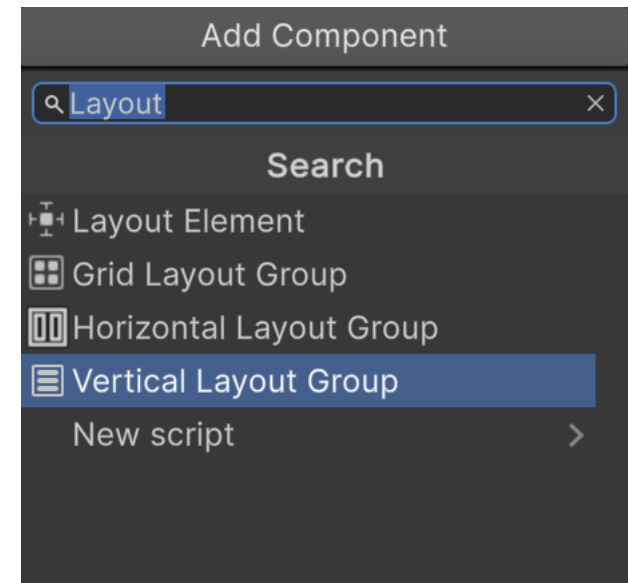


경계에 해당되는 픽셀 설정

여러 버튼을 담기 위해 빈 객체를 버튼 위에 생성

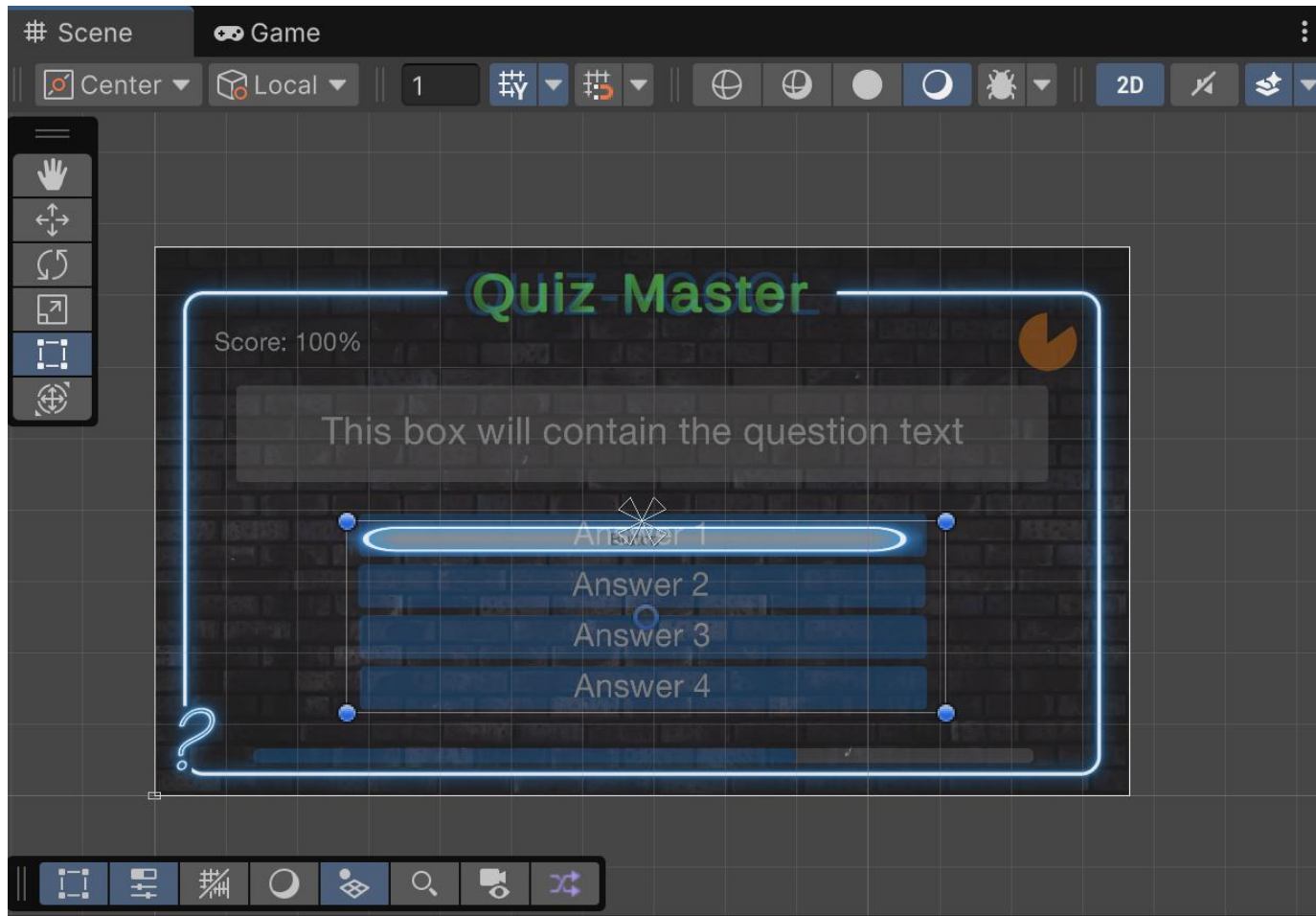


이름을 AnswerButtonGroup과
같이 설정



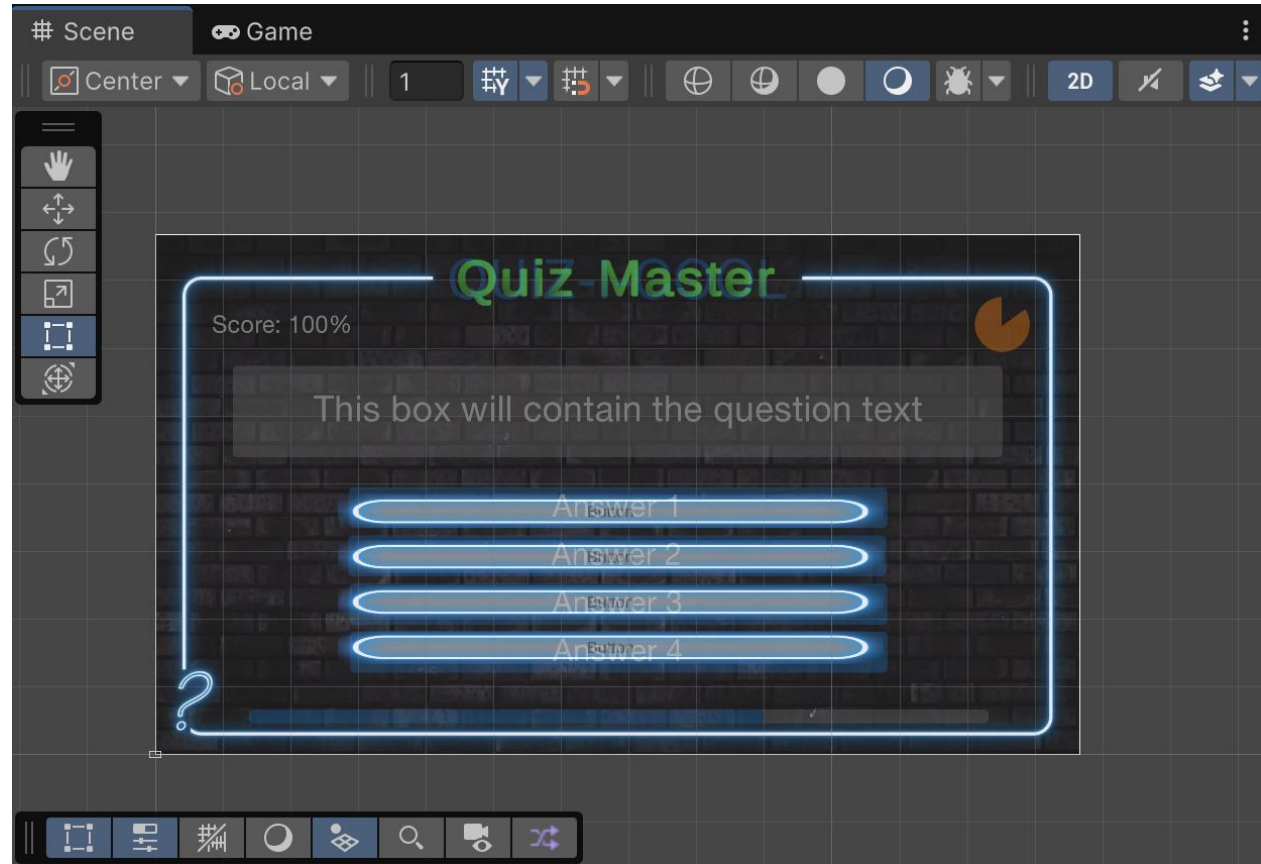
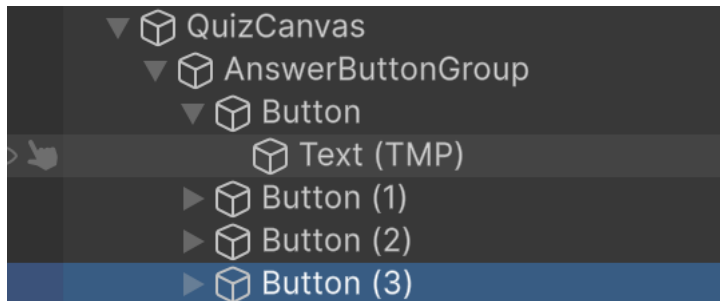
여기에 수직 레이아웃 그룹 컴포넌트 추가함
(버튼을 수직으로 배치할 수 있게 함)

버튼 그룹의 크기 조정



적당한 크기로 그룹 변형

버튼을 복사하여 4 개로 구성

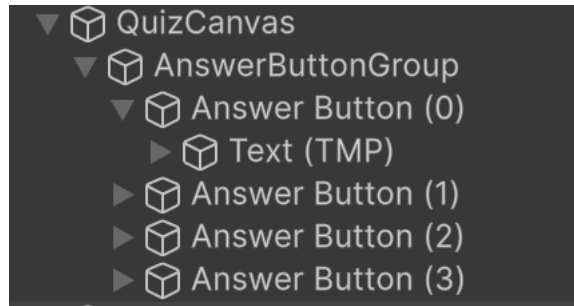


레이아웃에 따라 배치가 자동으로 됨

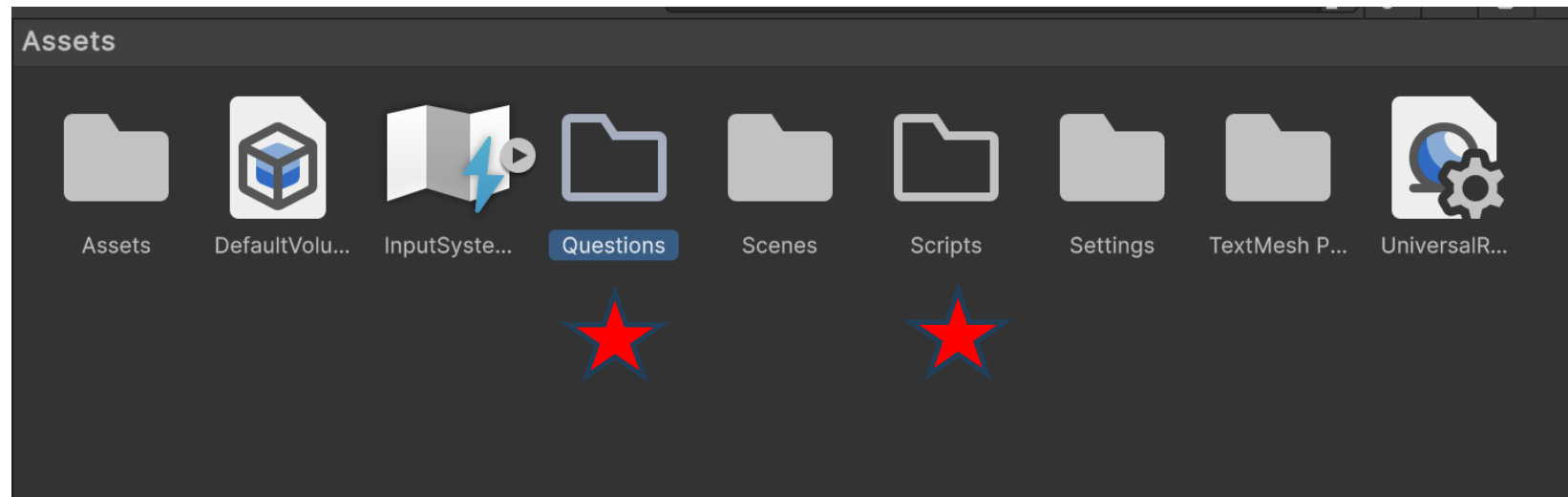
버튼 크기 조정



Button의 이름 정리



스크립터블 오브젝트를 위해 폴더 준비



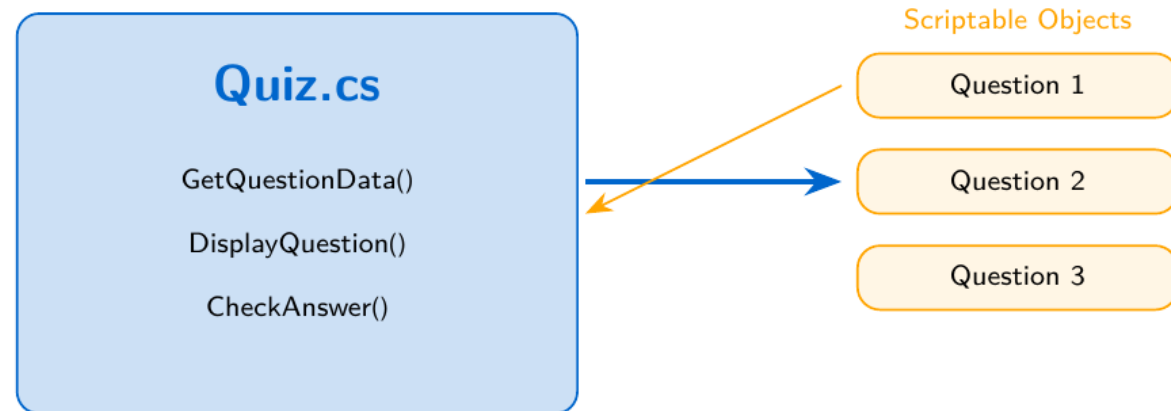
스크립터블 객체 (Scriptable Objects) 개념

ScriptableObject란?

- 데이터 컨테이너 - 씬 밖에서 데이터 유지
- GameObject에 연결할 필요 없음
- 같은 데이터를 여러 곳에서 참조 ⇒ **메모리 절약**
- 경량이며 사용 편리
- 일관성을 유지하기 위한 **템플릿**으로 기능

활용 예시

- RPG 무기/아이템 스탯
- 카드 게임의 카드 정보
- 이번 강의: 퀴즈 질문 데이터



스크립터블 객체(scriptable objects)

- 일종의 데이터 컨테이너
- 스크립트 밖에서 데이터를 유지
- 데이터를 한 곳에 저장함으로써 메모리 절약 가능
- 게임 객체에 연결할 필요 없음
- 경량이며 사용이 편리
- 일관성을 유지하기 위한 템플릿으로 기능
- 사용가능 예
 - RPG 게임에서의 무기 스탯
 - 카드 게임의 카드 정보 등

* 여기서는 질문 데이터

사용 방법의 개념

코드: quiz.cs

스크립터블 객체

GetQuestionData()

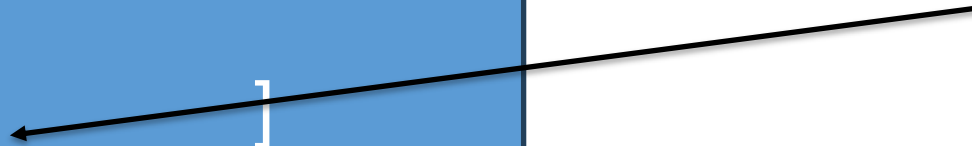
[]

DisplayQuestion()
CheckAnswer()

Question 1

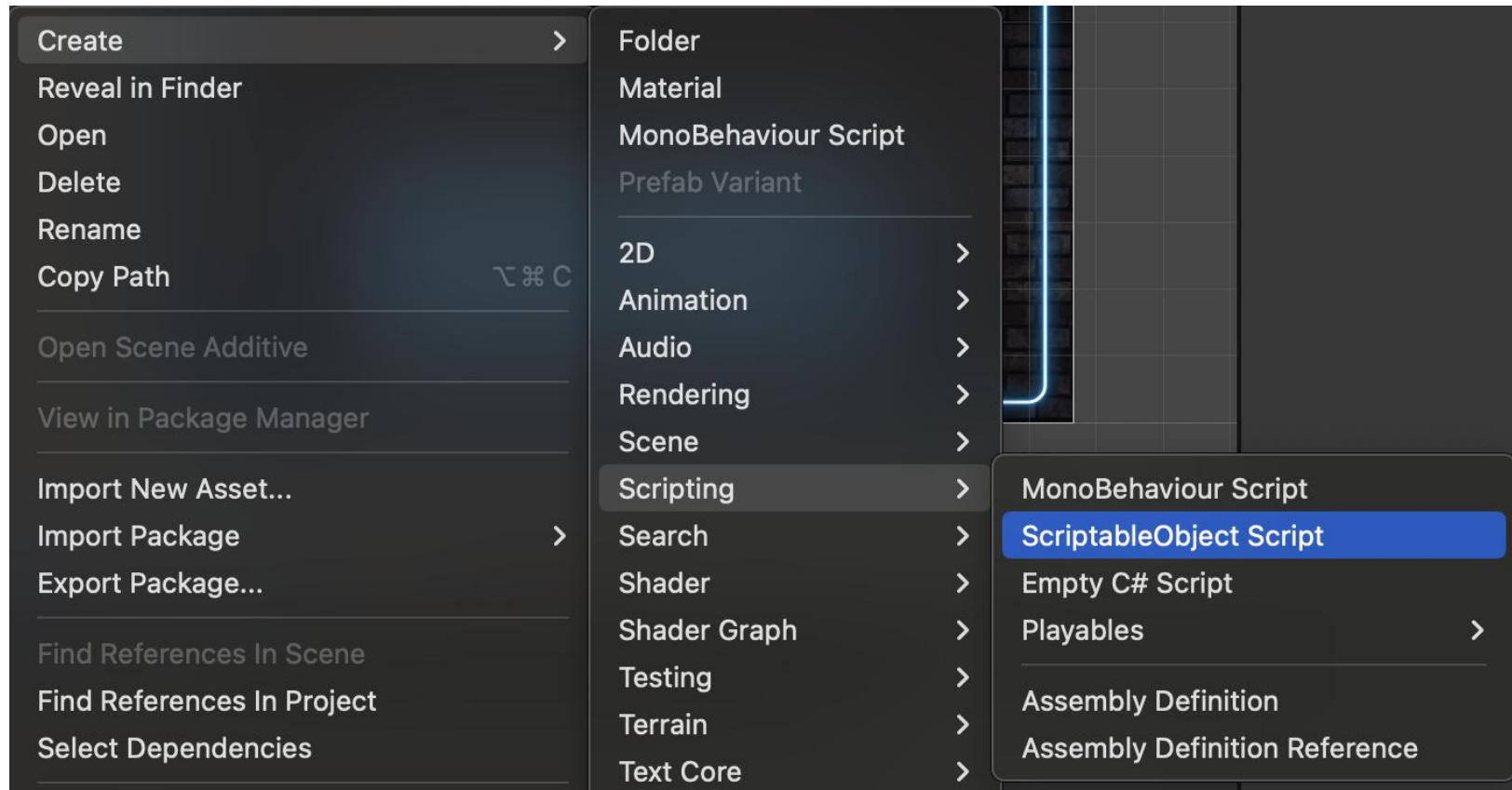
Question 2

Question 3



Scriptable Object 코드 작성

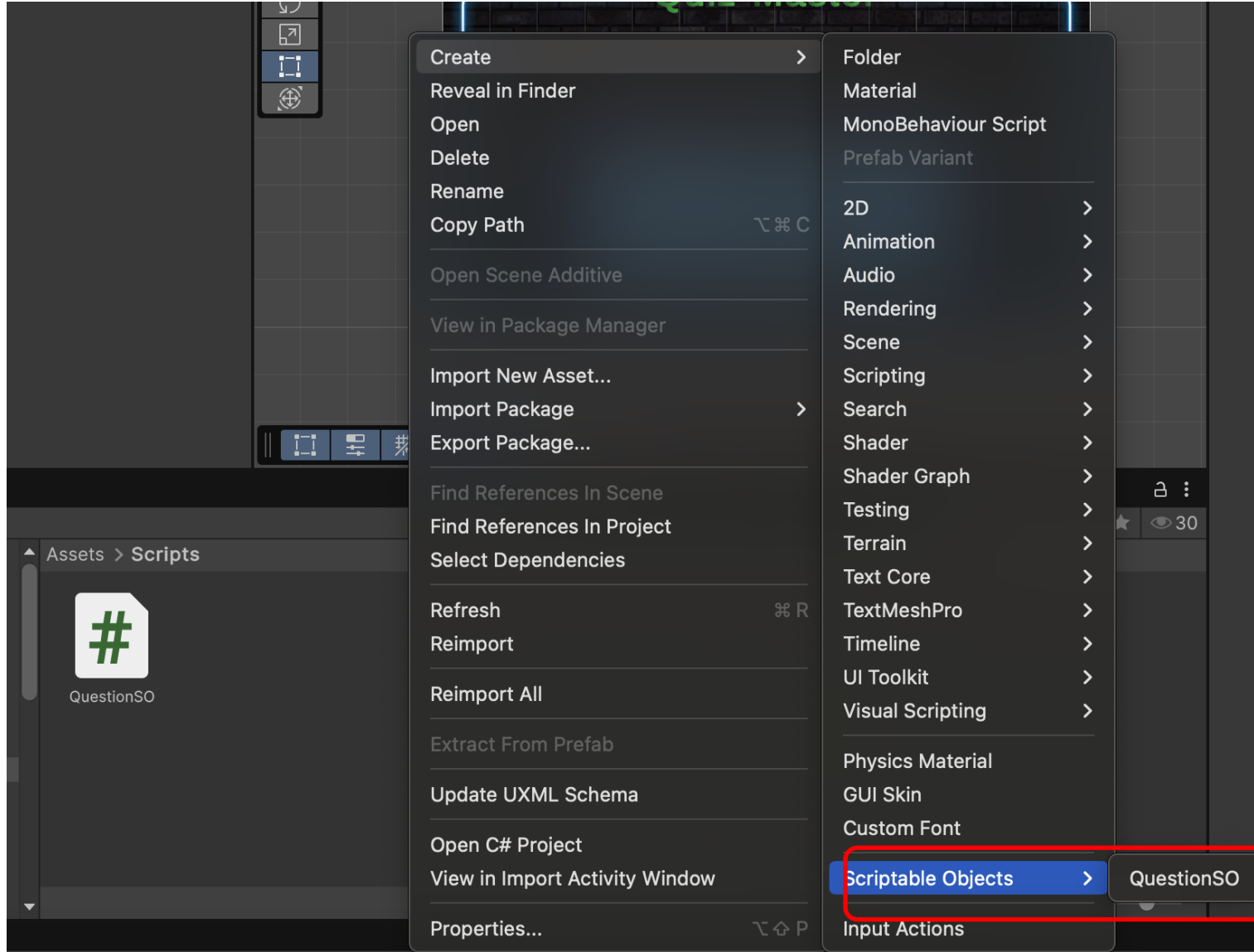
- Scripts 폴더 내에 스크립트 생성. (ScriptableObject Script로 생성)
 - QuestionSO.cs



기본 코드의 모양이 MonoBehaviour와 다름

```
1  using UnityEngine;
2
3  [CreateAssetMenu(fileName = "QuestionSO", menuName = "Scriptable Objects/QuestionSO")]
   0 references
4  public class QuestionSO : ScriptableObject
5  {
6
7  }
8  |
```

스크립터블 객체를 생성할 수 있게 됨



현재로서는 특별한 정보가 담기지
않음

QuestionSO.cs 수정

The image shows a Unity IDE window with the `QuestionSO.cs` script open. The script is a C# class that inherits from `ScriptableObject`. It has a `TextArea(2, 6)` attribute and a `SerializeField` string property named `question` with the value `"Enter New Question Text Here"`. The Hierarchy panel on the right shows the `QuestionSO` object under the `Scriptable Objects` folder.

```
C# QuestionSO.cs ×
Assets > Scripts > C# QuestionSO.cs > ...
1  using UnityEngine;
2
3  [CreateAssetMenu(fileName = "QuestionSO", menuName = "Scriptable Objects/QuestionSO")]
   0 references
4  public class QuestionSO : ScriptableObject
5  {
6      [TextArea(2, 6)]
       0 references
7      [SerializeField] string question = "Enter New Question Text Here";
8
9  }
10
```

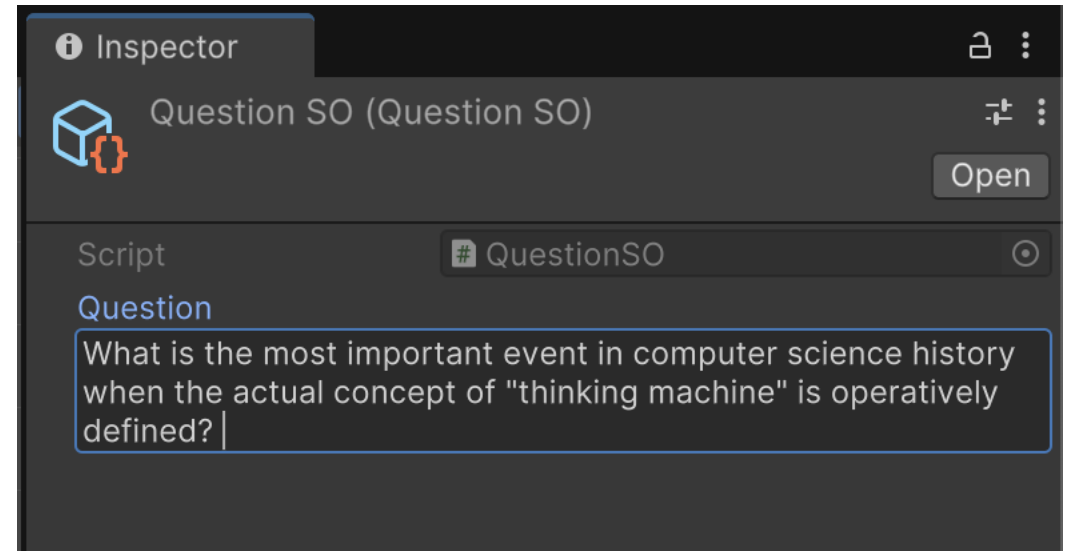
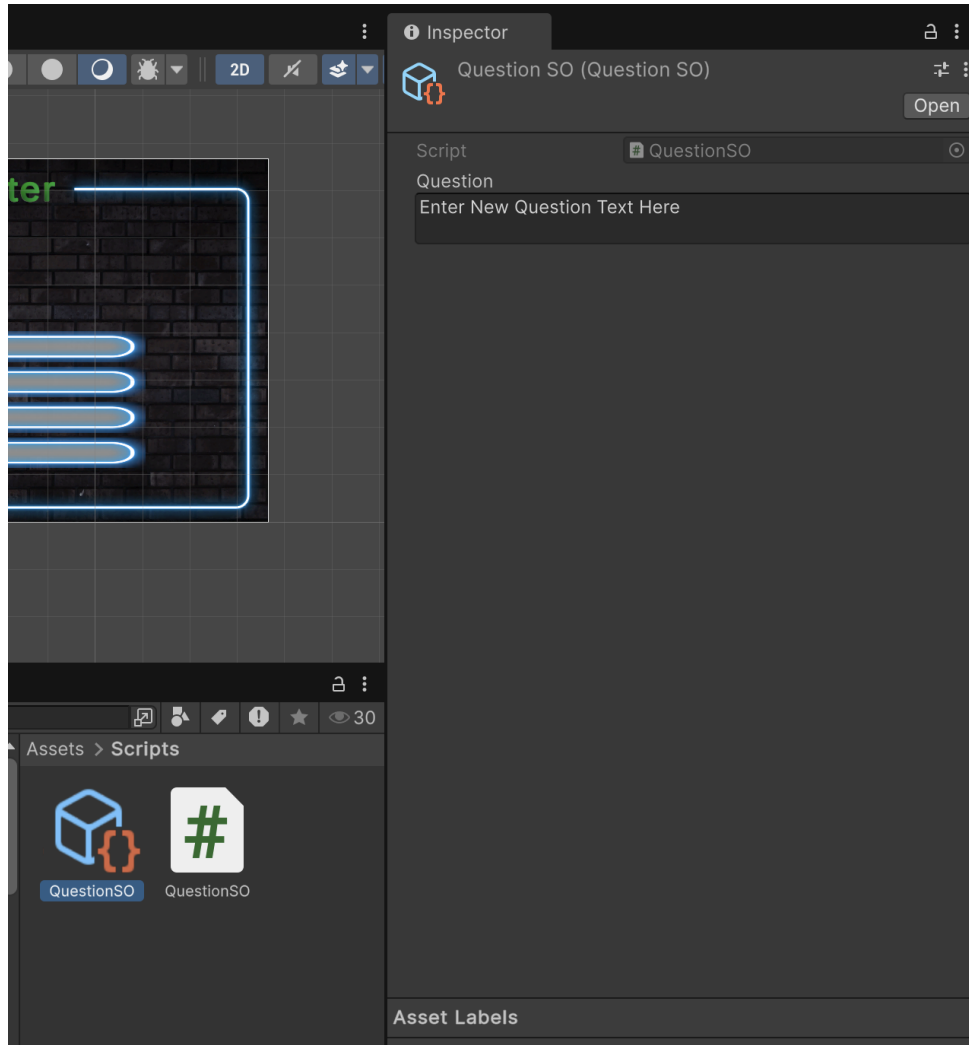
Context Menu:

- Create >
- Reveal in Finder
- Open
- Delete
- Rename
- Copy Path ⌘ ⌘ C
- Open Scene Additive
- View in Package Manager
- Import New Asset...
- Import Package >
- Export Package...
- Find References In Scene
- Find References In Project
- Select Dependencies
- Refresh ⌘ R
- Reimport
- Reimport All
- Extract From Prefab
- Update UXML Schema
- Open C# Project
- View in Import Activity Window

Hierarchy Panel:

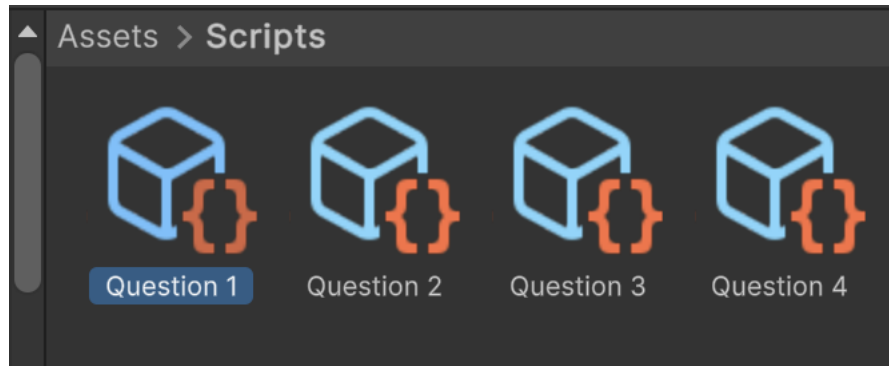
- Folder
- Material
- MonoBehaviour Script
- Prefab Variant
- 2D >
- Animation >
- Audio >
- Rendering >
- Scene >
- Scripting >
- Search >
- Shader >
- Shader Graph >
- Testing >
- Terrain >
- Text Core >
- TextMeshPro >
- Timeline >
- UI Toolkit >
- Visual Scripting >
- Physics Material
- GUI Skin
- Custom Font
- Scriptable Objects >
- QuestionSO

Scriptable Object 생성



여섯 줄까지 늘어나는 TextArea 입력 공간에
question 데이터 채울 수 있음

여러 개의 질문 데이터를 생성



게터(getter)

- 읽기 전용인 변수에 대한 “읽기” 접근을 제공
- Private 데이터의 내용을 보호

```
using UnityEngine;

[CreateAssetMenu(fileName = "QuestionSO", menuName = "Scriptable Objects/QuestionSO")]
0 references
public class QuestionSO : ScriptableObject
{
    [TextArea(2, 6)]
    1 reference
    [SerializeField] string question = "Enter New Question Text Here";

    0 references
    public string GetQuestion()
    {
        return question;
    }
}
```

배열을 이용한 정답 저장

```
public class QuestionSO : ScriptableObject
{
    [TextArea(2, 6)]
    1 reference
    [SerializeField] string question = "Enter New Question Text Here";

    1 reference
    [SerializeField] string[] answers = new string[4];
    1 reference
    [SerializeField] int correctAnswerIndex;

    0 references
    public string GetQuestion()
    {
        return question;
    }

    0 references
    public int GetCorrectAnswerIndex()
    {
        return correctAnswerIndex;
    }

    0 references
    public string GetAnswer(int idx) {
        return answers[idx];
    }
}
```

Question 1 (Question SO) Open

Script # QuestionSO

Question

What is the most important event in computer science history when the actual concept of "thinking machine" is operatively defined?

▶ Answers 4

Correct Answer Index 0



Script # QuestionSO

Question

What is the most important event in computer science history when the actual concept of "thinking machine" is operatively defined?

▼ Answers 4

=	Element 0	
=	Element 1	
=	Element 2	
=	Element 3	

+ -

Correct Answer Index 0

QuestionSO.cs – 스크립터블 오브젝트 코드

QuestionSO.cs

```
1 using UnityEngine;
2
3 [CreateAssetMenu(fileName = "QuestionSO",
4                 menuName = "Scriptable Objects/QuestionSO")]
5 public class QuestionSO : ScriptableObject
6 {
7     [TextArea(2, 6)]
8     [SerializeField] string question = "Enter New Question Text Here";
9
10    [SerializeField] string[] answers = new string[4];
11    [SerializeField] int correctAnswerIndex;
12
13    public string GetQuestion()
14    {
15        return question;
16    }
17
18    public int GetCorrectAnswerIndex()
19    {
20        return correctAnswerIndex;
21    }
22
23    public string GetAnswer(int idx)
24    {
25        return answers[idx];
26    }
27 }
```

QuestionSO – 핵심 개념 정리

[CreateAssetMenu] 어트리뷰트

- 에디터 메뉴에 생성 항목 추가
- menuName: 메뉴 경로
- 이후 Project 우클릭 → **Scriptable Objects** → QuestionSO

[TextArea(2,6)] 어트리뷰트

Inspector에서 최소 2줄, 최대 6줄의
멀티라인 텍스트 입력 공간 제공

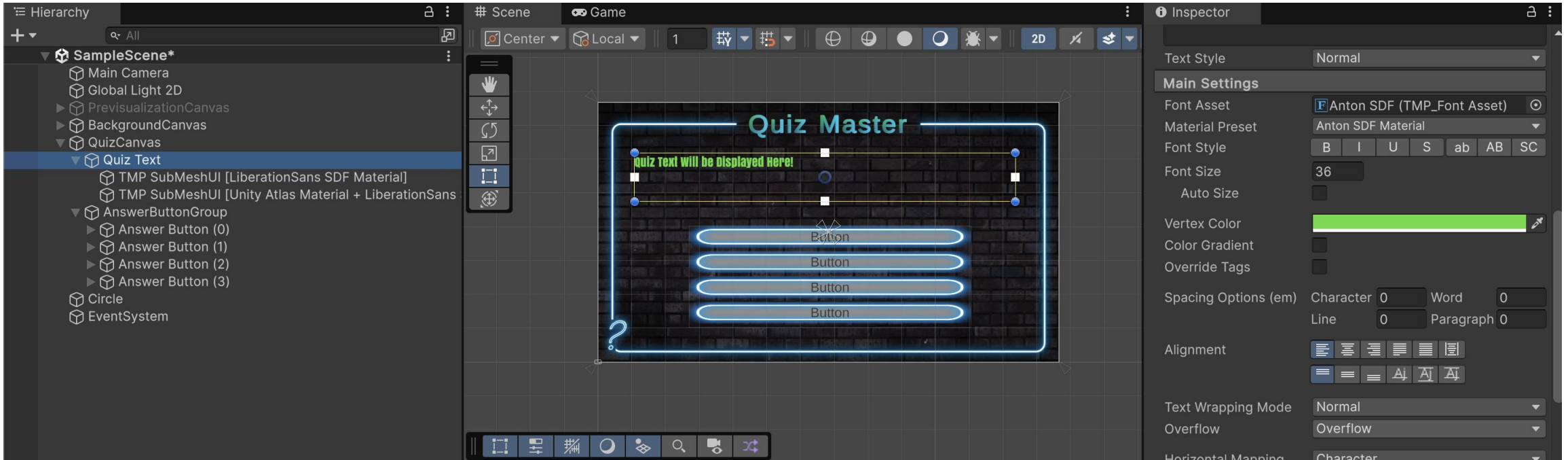
getter 메서드 패턴

Private 데이터 + Public getter

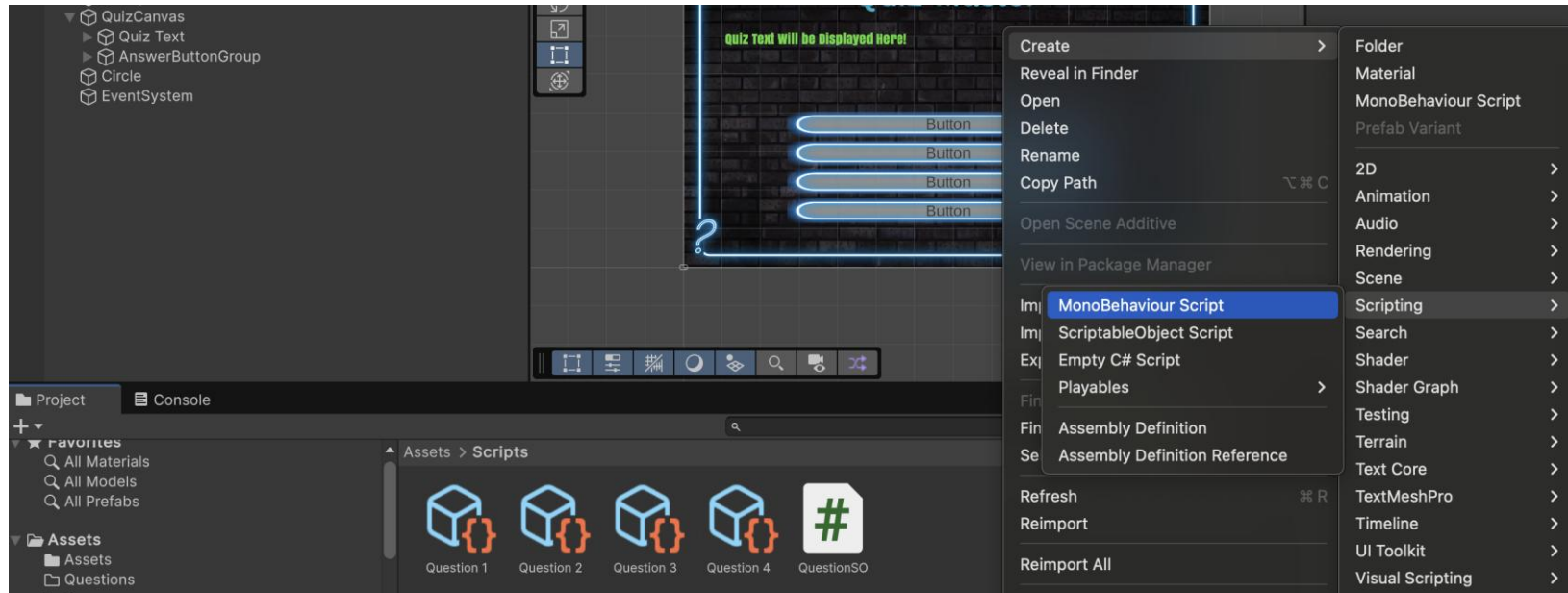
- 데이터를 외부에서 직접 수정 불가
- 읽기 전용 접근 제공
- 캡슐화(Encapsulation) 원칙

[SerializeField] 로
Inspector에서는 편집 가능

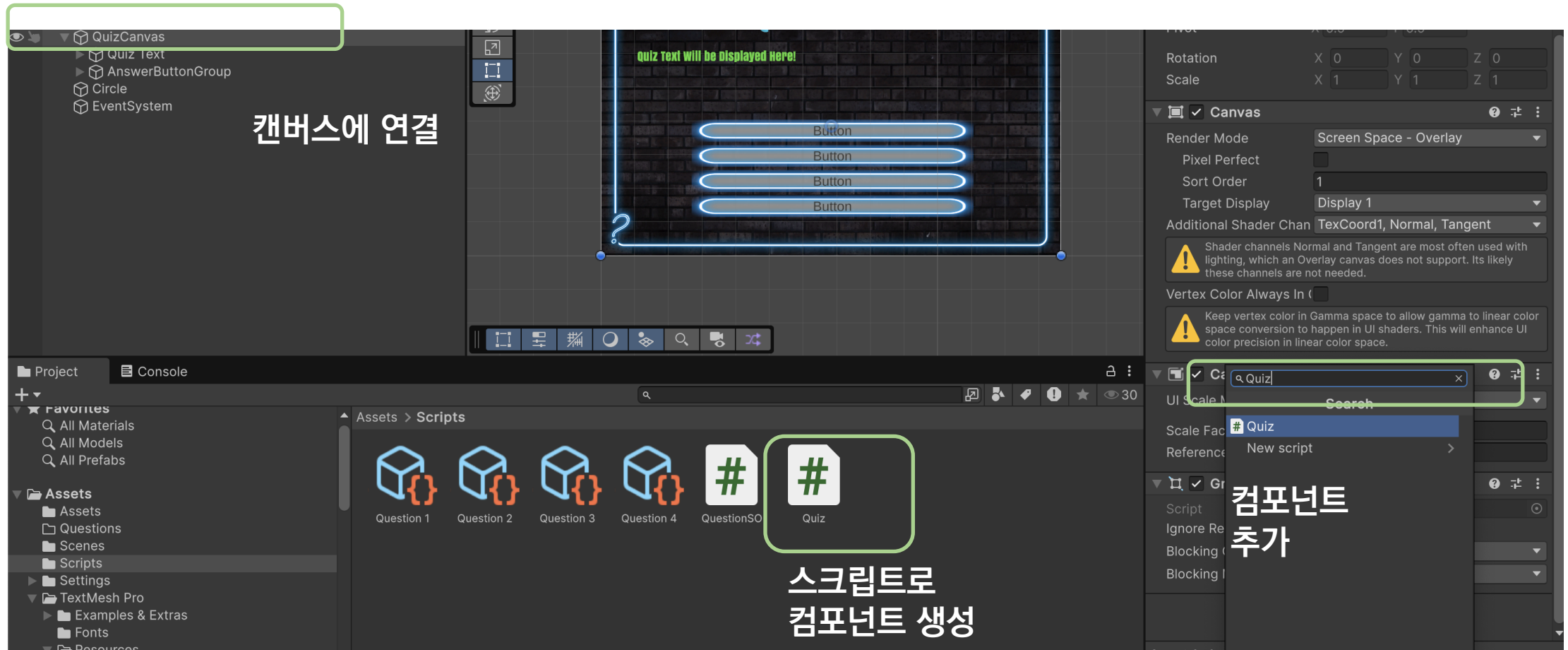
Quiz 텍스트가 나타날 공간에 TextMeshPro 배치



Quiz 캔버스에 스크립트를 설치하여 문제 다루기

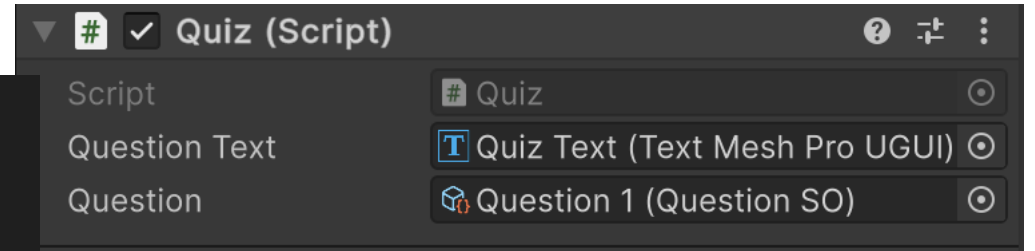


Quiz 캔버스에 스크립트를 설치하여 문제 다루기



Quiz 스크립트 수정

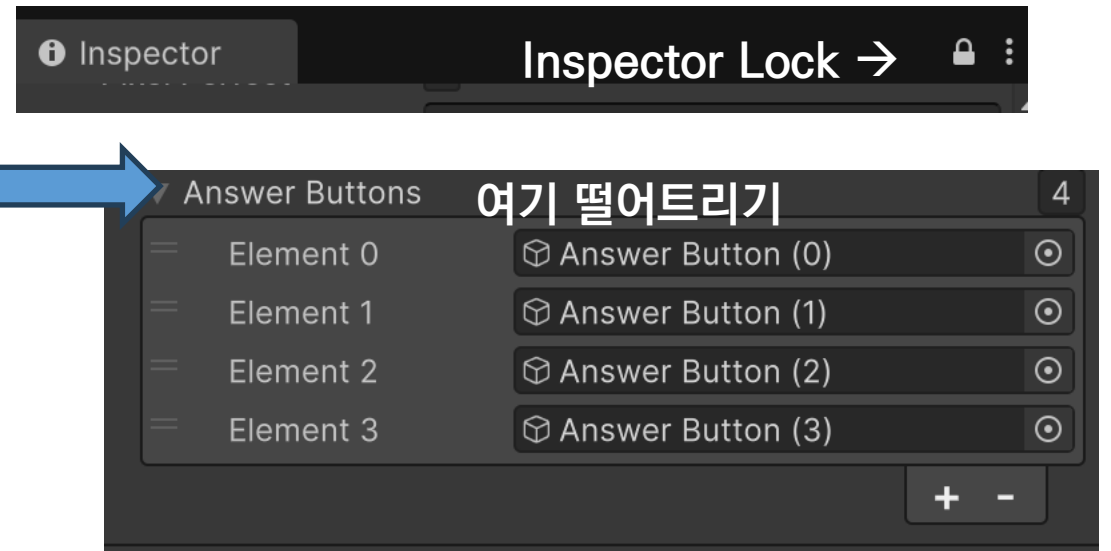
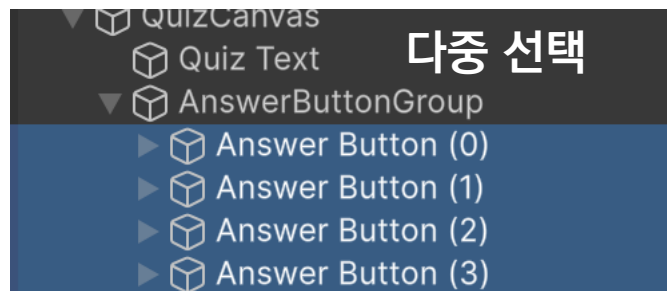
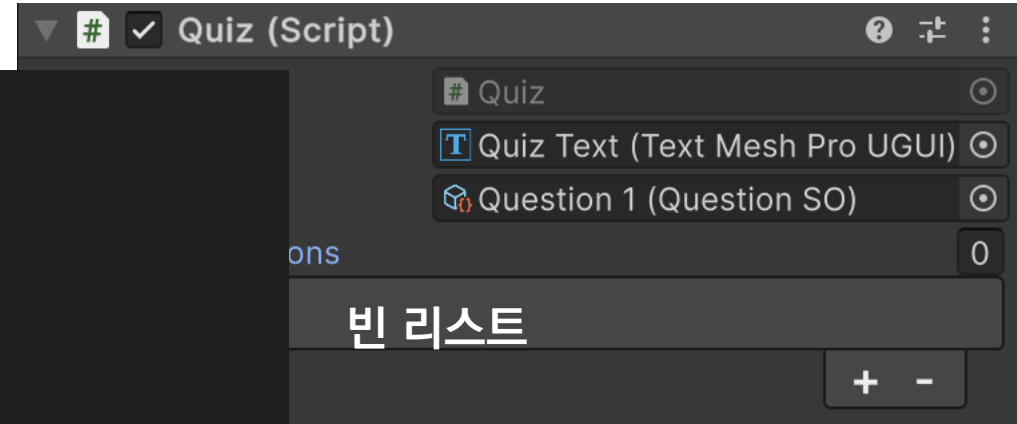
```
1  using UnityEngine;
2  using TMPro;
   0 references
3  public class Quiz : MonoBehaviour
4  {
   1 reference
5      [SerializeField] TextMeshProUGUI questionText;
   1 reference
6      [SerializeField] QuestionSO question;
7
8      // Update is called once per frame
   0 references
9      void Start()
10     {
11         questionText.text = question.GetQuestion();
12     }
13 }
14
```



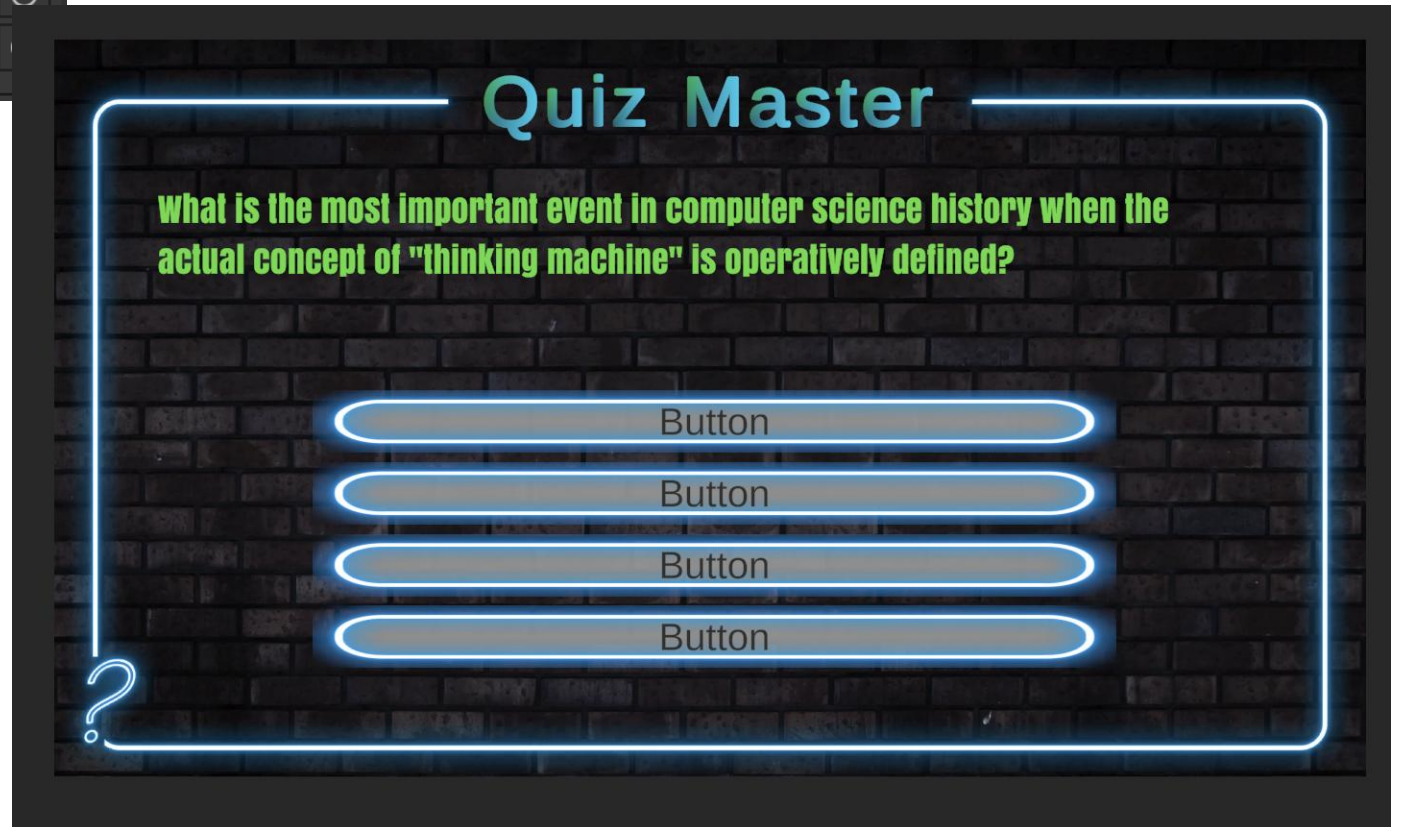
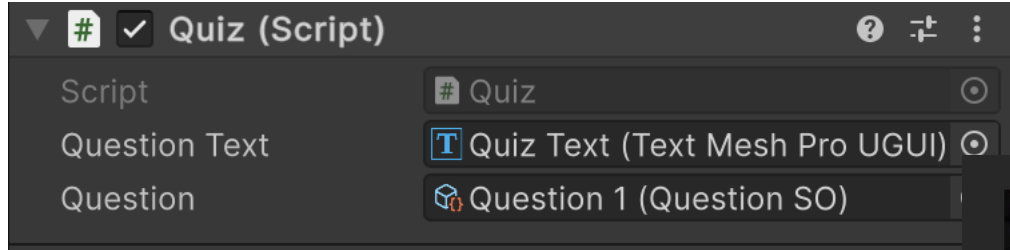
답변 가져오기

```
1 reference
[SerializeField] GameObject[] answerButtons;

// Update is called once per frame
0 references
void Start()
{
    questionText.text = question.GetQuestion();
}
```



Canvas의 Inspector에서 설정



답변 버튼에 저장된 답변 출력하기 (1개만)

```
public class Quiz : MonoBehaviour
{
    1 reference
    [SerializeField] TextMeshProUGUI questionText;
    2 references
    [SerializeField] QuestionSO question;

    1 reference
    [SerializeField] GameObject[] answerButtons;

    // Update is called once per frame
    0 references
    void Start()
    {
        questionText.text = question.GetQuestion();

        TextMeshProUGUI buttonText =
            answerButtons[0].GetComponentInChildren<TextMeshProUGUI>();

        buttonText.text = question.GetAnswer(0);
    }
}
```

Quiz Master

important event in computer science history when the
"thinking machine" is operatively defined?

on Turing's Article on Imitation Game

Button

Button

Button

For 문을 이용한 전체 답변 표시

```
void Start()
{
    questionText.text = question.GetQuestion();

    for(int i = 0; i < answerButtons.Length; i++)
    {
        TextMeshProUGUI buttonText =
            answerButtons[i].GetComponentInChildren<TextMeshProUGUI>();

        buttonText.text = question.GetAnswer(i);
    }
}
```

Quiz Master

What is the most important event in computer science history when the "Turing machine" is operatively defined?

Alan Turing's Article on Imitation Game

Steve Jobs' MacOS Invention

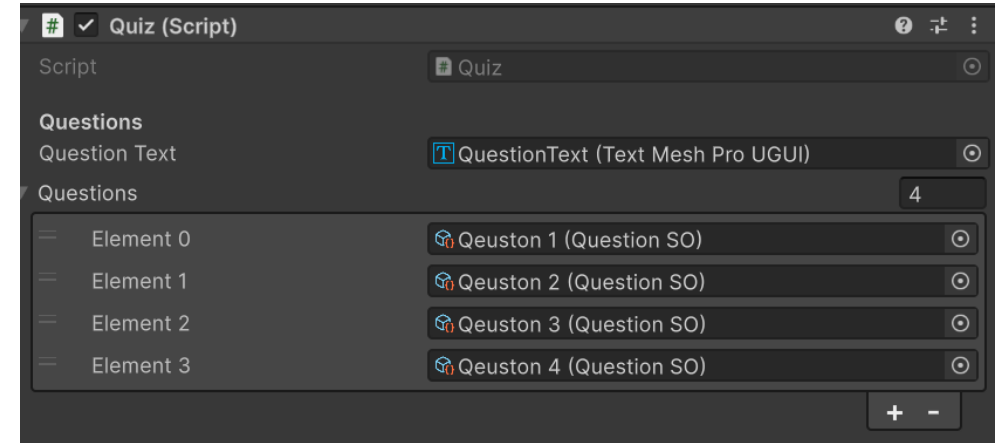
Bill Gates' DOS release

Charles Babbage's Calculation Machine



Quiz Script를 수정하자 (복수의 질문)

```
Quiz.cs
C:\> Users > young > UnityProjects > QuizUI > Assets > Scripts > Quiz.cs
1  using UnityEngine;
2  using TMPro;
3  using UnityEngine.UI;
4  using System.Collections.Generic;
5
6  public class Quiz : MonoBehaviour
7  {
8      [Header("Questions")]
9
10     [SerializeField] TextMeshProUGUI questionText;
11     [SerializeField] List<QuestionSO> questions = new List<QuestionSO>();
12     QuestionSO question;
13
14     [Header("Answers")]
15     [SerializeField] GameObject[] answerButtons;
16     int correctAnswerIndex;
17
18     [Header("Button Colors")]
19     [SerializeField] Sprite defaultAnswerSprite;
20     [SerializeField] Sprite correctAnswerSprite;
21
```



```
void Start()
{
    question = questions[0];
    questionText.text = question.GetQuestion();
    for (int i = 0; i < answerButtons.Length; i++)
    {
        TextMeshProUGUI buttonText = answerButtons[i].GetComponentInChildren<TextMeshProUGUI>();

        buttonText.text = question.GetAnswer(i);
    }
}
```


Quiz.cs – 유틸리티 메서드

버튼 상태 제어

```
1 void SetButtonState(bool state)
2 {
3     for (int i = 0; i < answerButtons.Length; i++)
4     {
5         Button button =
6             answerButtons[i].GetComponent<Button>();
7         button.interactable = state;
8     }
9 }
10
11 void SetDefaultButtonSprites()
12 {
13     for (int i = 0; i < answerButtons.Length; i++)
14     {
15         Image img =
16             answerButtons[i].GetComponent<Image>();
17         img.sprite = defaultAnswerSprite;
18     }
19 }
```

다음 문제 로드

```
1 void GetNextQuestion()
2 {
3     if (questions.Count > 0)
4     {
5         SetButtonState(true);
6         SetDefaultButtonSprites();
7         GetRandomQuestion();
8         DisplayQuestion();
9         progressBar.value++;
10    }
11 }
12
13 void GetRandomQuestion()
14 {
15     int index = Random.Range(
16         0, questions.Count);
17     question = questions[index];
18     questions.Remove(question);
19 }
```

List 주요 메서드

.Count	개수	.Add(item) 추가	.Remove(item) 삭제
.RemoveAt(idx)	위치 삭제	.Contains(item) 존재 확인	.Clear() 전체 삭제

이벤트 처리기 – OnClick 설정

버튼 OnClick 설정 순서

- 1 버튼 4개 다중 선택 (Ctrl/Cmd + 클릭)
- 2 Inspector → Button → **On Click()**
- 3 + 버튼 클릭
- 4 Object 슬롯에 **QuizCanvas** 드래그
- 5 함수 선택: **Quiz** → **OnAnswerSelected (int)**
- 6 각 버튼별로 인자값 설정 (0, 1, 2, 3)

Inspector Lock 활용

Inspector 오른쪽 자물쇠 아이콘으로
Inspector Lock 활성화
⇒ 다른 오브젝트 선택해도
현재 Inspector 고정

Runtime Only vs. Editor And Runtime

Runtime Only: 게임 실행 중에만 동작
(기본값, 권장)

다음으로 해야 할 일은?

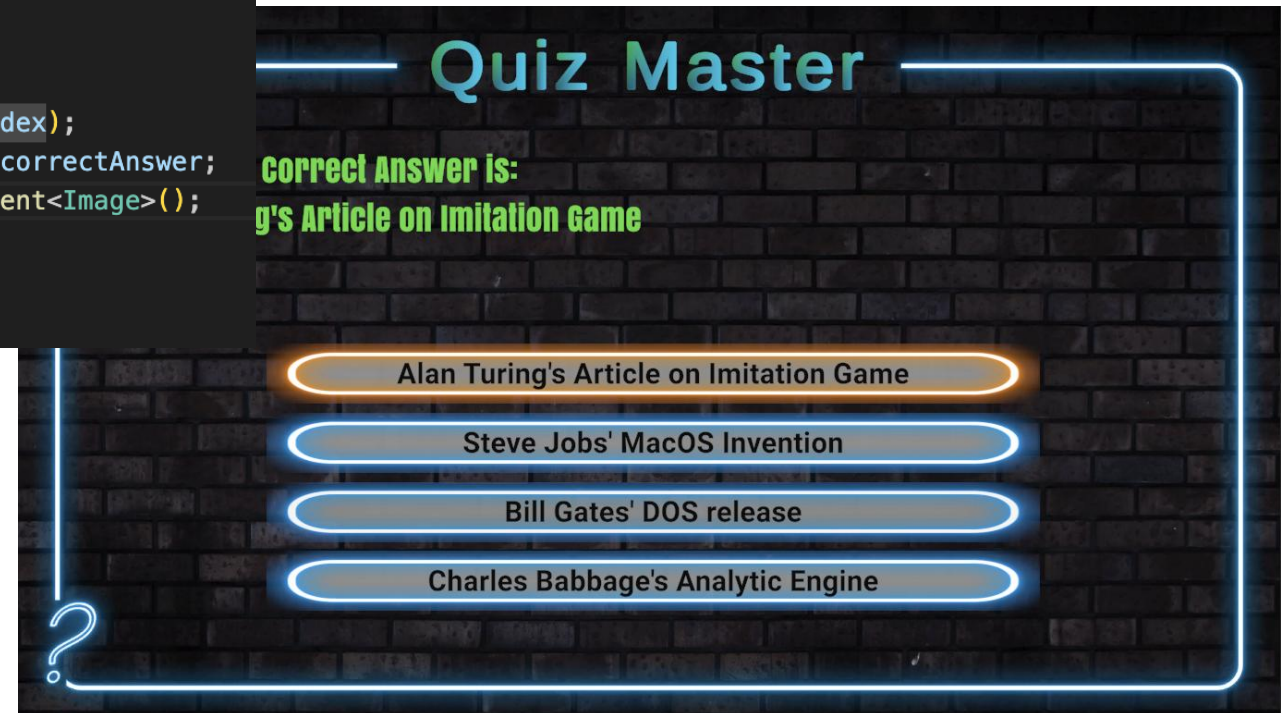
답을 하기 위해 버튼을 누르면 정답이 맞는지 확인하기

하나의 문제에 답을 하면 다음 문제로 넘어가기

동작 개선

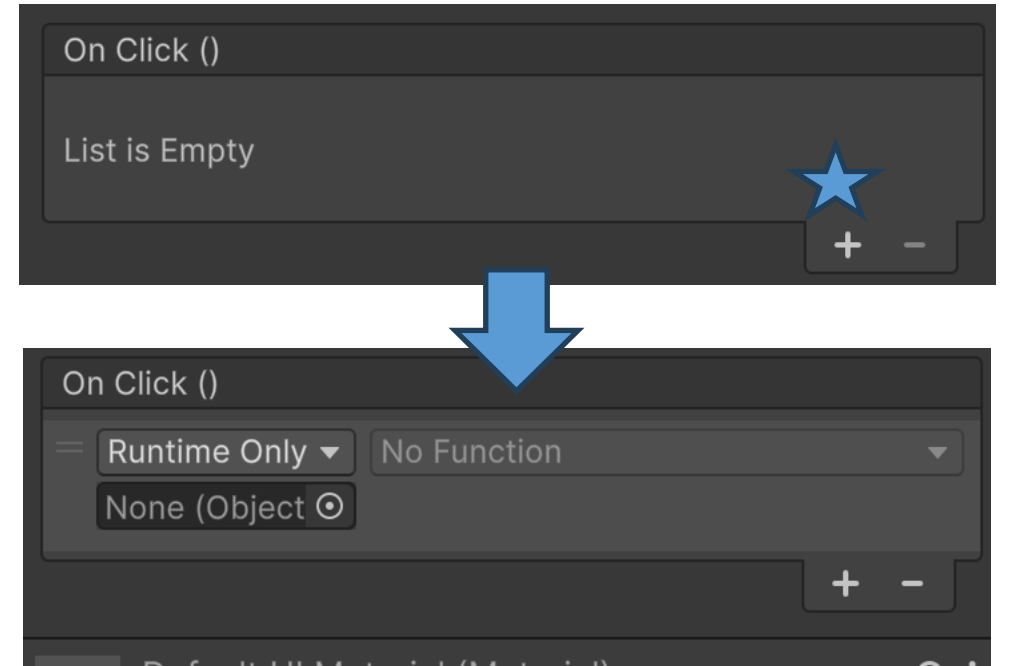
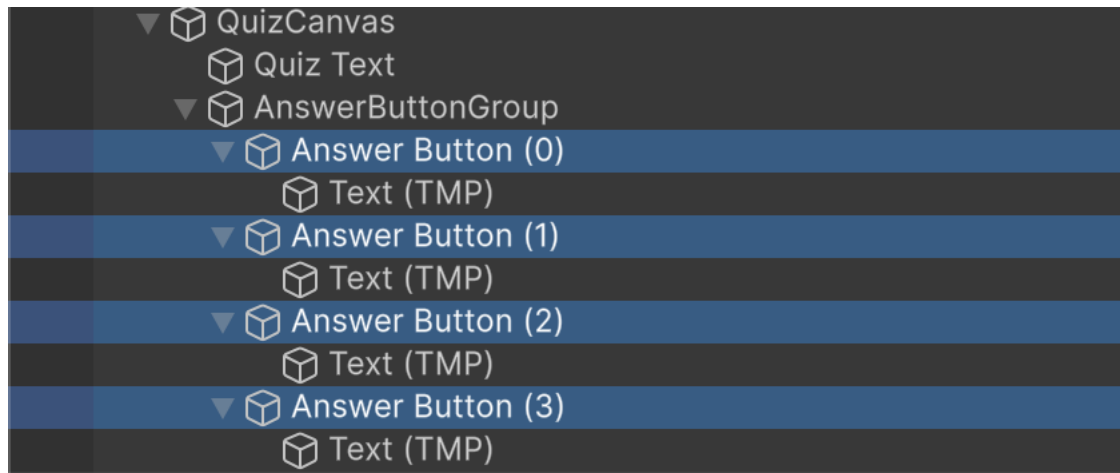
```
public void OnAnswerSelected(int index)
{
    correctAnswerIndex = question.GetCorrectAnswerIndex();
    Image buttonImage;

    if(index == correctAnswerIndex)
    {
        questionText.text = "Correct!";
        buttonImage = answerButtons[index].GetComponent<Image>();
    }
    else {
        string correctAnswer = question.GetAnswer(correctAnswerIndex);
        questionText.text = "Sorry, the Correct Answer is: \n" + correctAnswer;
        buttonImage = answerButtons[correctAnswerIndex].GetComponent<Image>();
    }
    buttonImage.sprite = correctAnswerSprite;
}
```

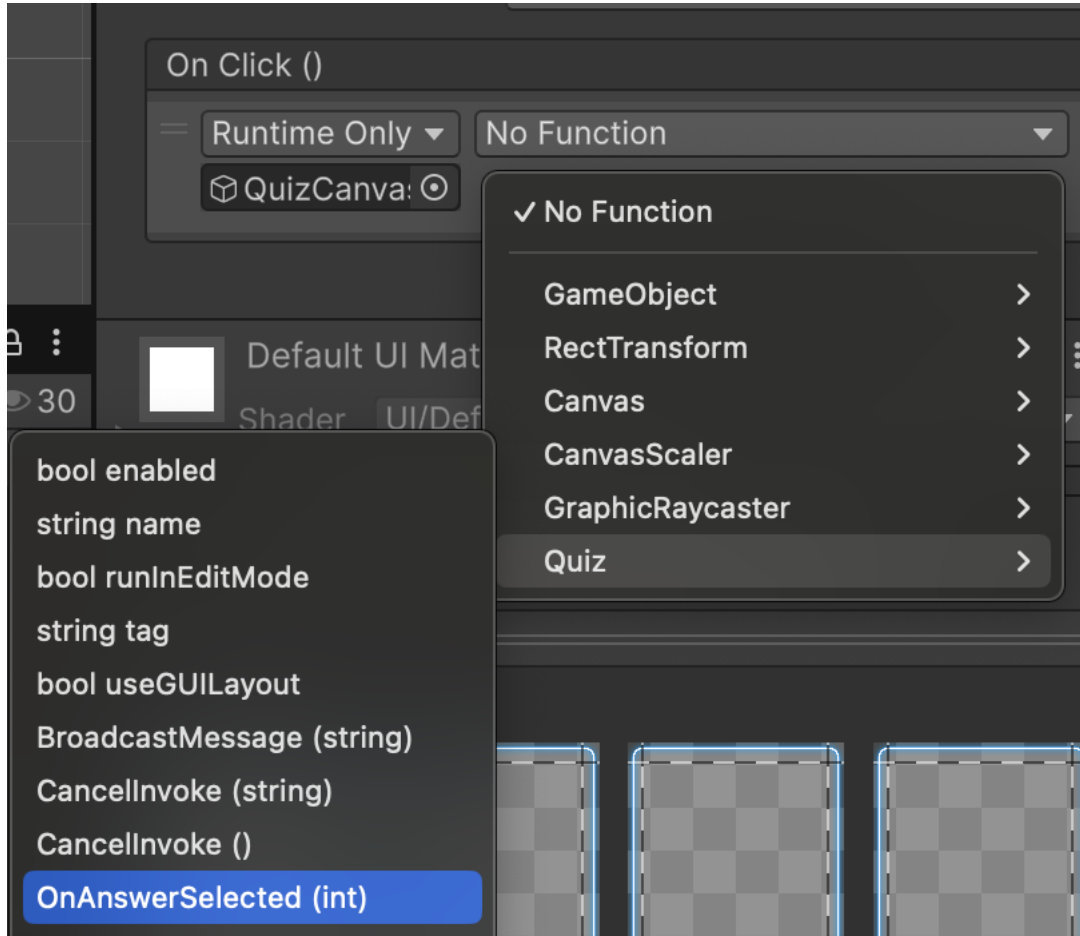


이벤트 처리기 설정

버튼들을 다중 선택 (윈도: ctrl+선택. / MacOS: cmd+선택)

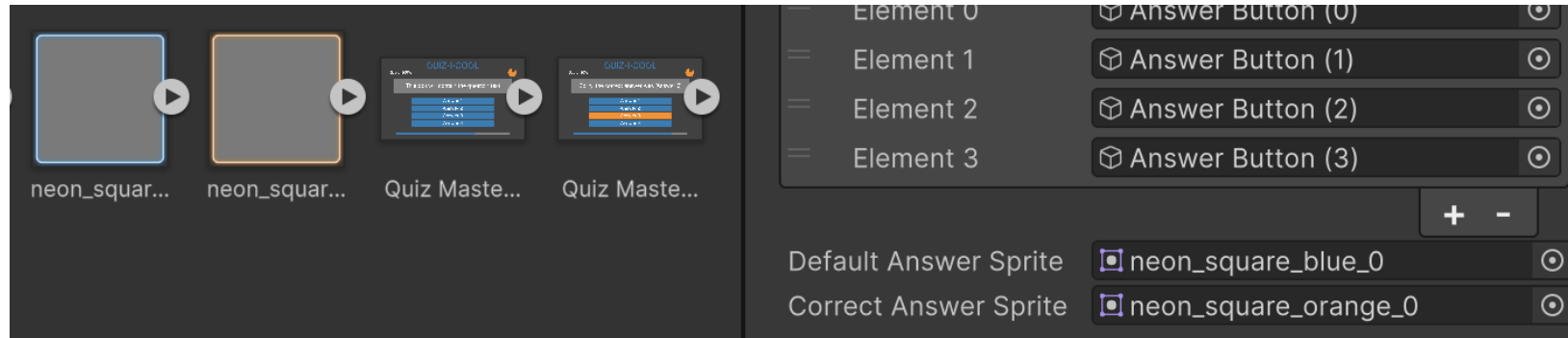


이벤트를 처리할 함수를 지정하고 각 버튼별로 인자 설정



스프라이트 바꾸기

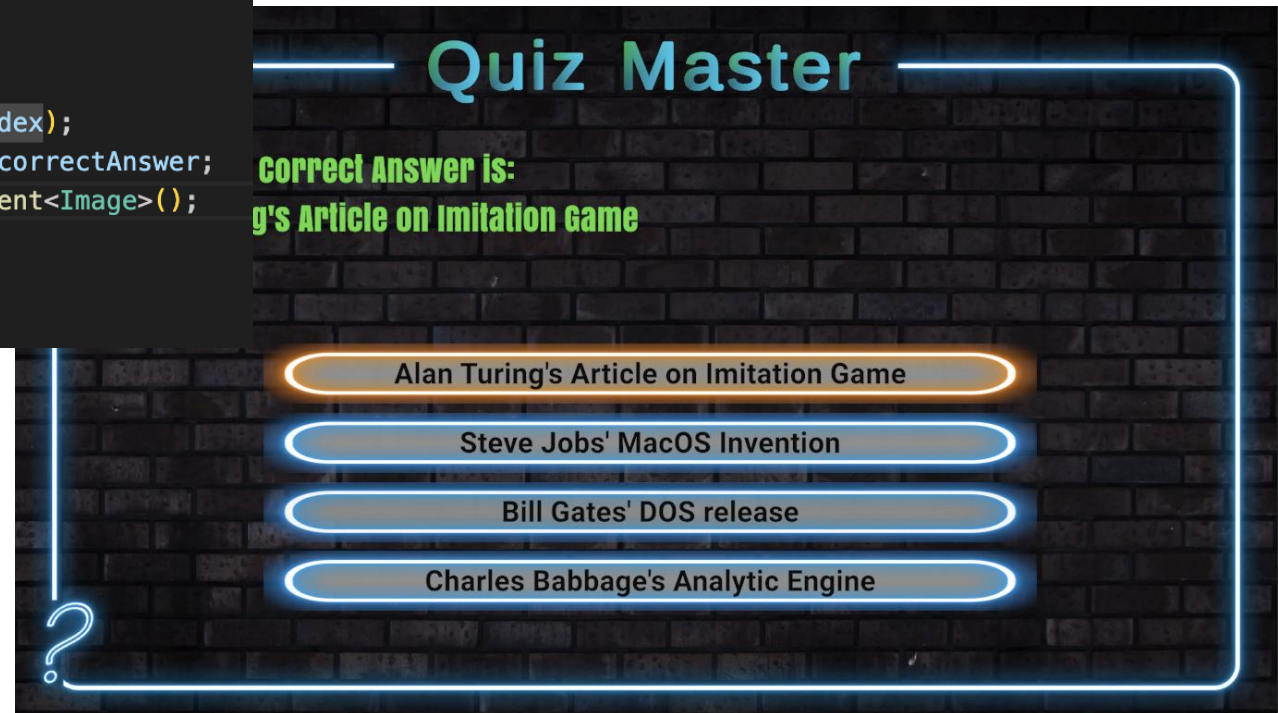
Canvas에 두 개의 스프라이트 지정



동작 개선

```
public void OnAnswerSelected(int index)
{
    correctAnswerIndex = question.GetCorrectAnswerIndex();
    Image buttonImage;

    if(index == correctAnswerIndex)
    {
        questionText.text = "Correct!";
        buttonImage = answerButtons[index].GetComponent<Image>();
    }
    else {
        string correctAnswer = question.GetAnswer(correctAnswerIndex);
        questionText.text = "Sorry, the Correct Answer is: \n" + correctAnswer;
        buttonImage = answerButtons[correctAnswerIndex].GetComponent<Image>();
    }
    buttonImage.sprite = correctAnswerSprite;
}
```



도전과제

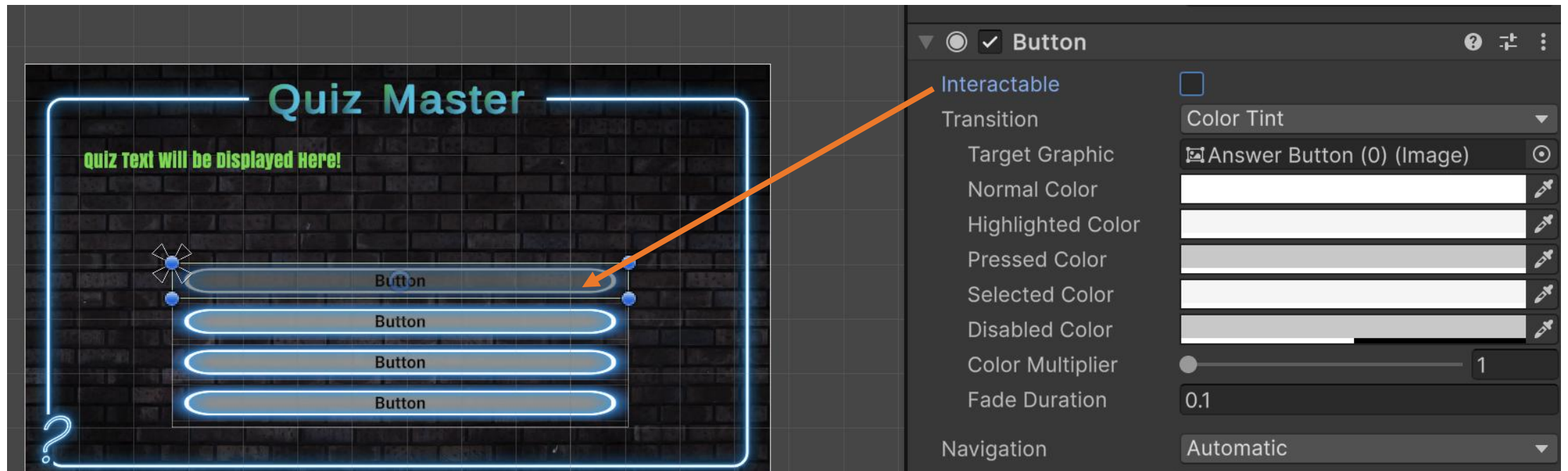
부정행위 방지

버튼을 한 번 선택하면 바꿀 수 없게 해야 함
전체적인 게임의 흐름



버튼의 활성화/비활성화

Interactable option



코드 수정 - 질문 표시를 별도의 메소드로

```
void Start()
{
    DisplayQuestion();
}

1 reference
void DisplayQuestion()
{
    questionText.text = question.GetQuestion();

    for(int i = 0; i < answerButtons.Length; i++)
    {
        TextMeshProUGUI buttonText =
            answerButtons[i].GetComponentInChildren<TextMeshProUGUI>();

        buttonText.text = question.GetAnswer(i);
    }
}
```

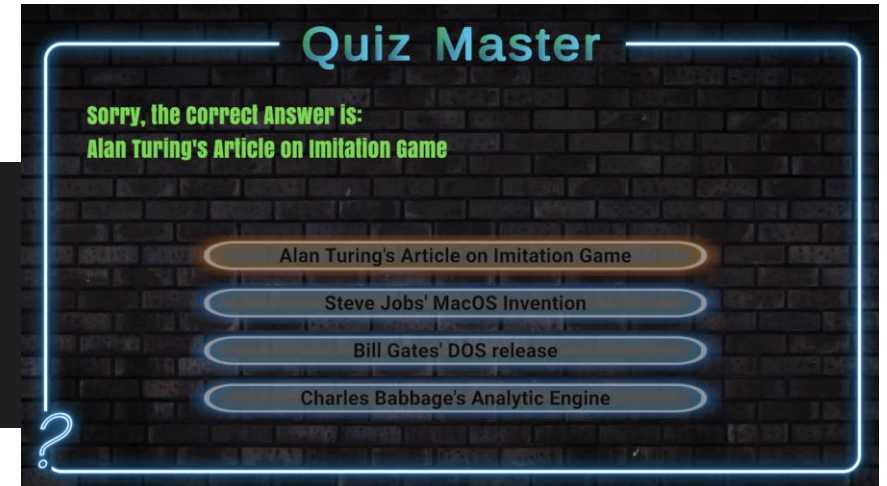
동일한 동작을 보여 줌 (당연)

버튼 상태 설정 메소드 추가

SetButtonState

```
1 reference
void SetButtonState(bool state)
{
    for(int i = 0; i < answerButtons.Length; i++)
    {
        Button button = answerButtons[i].GetComponent<Button>();
        button.interactable = state;
    }
}
```

```
public void OnAnswerSelected(int index)
{
    ...
    SetButtonState(false);
}
```



답변 선택 후에는 버튼이 비활성화 됨

새로운 문제 출력

모든 버튼을 활성화

모든 버튼에 디폴트 스프라이트 설정

- 모든 버튼을 돌며 이미지 컴포넌트 얻기
- 이미지 컴포넌트를 디폴트 스프라이트로 전환

1 reference

```
void GetNextQuestion()
{
    SetButtonState(true);
    SetDefaultButtonSprites();
    DisplayQuestion();
}
```

1 reference

```
void SetDefaultButtonSprites()
{
    for(int i = 0; i < answerButtons.Length; i++)
    {
        Image buttonImage = answerButtons[i].GetComponent<Image>();
        buttonImage.sprite = defaultAnswerSprite;
    }
}
```

```
void Start()
```

```
{
```

```
    //DisplayQuestion();
```

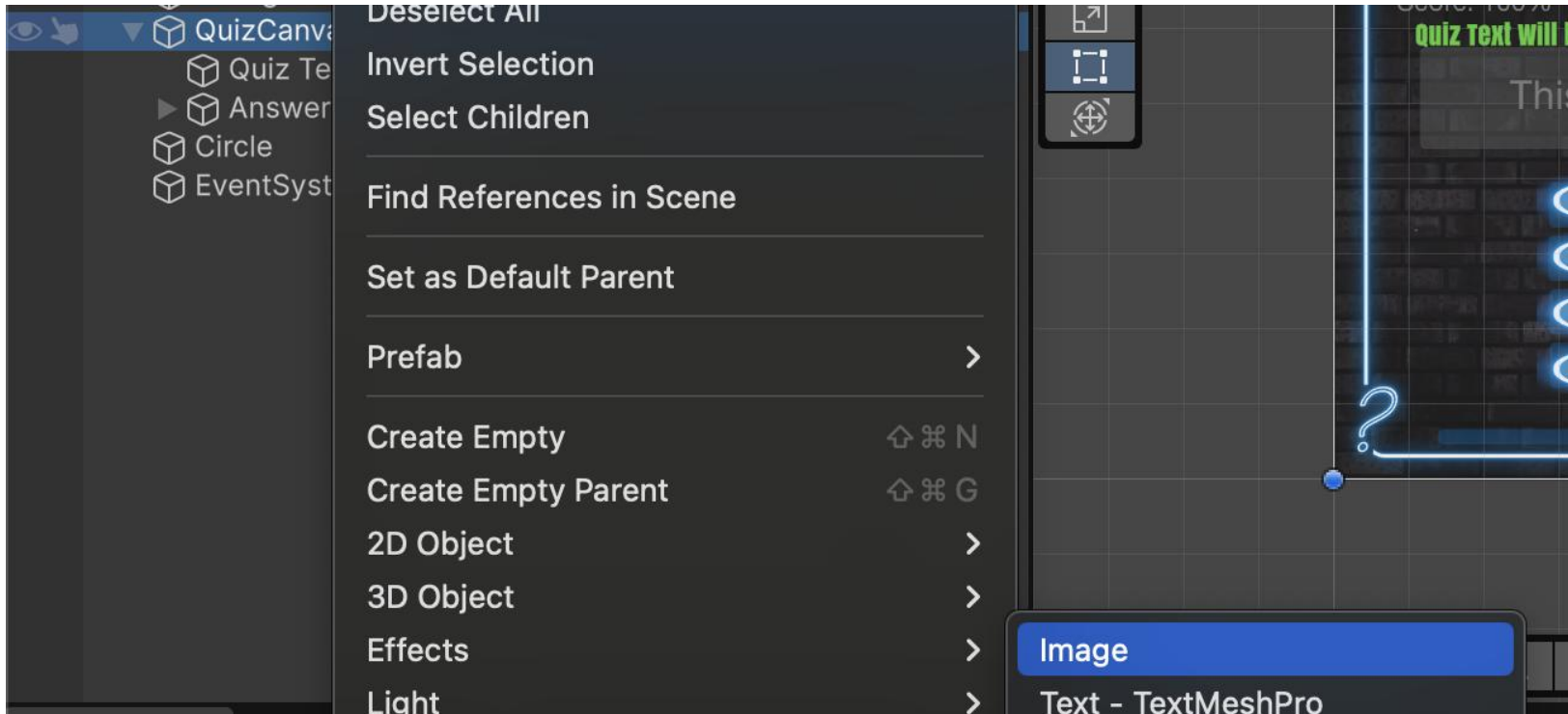
```
    GetNextQuestion();
```

```
}
```

다음 문제로 넘어가기 전에

타이머 다루기

- 문제를 보여주고 답변 제한 시간 설정
- 답변 제한 시간이 지나면 → 정답 보여 주기
- 정답 보여 주기 시간 설정 → 정답 보여주기 시간이 지나면 → 다음 문제 보이기

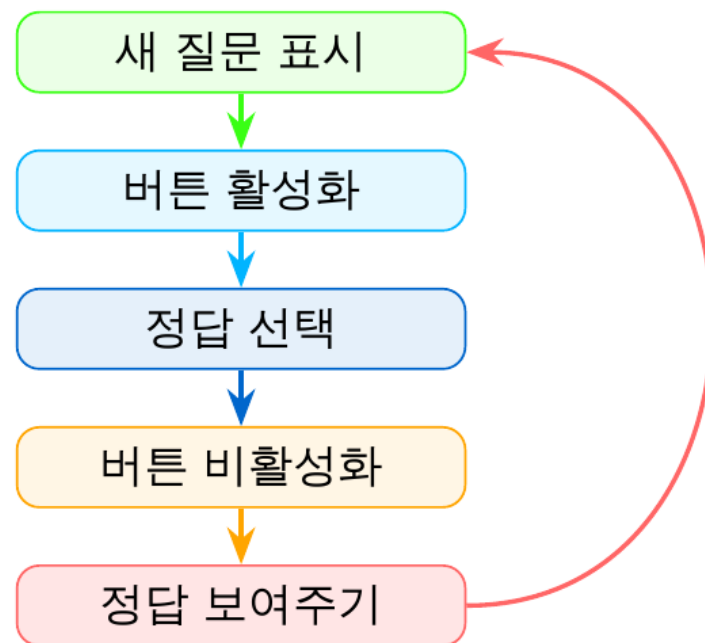


Timer 상태를 보일
이미지를 캔버스에 추가

타이머 설계

타이머 역할

- 게임 상태 관리:
 - **isAnsweringQuestion = true**: 답변 대기
 - **isAnsweringQuestion = false**: 정답 표시
- 타이머 이미지의 fillAmount를 업데이트
- 시간 초과 시 자동으로 다음 상태로 전환
- loadNextQuestion = true로 다음 문제 트리거



Timer.cs 구현

Timer.cs (상단)

```
1 using UnityEngine;
2
3 public class Timer : MonoBehaviour
4 {
5     [SerializeField] float timeToCompleQuestion = 30f;
6     [SerializeField] float timeToShowCorrectAnswer = 10f;
7
8     public bool loadNextQuestion;
9     public float fillFraction;
10    public bool isAnsweringQuestion;
11
12    float timerValue;
13
14    void Update()
15    {
16        UpdateTimer();
17    }
18
19    public void CancelTimer()
20    {
21        timerValue = 0;
22    }
23
24    public void process_Answer()
25    {
26        isAnsweringQuestion = false;
27        timerValue = timeToShowCorrectAnswer;
28    }
```

UpdateTimer()

```
1 void UpdateTimer()
2 {
3     timerValue -= Time.deltaTime;
4
5     if (isAnsweringQuestion)
6     {
7         if (timerValue > 0)
8         {
9             fillFraction = timerValue
10                / timeToCompleQuestion;
11         }
12         else
13         {
14             process_Answer();
15         }
16     }
17     else // showing correct answer
18     {
19         if (timerValue > 0)
20         {
21             fillFraction = timerValue
22                / timeToShowCorrectAnswer;
23         }
24         else
25         {
26             isAnsweringQuestion = true;
27             timerValue = timeToCompleQuestion;
28             loadNextQuestion = true;
29         }
30     }
31 }
```


타이머 이미지 – Filled Image

Filled Image 설정

- Image Type: **Filled**
- Fill Method: **Radial 360**
- Fill Origin: **Top**
- Fill Amount: 0.0 ~ 1.0 (코드로 제어)
- Clockwise: 체크 해제

```
timerImage.fillAmount =  
timer.fillFraction;
```

원형 타이머 효과

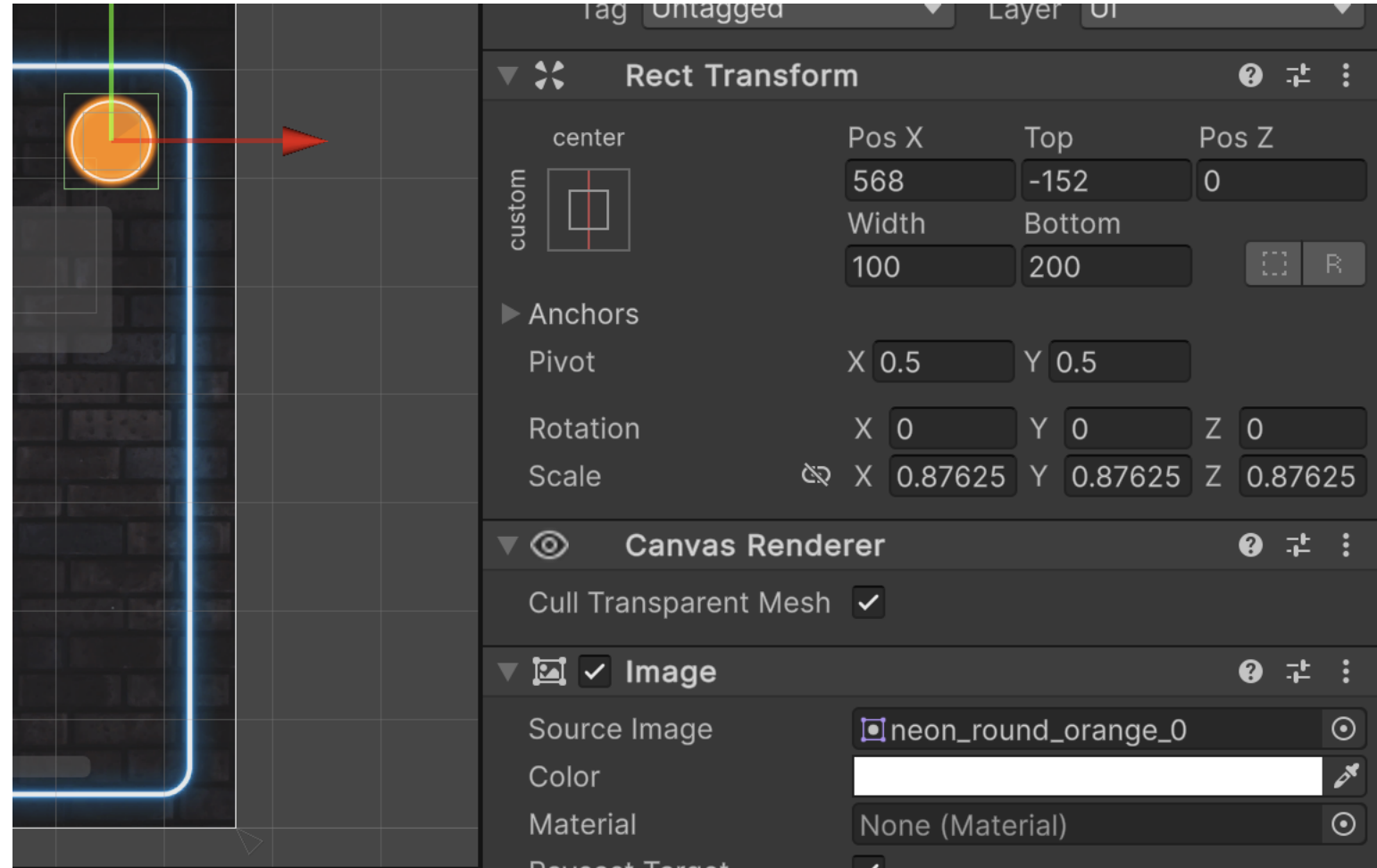
시간이 지날수록 원이 줄어드는
직관적인 타이머 UI 구현 가능

Quiz.cs 연결 코드

```
1  // Quiz.cs - Start()  
2  void Start()  
3  {  
4      timer = FindFirstObjectByType<Timer>();  
5      timer.loadNextQuestion = true;  
6      progressBar.maxValue = questions.Count;  
7      progressBar.value = 0;  
8  }  
9  
10 // Quiz.cs - Update()  
11 void Update()  
12 {  
13     timerImage.fillAmount =  
14         timer.fillFraction;  
15  
16     if (timer.loadNextQuestion)  
17     {  
18         GetNextQuestion();  
19         timer.loadNextQuestion = false;  
20     }  
21 }
```

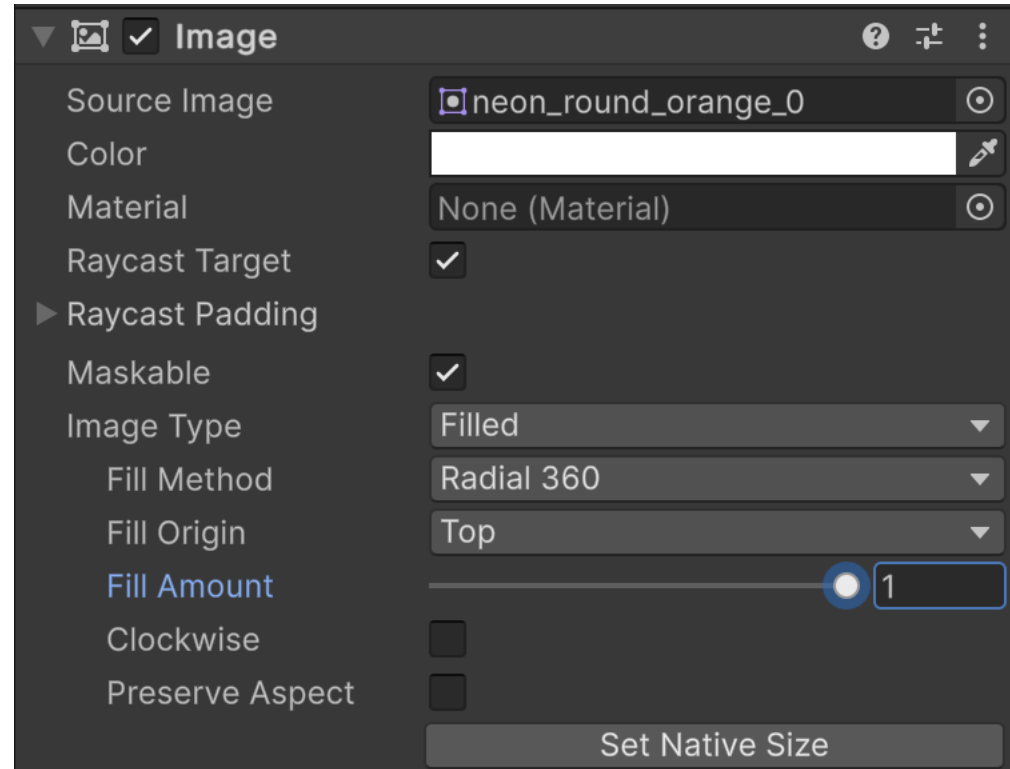
타이머 이미지

스프라이트 설정

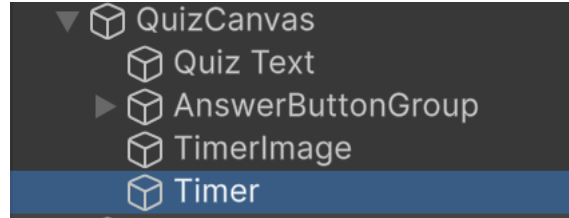


타이머 이미지를 “filled type”으로 변경

Fill Amount로 모양 조절 가능

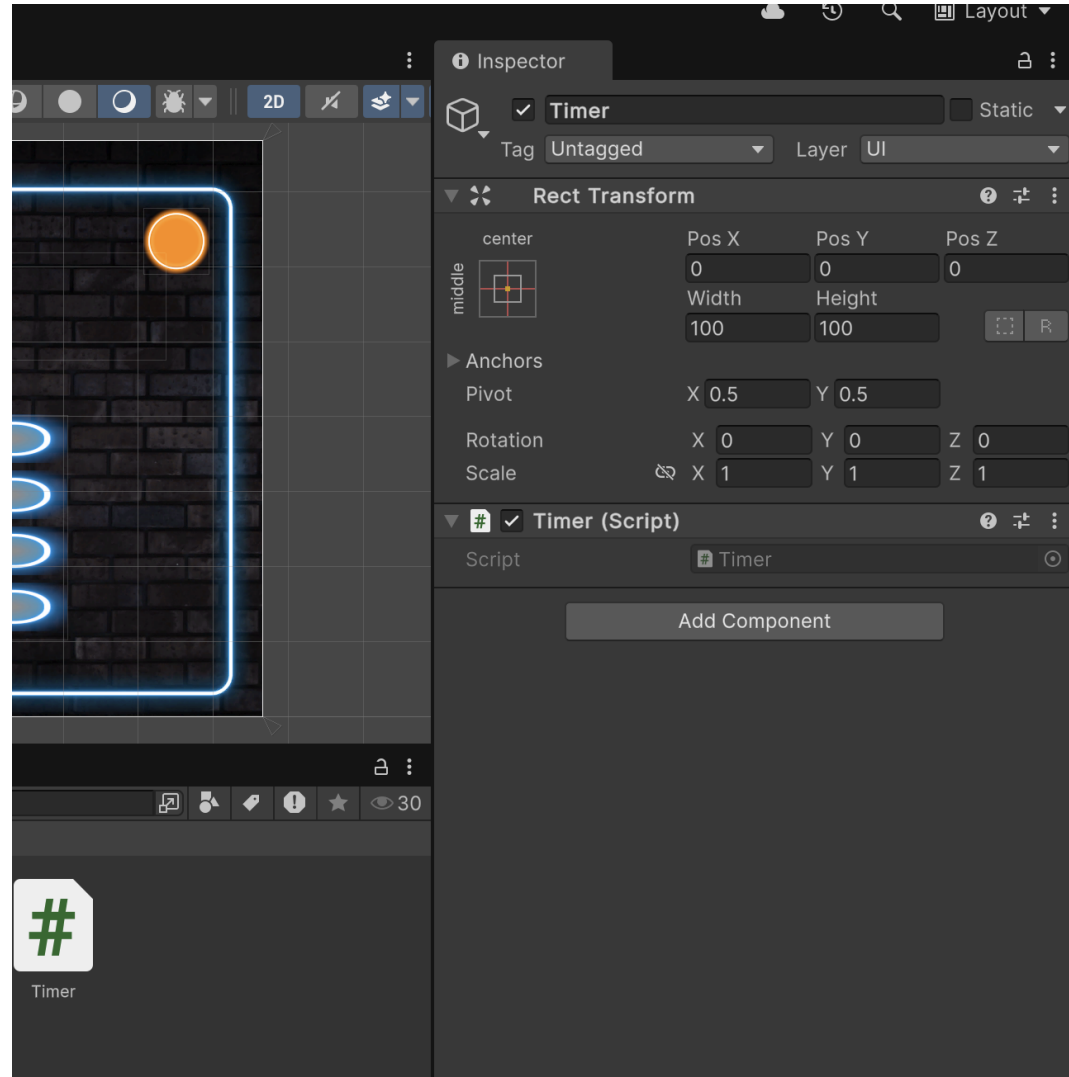


타이머의 동작을 위해 타이머 객체 생성 / 스크립트 생성



Empty Object “Timer”

Timer 객체에 Timer.cs
스크립트 생성해 연결



타이머가 할 일

게임이 어떤 상태인지 확인

- 답변을 기다리는 상태
- 정답을 보여주는 상태

타이머가 시간 측정을 하도록 함

타이머 이미지의 상태를 타이머 상태에 맞춰 변경

타이머가 다 돌았으면 게임 상태 변경

Timer.cs

```
using UnityEngine;
using UnityEngine.UIElements;

0 references
public class Timer : MonoBehaviour
{
    2 references
    [SerializeField] float timeToCompleQuestion = 30f;
    2 references
    [SerializeField] float timeToShowCorrectAnswer = 10f;

    1 reference
    public bool loadNextQuestion;
    2 references
    public float fillFraction;
    3 references
    public bool isAnsweringQuestion;

    8 references
    float timerValue;

    0 references
    void Update()
    {
        UpdateTimer();
    }

    0 references
    public void CancelTimer()
    {
        timerValue = 0;
    }
}
```

```
public void UpdateTimer()
{
    timerValue -= Time.deltaTime;

    if(isAnsweringQuestion)
    {
        if(timerValue > 0 )
        {
            fillFraction = timerValue / timeToCompleQuestion;
        }
        else
        {
            isAnsweringQuestion = false;
            timerValue = timeToShowCorrectAnswer;
        }
    }
    else // showing correct answer
    {
        if (timerValue > 0 )
        {
            fillFraction = timerValue / timeToShowCorrectAnswer;
        }
        else
        {
            isAnsweringQuestion = true;
            timerValue = timeToCompleQuestion;
            loadNextQuestion = true;
        }
    }
}
```

Quiz.cs – 기본 구조

Quiz.cs (멤버 변수)

```
1 using UnityEngine;
2 using UnityEngine.UI;
3 using TMPro;
4 using System.Collections.Generic;
5
6 public class Quiz : MonoBehaviour
7 {
8     [Header("Questions")]
9     [SerializeField] TextMeshProUGUI questionText;
10    [SerializeField] List<QuestionSO> questions
11        = new List<QuestionSO>();
12    QuestionSO question;
13
14    [Header("Answers")]
15    [SerializeField] GameObject[] answerButtons;
16    int correctAnswerIndex;
17
18    [Header("Button Colors")]
19    [SerializeField] Sprite defaultAnswerSprite;
20    [SerializeField] Sprite correctAnswerSprite;
21
22    [Header("Timer")]
23    [SerializeField] Image timerImage;
24    Timer timer;
25
26    [Header("Progress Bar")]
27    [SerializeField] Slider progressBar;
```

[Header] 어트리뷰트

Inspector에서 섹션 헤더를 표시
관련 필드들을 그룹으로 묶어 가독성 향상

List vs. Array

Array 크기 고정
 string[] arr
 = new string[4];

List 크기 동적
 List<T> list
 = new List<T>();

Quiz.cs – 질문 표시 및 답변 처리

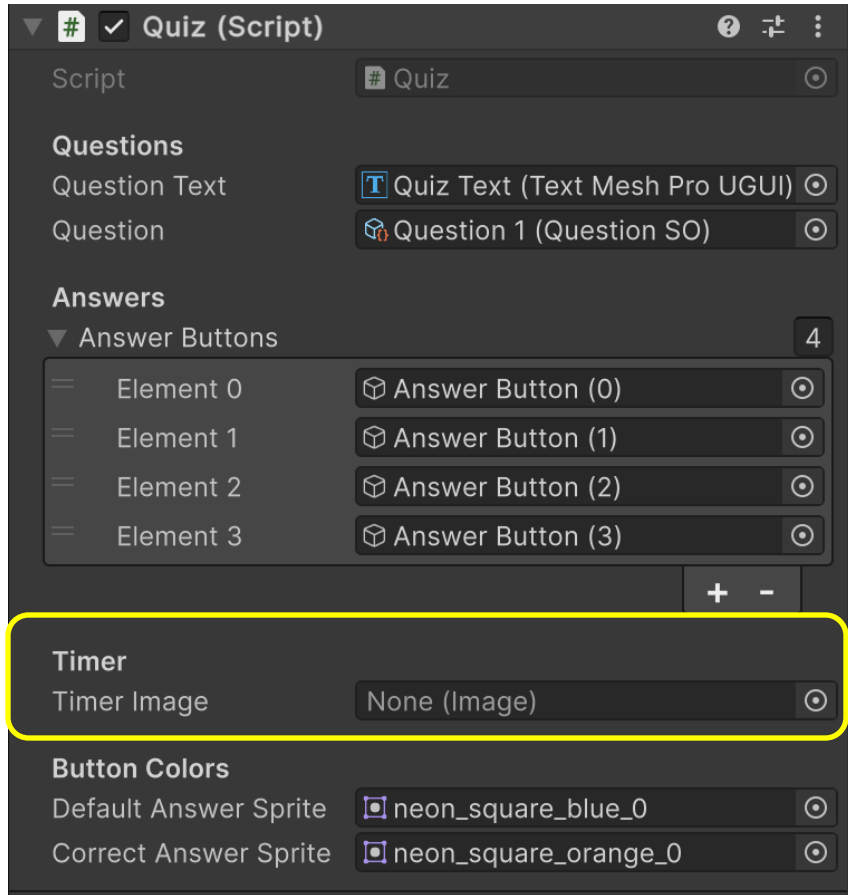
질문 표시

```
1 void Start()
2 {
3     timer = FindFirstObjectByType<Timer>();
4     timer.loadNextQuestion = true;
5     progressBar.maxValue = questions.Count;
6     progressBar.value = 0;
7 }
8
9 void Update()
10 {
11     timerImage.fillAmount = timer.fillFraction;
12     if (timer.loadNextQuestion)
13     {
14         GetNextQuestion();
15         timer.loadNextQuestion = false;
16     }
17 }
18
19 void DisplayQuestion()
20 {
21     questionText.text = question.GetQuestion();
22     for (int i = 0; i < answerButtons.Length; i++)
23     {
24         var btnText =
25             answerButtons[i]
26             .GetComponentInChildren<TextMeshProUGUI>();
27         btnText.text = question.GetAnswer(i);
28     }
29 }
```

답변 선택 처리

```
1 public void OnAnswerSelected(int index)
2 {
3     correctAnswerIndex =
4         question.GetCorrectAnswerIndex();
5     Image buttonImage;
6     if (index == correctAnswerIndex)
7     {
8         questionText.text = "Correct!";
9         buttonImage =
10             answerButtons[index]
11             .GetComponent<Image>();
12     }
13     else
14     {
15         string correctAnswer =
16             question.GetAnswer(correctAnswerIndex);
17         questionText.text =
18             "Sorry, the Correct Answer is:\n"
19             + correctAnswer;
20         buttonImage =
21             answerButtons[correctAnswerIndex]
22             .GetComponent<Image>();
23     }
24     buttonImage.sprite = correctAnswerSprite;
25     timer.CancelTimer();
26     SetButtonState(false);
27 }
```


Quiz script 변경



```
[Header("Questions")]
3 references
[SerializeField] TextMeshProUGUI questionText;
4 references
[SerializeField] QuestionSO question;

[Header("Answers")]
8 references
[SerializeField] GameObject[] answerButtons;
4 references
int correctAnswerIndex;

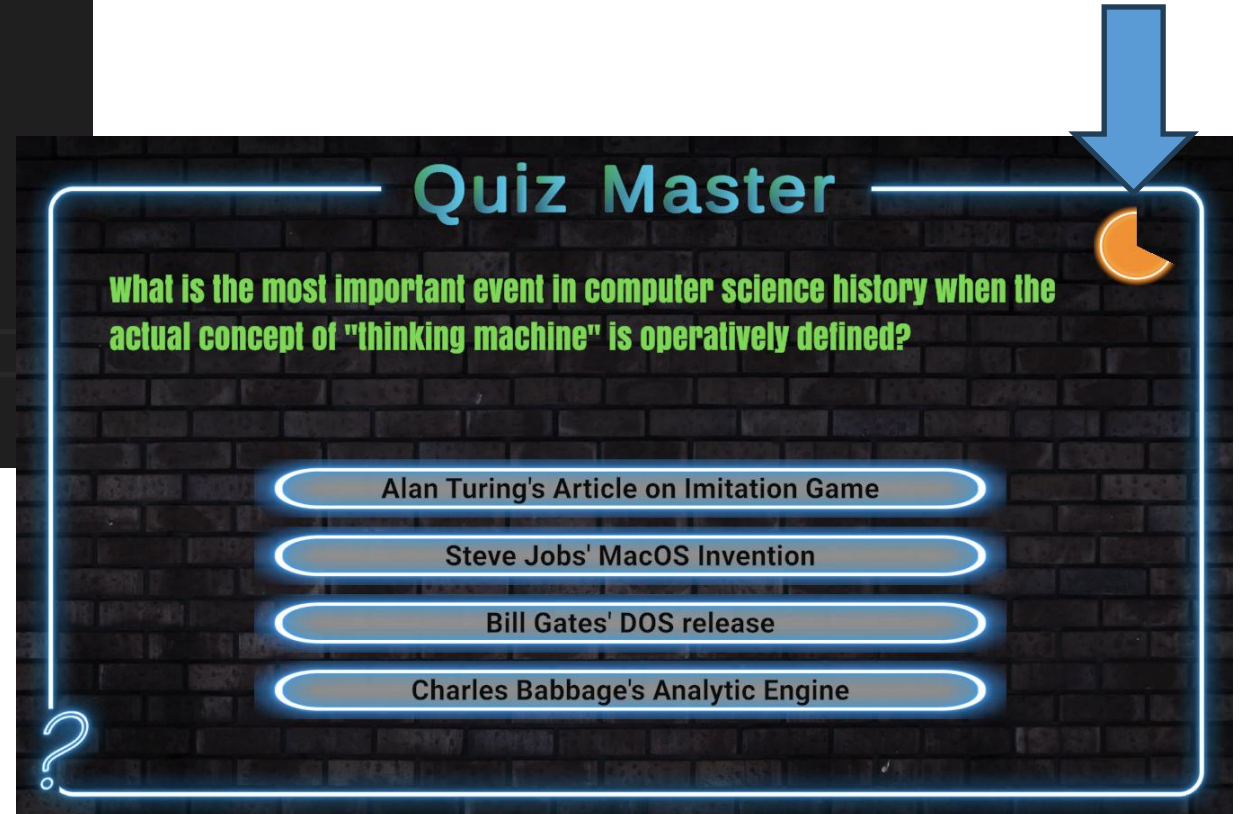
[Header("Timer")]
0 references
[SerializeField] Image timerImage;
0 references
Timer timer;

[Header("Button Colors")]
1 reference
[SerializeField] Sprite defaultAnswerSprite;
1 reference
[SerializeField] Sprite correctAnswerSprite;
```

Quiz와 Timer 스크립트 연결

```
0 references
void Start()
{
    //DisplayQuestion();
    GetNextQuestion();
    timer = FindFirstObjectByType<Timer>();
}

0 references
void Update()
{
    timerImage.fillAmount = timer.fillFraction;
}
```



일찍 답을 했을 때

```
public void OnAnswerSelected(int index)
{
    correctAnswerIndex = question.GetCorrectAnswerIndex();
    Image buttonImage;

    if(index == correctAnswerIndex)
    {
        questionText.text = "Correct!";
        buttonImage = answerButtons[index].GetComponent<Image>();
    }
    else {
        string correctAnswer = question.GetAnswer(correctAnswerIndex);
        questionText.text = "Sorry, the Correct Answer is: \n" + correctAnswer;
        buttonImage = answerButtons[correctAnswerIndex].GetComponent<Image>();
    }
    buttonImage.sprite = correctAnswerSprite;

    timer.process_Answer(); // Answer is given, the timer should be changed

    SetButtonState(false);
}
```

답이 주어졌으므로
정답을 확인하는 단계로
타이머 전환



Timer의 변화

```
public void UpdateTimer()
{
    timerValue -= Time.deltaTime;

    if(isAnsweringQuestion)
    {
        if(timerValue > 0 )
        {
            fillFraction = timerValue / timeToCompleetQuestion;
        }
        else
        {
            process_Answer();
        }
    }
}
```

2 references

```
public void process_Answer()
{
    isAnsweringQuestion = false;
    timerValue = timeToShowCorrectAnswer;
}
```

정답 보여주기가 끝나면 문제 다시 로드 - Quiz.cs

```
void Start()
{
    //DisplayQuestion();
    GetNextQuestion();
    timer = FindFirstObjectByType<Timer>();
    timer.loadNextQuestion = false;
}
0 references
void Update()
{
    timerImage.fillAmount = timer.fillFraction;
    if (timer.loadNextQuestion)
    {
        GetNextQuestion();
        timer.loadNextQuestion = false;
    }
}
```

같은 문제만
반복해서 보이게 됨
→ 문제를 바꾸기

List 사용하기

배열

→ `int[] oddNumbers = new int[5]`

리스트

→ `List<int> oddNumbers = new List<int> ()`

List를 활용한 다중 질문 관리

배열 vs. List

배열	List
크기 고정	동적 크기
arr[i]	list[i]
arr.Length	list.Count
삭제 불편	.Remove() 간편

랜덤 질문 선택 알고리즘

```
1 void GetRandomQuestion()  
2 {  
3     // 0 ~ Count-1 범위 랜덤 인덱스  
4     int index = Random.Range(  
5         0, questions.Count);  
6     question = questions[index];  
7     // 이미 나온 질문 제거 중복( 방지)  
8     questions.Remove(question);  
9 }
```

첫 시작 버그 해결

문제: Start()에서 GetNextQuestion()을 바로 호출하면 Update()에서도 실행되어 두 문제가 한 번에 로드됨

해결: Start()에서는 타이머의 loadNextQuestion = true만 설정하고, 실제 로드는 Update()에서 처리

Inspector에서 Questions 리스트 설정

QuizCanvas의 Quiz 컴포넌트 → Questions 리스트에 생성한 QuestionSO Asset들을 드래그

List의 유용한 메소드

아이템 개수 체크: `List.Count`

특정 아이템 존재 확인: `List.Contains(item)`

아이템 추가: `List.Add(item)`

아이템 삭제: `List.Remove(item)`

특정 위치 아이템 삭제: `List.RemoveAt(idx)`

리스트 모두 삭제: `List.Clear()`

진행 바 (Progress Bar) – Slider

Slider 생성

Canvas 우클릭 → UI → Slider

Slider 컴포넌트 설정:

- Min Value: 0
- Max Value: 질문 개수
- Whole Numbers: 체크
- Interactable: 체크 해제
(사용자가 조작 못 하게)

Slider Hierarchy

▽ Slider

Background -- 배경 이미지

▽ Fill Area

Fill -- 진행 채우기 이미지

▽ Handle Slide Area

Handle -- 핸들 이미지

각 자식 오브젝트의 Image에
스프라이트를 설정하여 커스터마이징

Slider 코드 연동

```
1 // Start()
2 progressBar.maxValue = questions.Count;
3 progressBar.value = 0;
4
5 // GetNextQuestion()
6 progressBar.value++;
```

여러 질문에서 하나를 다루게 변경

여러 질문은 리스트로 담고, SerializeField로 설정

```
public class Quiz : MonoBehaviour
{
    [Header("Questions")]
    3 references
    [SerializeField] TextMeshProUGUI questionText;
    0 references
    [SerializeField] List<QuestionSO> questions = new List<QuestionSO>();
    4 references
    QuestionSO question;
```

다음 질문을 랜덤하게 가져오기

리스트에서 선택

```
void GetNextQuestion()
{
    SetButtonState(true);
    SetDefaultButtonSprites();
    GetRandomQuestion();
    DisplayQuestion();
}
```

1 reference

```
void GetRandomQuestion()
{
    int index = Random.Range(0, questions.Count);
    question = questions[index];
    questions.Remove(question);
}
```

Questions

Question Text

T Quiz Text (Text Mesh Pro UGUI) ⦿

▼ Questions

0

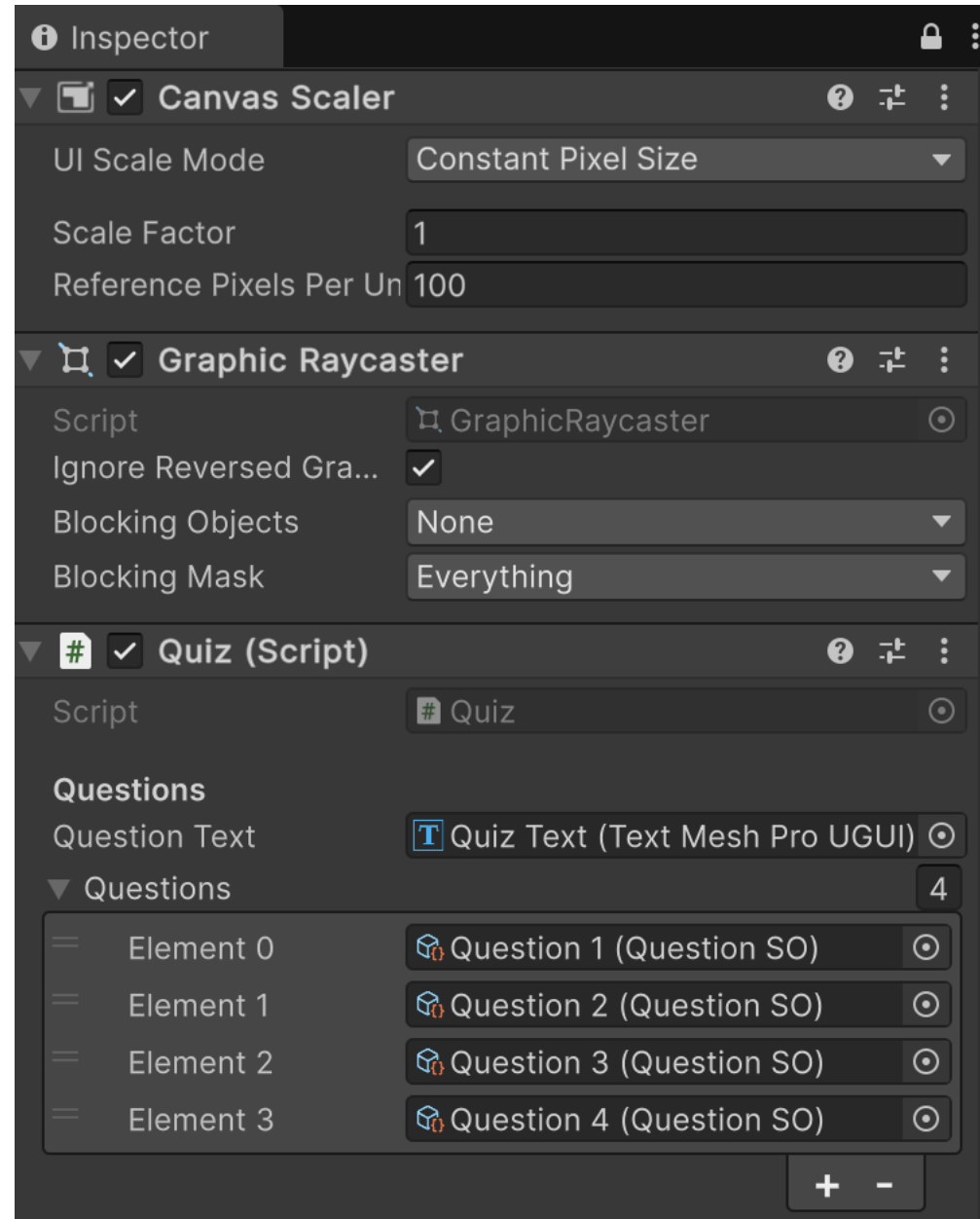
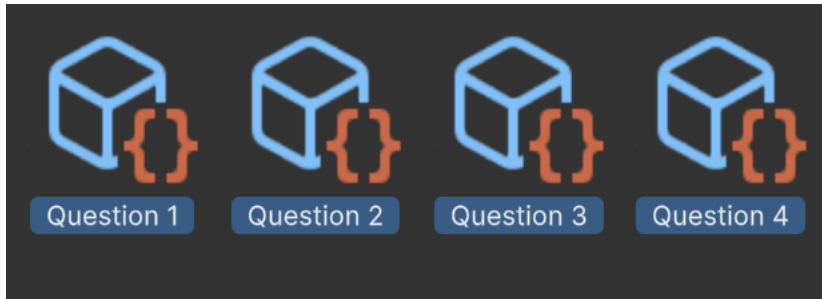
List is empty

+

-

현재 질문 리스트는 비어
있음

리스트 채우기

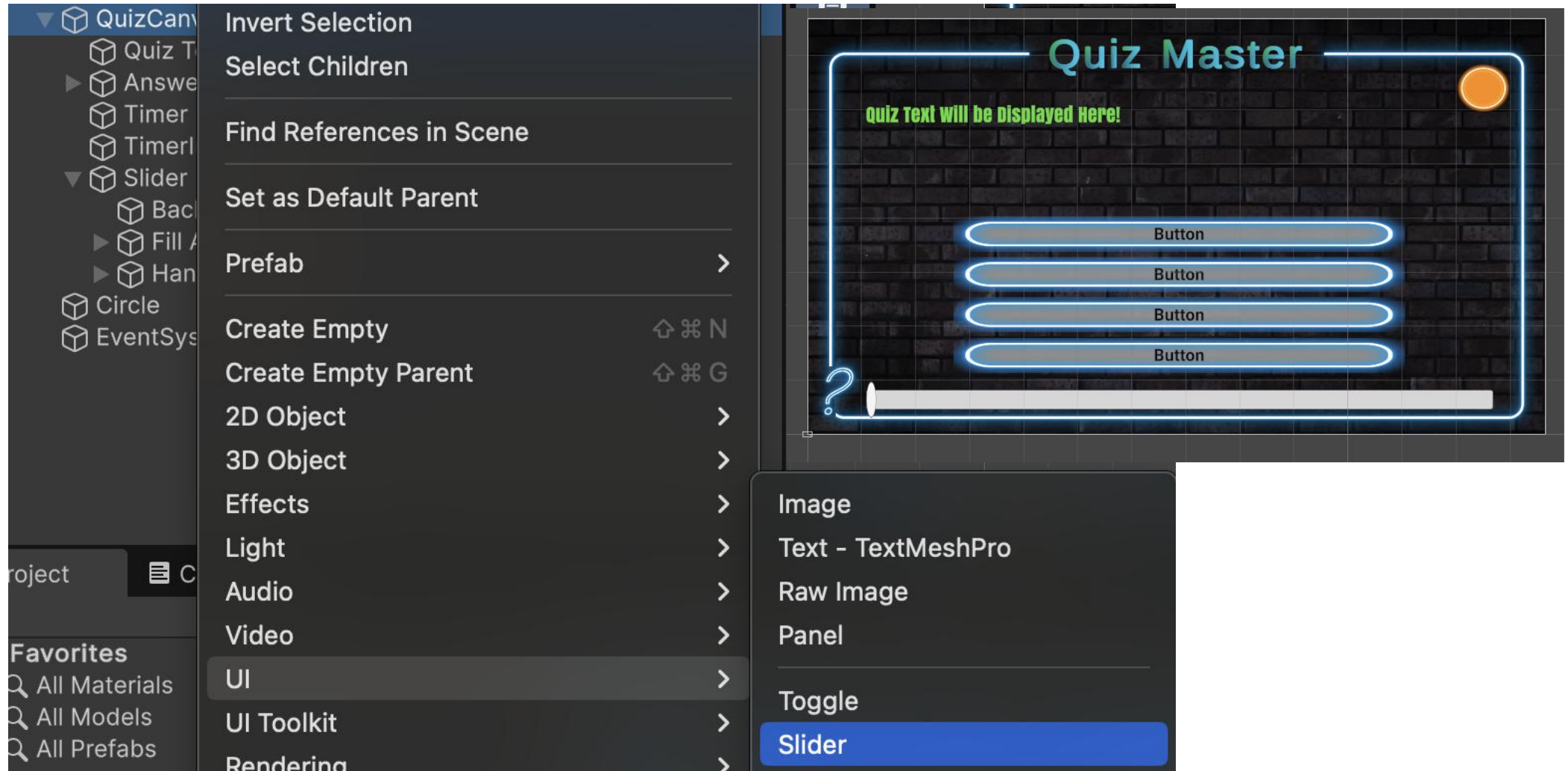


첫 시작에서 두 문제 가져오는 버그 해결

문제를 처음에 가져온 뒤
바로 Update에서 가져오는
문제 해결

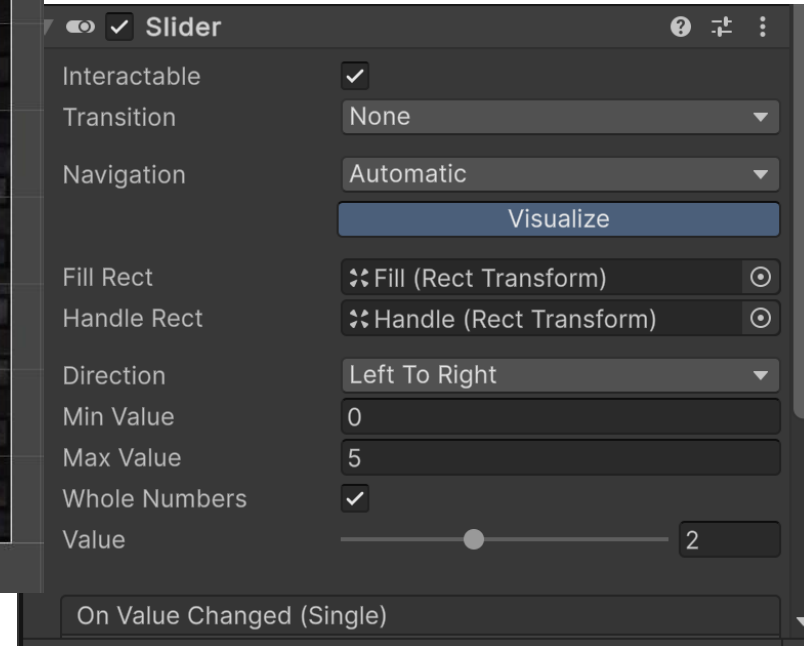
```
void Start()
{
    //DisplayQuestion();
    //GetNextQuestion();
    timer = FindFirstObjectByType<Timer>();
    timer.loadNextQuestion = true;
}
0 references
void Update()
{
    timerImage.fillAmount = timer.fillFraction;
    if (timer.loadNextQuestion)
    {
        GetNextQuestion();
        timer.loadNextQuestion = false;
    }
}
```

진행상황 보여주기 – Progress Bar



슬라이더 각 요소에 스프라이트 설정

- ▼  Slider
 -  Background
- ▼  Fill Area
 -  Fill
- ▼  Handle Slide Area
 -  Handle



Quiz.cs 고쳐서 progressBar를 인스펙터에서 지정

```
[Header("Progress Bar")]  
0 references  
[SerializeField] Slider progressBar;
```

Progress Bar

Progress Bar

Slider (Slider)

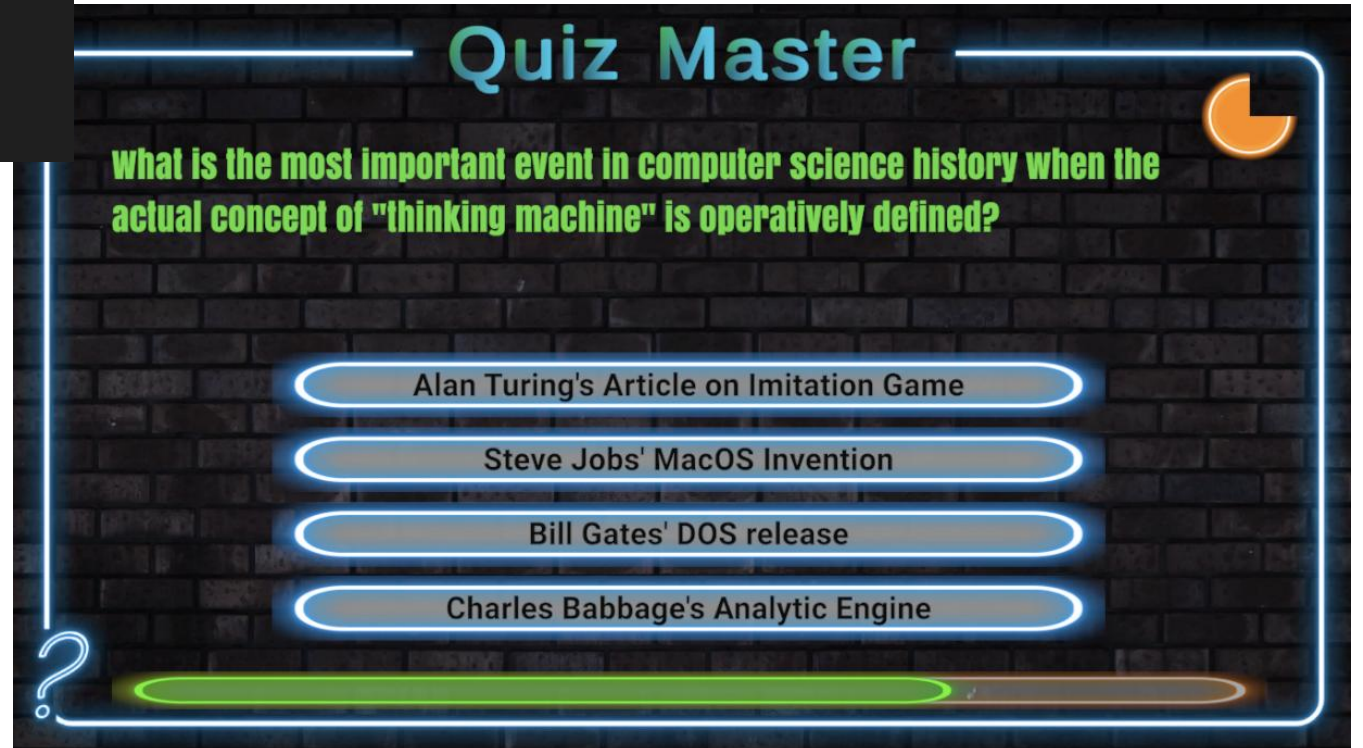
코드 수정하여 슬라이더 동작 완성

```
void Start()
{
    //DisplayQuestion();
    //GetNextQuestion();
    timer = FindFirstObjectByType<Timer>();
    timer.loadNextQuestion = true;

    progressBar.maxValue = questions.Count;
    progressBar.value = 0;
}
```

```
void GetNextQuestion()
{
    if (questions.Count > 0)
    {
        SetButtonState(true);
        SetDefaultButtonSprites();
        GetRandomQuestion();
        DisplayQuestion();
        progressBar.value++;
    }
}
```

문제점: 슬라이더를 조작할 수 있다
해결법: Interactable flag



도전 과제

도전 과제 1 – 점수 출력

화면에 점수를 출력하자

- TMP Text로 Score 표시 UI 추가
- 정답 선택 시 점수 증가
- Score: 맞춘 수 / 전체 수

도전 과제 2 – 게임 종료 화면

모든 문제를 풀면 최종 점수를
출력하고,
다시 도전할지를 묻는 버튼을 달아
보자

- `questions.Count == 0` 감지
- 결과 패널 활성화
- "다시 시작" 버튼: SceneManager로 씬 재로드

힌트 – 씬 재로드

```
using UnityEngine.SceneManagement;  
SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
```

전체 오브젝트 구조 및 스크립트 연결

Hierarchy 전체 구조

```
SampleScene
├── Main Camera
├── Global Light 2D
├── ▾ PrevisualizationCanvas
│   └── Image (배경 미리보기)
├── ▾ BackgroundCanvas (Sort:0)
│   ├── Image (배경 스프라이트)
│   ├── Text TMP (타이틀)
│   └── ▾ QuizCanvas (Sort:1, Quiz.cs)
│       ├── Quiz Text (TMP)
│       ├── ▾ AnswerButtonGroup (VLG)
│       │   └── Answer Button (0~3)
│       ├── TimerImage (Filled)
│       ├── Timer (Timer.cs)
│       └── Slider (Progress Bar)
└── EventSystem
```

스크립트 요약

스크립트	역할
QuestionSO.cs	질문 데이터 (SO)
Quiz.cs	문제 표시, 정답 확인, UI 업데이트
Timer.cs	타이머 카운트, 상태 전환

Assets 폴더 구조

```
Assets/
├── Scripts/ -- Quiz.cs, Timer.cs, QuestionSO.cs
├── Questions/ -- Question1~4.asset
├── TextMesh Pro/
└── Settings/
```

학습 내용 요약

이번 강의에서 배운 것

- **Canvas** – UI의 루트 오브젝트
- **Rect Transform** – UI 레이아웃
- **Anchor/Pivot** – 반응형 UI
- **Image Types** – Simple/Sliced/Filled
- **TextMesh Pro** – 고품질 텍스트
- **Button + OnClick** 이벤트
- **Vertical Layout Group**
- **Sort Order** – 캔버스 레이어
- **ScriptableObject** – 데이터 분리
- **List<T>** – 동적 컬렉션
- **Slider** – 진행 바
- **Filled Image** – 원형 타이머

핵심 설계 패턴

- ① **데이터 분리**: ScriptableObject로 데이터와 로직을 분리
- ② **상태 관리**: Timer가 게임 상태를 관리하고 Quiz에 신호 전달
- ③ **캡슐화**: private + getter 패턴
- ④ **UI 반응**: Update()에서 매 프레임 UI 상태를 동기화

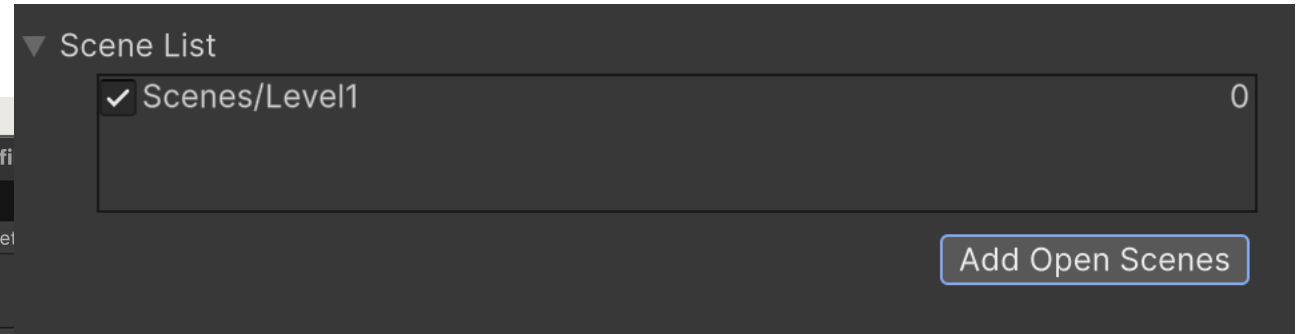
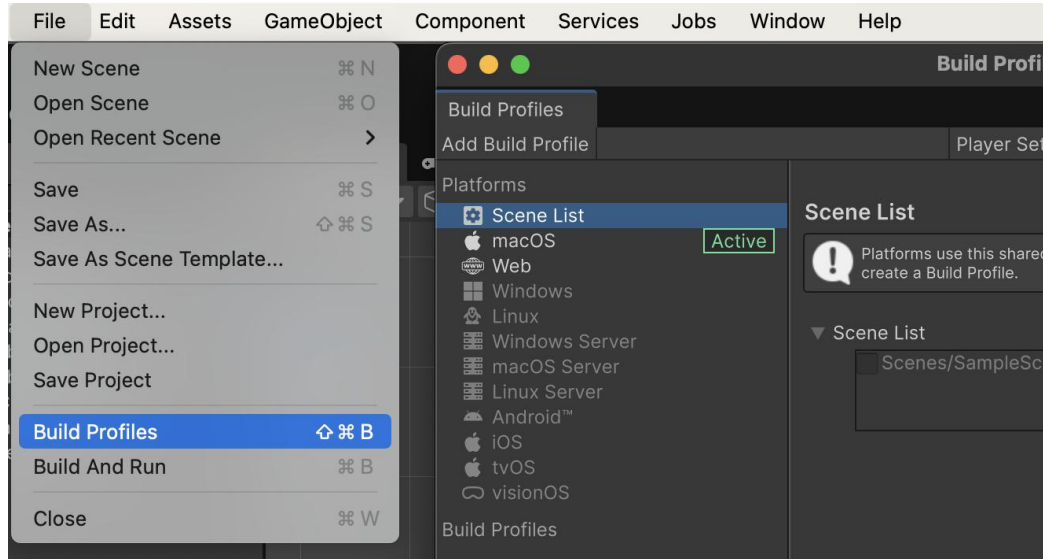
다음 강의 예고

이제 UI를 이용해 게임을 만들 수 있나요?
도전 과제를 완성해 보세요!

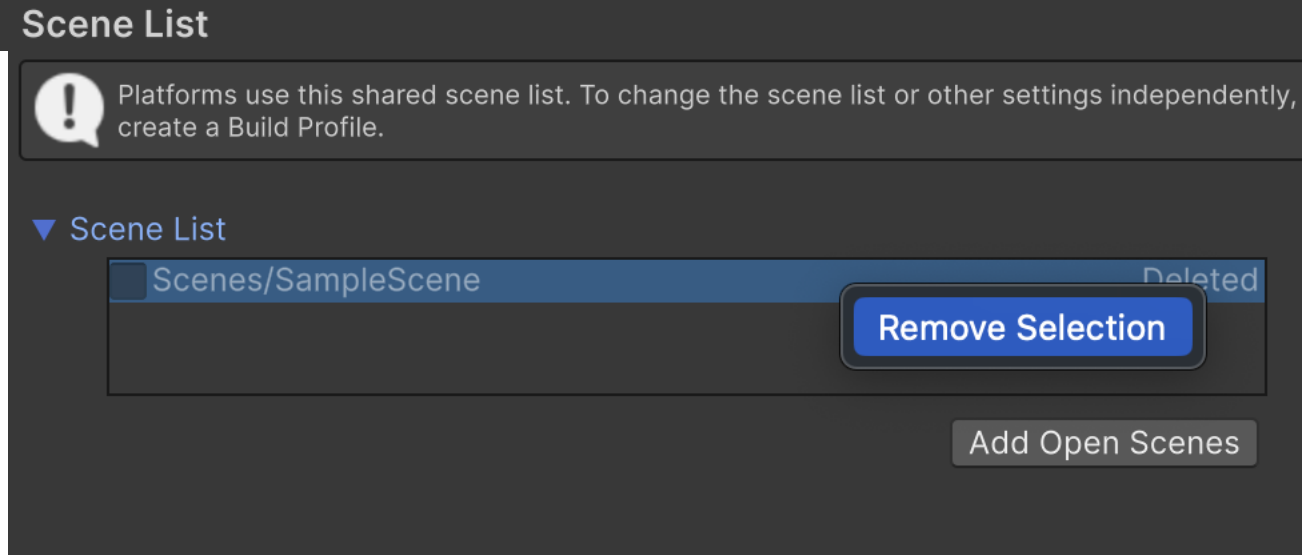


이제 U를 이용해
게임을 만들 수 있나요?

Build Profile 다시 확인



SampleScene0이 회색으로 표시되고 지워졌다 나타남



Remove하자